

Matematine Tehnologije

ANTONIO MONTERO

January 14, 2026

Contents

0.1 LaTeX

0.1.1 Uvod v LaTeX

Kaj je LaTeX?

LaTeX je program za oblikovanje besedil. LaTeX je nadgradnja programa TeX, ki je razvil Donald E. Knuth. leta 1997. Prvo verzijo LaTeX-a je napisal Leslie Lamport (1980). LaTeX je paket dodatnih ukazov za TeX, ki avtorjem omogoajo oblikovanje in izpisovanje dokumentov v najvišji kakovosti s pomočjo e pripravljenih profesionalnih vzorcev.

Trenutna standardna verzija je LaTeX2e (1994), zanj pa je odgovoren Frank Mittelbach. LaTeX se izgovori 'lâteh', po angleško pa obiajno reemo 'lejteh'.

Ko eli *avtor* kaj objaviti, polje svoj rokopis zaloniku. Tam *knjii opremljevalec* dolo obliko dokumenta (irino stolpcev, obliko rk, velikost presledkov pred in po naslovih ...). Opremljevalec svoja navodila skupaj z rokopisom preda *rkostavcu*, ki na podlagi teh podatkov stavi knjigo. V LaTeX-u prevzame vloži knjinega oblikovalca in rkostavca, da se avtor osredotoi na vsebino Figure ??.

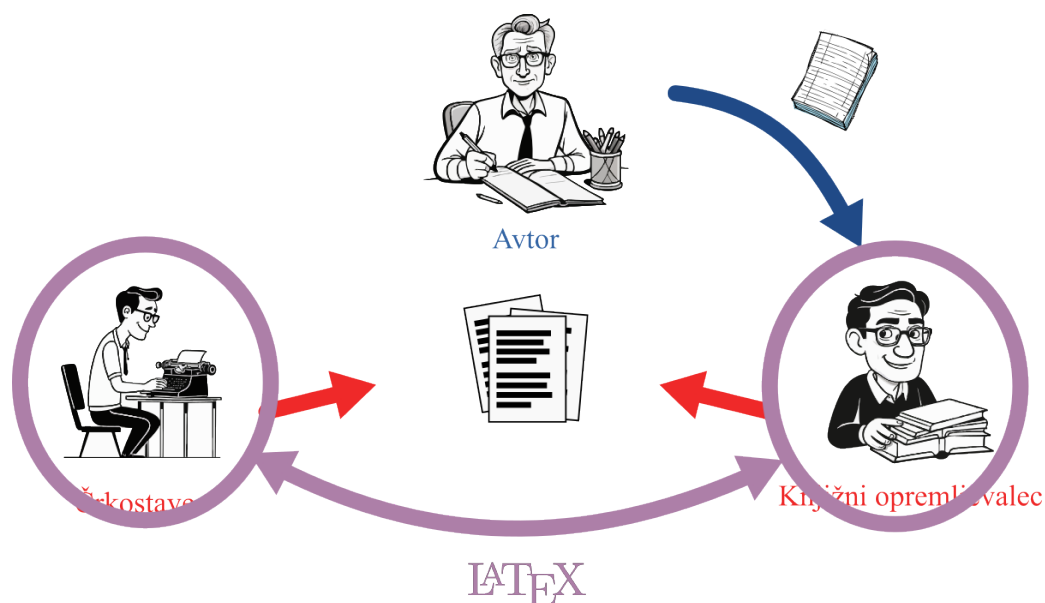


Figure 1: Kaj je LaTeX?

Pomebno: Filozofija LaTeXa

Oblikovanje dokumentov z LaTeXom sledi naslednjemu naelu:

mi (avtorji) se ukvarjamo z **vsebino** dokumenta, LaTeX pa poskrbi za **oblikovanje**.

Ali, enakovredno,

ne skrbimo za **oblikovanje**, zato se lahko osredotoimo na **vsebino**.

Kaj zmore LaTeX?

LaTeX zmore skoraj vse, kar zmorejo drugi urejevalniki besedil. Vendar pa je LaTeX posebej primeren za pisanje znanstvenih in tehničnih besedil, ki vsebujejo veliko matematičnih formul in simbolov. LaTeX omogoča tudi enostavno ustvarjanje tabel, grafov in drugih elementov, ki so pogosto potrebni v tehničnih besedilih. Poleg tega LaTeX omogoča enostavno ustvarjanje kazal, bibliografij in drugih elementov, ki so pomembni za znanstvena besedila.

eprav je LaTeX optimiziran za izdelavo znanstvenih in tehničnih besedil, ni omejen samo na to. Danes uporabljam LaTeX za pisanje vseh dokumentov, ki naj izgledajo profesionalno ali lepo. To sega od osebnih in poslovnih pisem do pesmarice za igranje kitare.

Vpraal sem moje sodelavce, kaj delajo z LaTeXom. Tukaj je nekaj zanimivih odgovorov:

- Knjnige, lanke, in zapiski predavanj;
- Predstavitve, s pomojo Beamer-ja [Beamer: Prestavitve v LaTeXu](#);
- Profesionalna dokumentacija (npr CV, projekte vloge);
- Pogodbe za stanovanje;
- Pisma;
- Pravila za druabne igre
- Note (za glasbo)
- Facebook, SMS, Instagram (da, res je!).
- **Zakljuna dela (diplome, magistrska, doktorat)**

e vas zanima, si lahko tukaj ogledate mojo doktorsko disertacijo, ki je bila napisana izključno z uporabo programa LaTeX.

LaTeX ni Word

MS Word, Google Docs, LibreOffice, so WYSIWYG (kar vidi, to tudi dobi). V teh programih avtor obliko dokumenta dolo interaktivno med samim vnosom besedila v računalnik. Med celotnim procesom na zaslonu vidi, kakna bo konna oblika, ko bo dokument natisnjen.

Pri LaTeXu ni mono videti konne oblike med samim tipkanjem besedila. Lahko pa vidimo konno obliko na zaslonu, potem ko datoteko prevedemo z LaTeXom. Tako lahko popravke naredimo e preden dokument izpiemo s tiskalnikom. V urejevalnikih WYSIWYG strukturo in obliko urejate hkrati z pisanjem besedila. Zlasti pri dolgih projektih se je zelo lahko izgubiti. LaTeX sili avtorja, da dolo logino strukturo besedila, LaTeX pa potem izbere najprimernejo obliko.

Zakaj LaTeX? Predstavljajmo si, da piete roman v svojem najljubem urejevalniku WYSIWYG. V tej knjigi se odvijata dve vzporedni zgodbi v razlinih dimenzijah. Da bi bralcu sporoili, katera od dimenzij je trenutno opisana, ste se odloili, da boste za obe uporabili razline barve. Vaa knjiga bi tako lahko izgledala takole:

Ne, ne more me zapustiti;‘ je zaklical Peredur Lancelotu.

Moram,“ je odgovoril. Usoda me klie. Toda ponovno se bova sreala. Obljubim.“

Shao je obutila nepojasnjeno alost, kot da bi pravkar izgubila nekaj ali nekoga, ki ji je bil pomemben.

Si v redu;‘ je vpraala Ashby, medtem ko je jedla svojo kao.

Ne izgleda dobro.“

V redu sem, samo utrujena.“

Ko ste konali prvi osnutek (ki obsega 300 strani), ste se odloili, da ga poljete po e-poti prijatelju, da vam bo povedal svoje mnenje. Vendar se je izkazalo, da lahko tiskalnik vaega prijatelja tiska samo rno-belo, zato ne more natisniti datoteke, ki ste mu jo poslali.

Enostavna reitev! Vse zeleno besedilo spremenite v kursivno pisavo, ostalo pa pustite v pisavi upright front. Po roni spremembi vsakega odstavka prvih 120 strani knjige ste se spomnili, da v nekaterih primerih za poudarjanje uporabljate kursivno pisavo: [Ali bo *to* jedla?]{.latex-line .text-green} Sedaj boste morali preveriti vsak odstavek za kurzivo in ga spremeniti v drug slog, preden spremenite vse zeleno besedilo v kurzivo.

Vidite neskonni problem?.

LaTeX reuje ta problem tako, da avtorju omogoaa, da se osredotoi na vsebino in logino strukturo besedila, medtem ko LaTeX poskrbi za oblikovanje.

Kako deluje LaTeX?

Klasina razlaga delovanja LaTeX-a je naslednja:

1. Avtor napie besedilo in ukaze LaTeX v datoteko `.tex` (npr. `ime_datoteke.tex`).
2. Datoteka `.tex` prevedemo z LaTeX-om, ki ustvari datoteko `.dvi` (npr. `ime_datoteke.dvi`).
3. Datoteke `.dvi` niso standardne, zato je treba datoteka `.dvi` pogosto spremeniti v datoteko `.pdf`; na koncu dobimo datoteko `ime_datoteke.pdf`, ki jo lahko natisnemo ali delimo z drugimi.

V sodobnem asu pogosto gremo iz `ime_datoteke.tex` datoteke neposredno v `ime_datoteke.pdf` npr. z pomojo programa `pdflatex`.

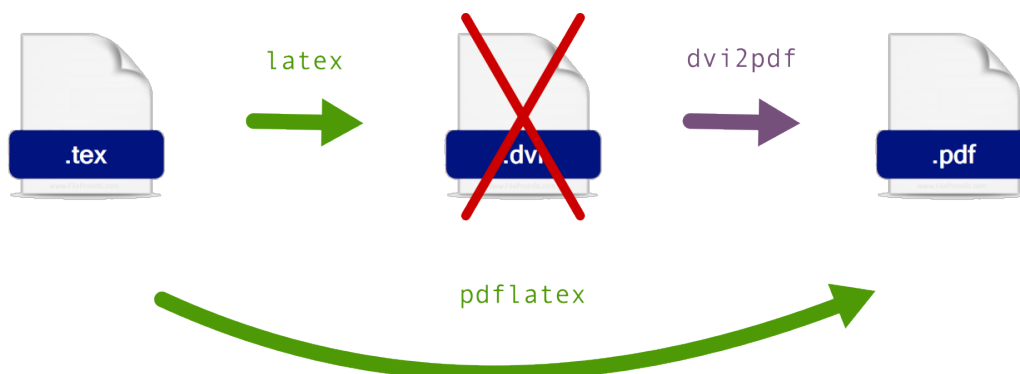


Figure 2: Kajo deluje LaTeX?

Kako zaeti z LaTeXom?

Lahko namestite na svoj raunalnik. e vas to elite, pogledjte [Nameanje LaTeXa na svoj raunalnik](#).

Uporabili bomo drugaen pristop in delali z LaTeXom na spletu. To bomo storili prek Overleaf, ki je platforma za delo z LaTeX projekti na spletu. Najprej morate ustvariti raun.

Ukazi in okolja

Kot smo e omenili, se moramo pri delu z LaTeXom osredotoiti na vsebino, oblikovanje pa prepustiti LaTeXu. Ker pa je LaTeX “samo” raunalniki program, potrebuje ve vodenja. Avtor mora navesti dodatne informacije, ki opisujejo logino strukturo njegovega dela. Te informacije se navedejo prek *ukazov* in *okolij*, ki se vpiejo okoli besedila.

Ukazi v LaTeXu imajo splono obliko:

```
\imeUkaza[neobvezni argument]{argument}
```

in se obiajno uporablja za dajanje LaTeXu doloenih navodil.

- Ukaz *vedno* se zane z znakom `\`.
- Argumentov je lahko ve, npr. `{prvi}{drugi}...{zadnji}` ali nobenega (npr. ukaz `\tiny`, vendar je to redko).
- Lahko je ve neobveznih argumentov: `[prvi, drugi, ..., zadnji]` ali noben (in potem ne napiemo nesar). Pazite razliko z obveznimi `argumenti`.

Okolje je posebna vrsta ukaza z naslednjo strukturo:

```
\begin{okolje}
Vsebina okolja: ukazi in besedilo
\end{okolje}
```

Okolja obiajno nadzorujejo, kako se bo del besedila/kode obnaal, ko bo natisnjen v dokument.

Zaenkrat vam ni treba skrbeti, kako delujejo ukazi in okolja. To bomo spoznavali sproti. V poglavju [Osnovna okolja](#) bomo podrobneje pogledali nekatera najpomembnejja okolja.

Table 1: Razredi LaTeX dokumentov

Razred	Opis
<code>article</code>	za lanke v znanstvenih revijah, kratka poroila in druge kraje dokumente.
<code>proc</code>	razred za zbornike, ki temelji na razredu <i>article</i> .
<code>report</code>	za dalja poroila, ki vsebujejo ve poglavij, manje knjige, doktorske disertacije, ...
<code>book</code>	za prave knjige.
<code>beamer</code>	za predstavitev.
<code>letter</code>	za pisma.

Preambula in telo datoteke

Vsaka datoteka LaTeX ima naslednjo obliko:

```
%Preambula
\documentclass{...}
\usepackage{...}
%...
% Telo
\begin{document}
  Vsebina dokumenta.
\end{document}
```

Najpomembnejše okolje v latexu je okolje `document`. To okolje se lahko uporabi najve enkrat na dokument in v tem okolju pa napiemo dejansko vsebino našega dokumenta. Vse, kar je znotraj tega okolja, se imenuje *telo* dokumenta.

e je najpomembnejše okolje `document`, mora biti najpomembnejši ukaz `\documentclass`. Vse, kar je med ukazom `\documentclass` in začetkom okolja `document`, se imenuje *preambula* dokumenta. Tukaj nadzorujemo splono delovanje LaTeXa.

Opozirilo!

Preambula obiajno vsebuje samo ukaze (v nasprotju z obiajnim besedilom) in obstajajo ukazi, ki se lahko uporabljajo samo v preambuli. Pogosta napaka v LaTeXu je, da se del kode napie na napano mesto.

Ukaz `\documentclass` doloa vrsto dokumenta, ki ga elimo ustvariti. Uporabljamo ga tako

```
\documentclass[monosti]{razred}
```

Tu `razred` doloa vrsto dokumenta, ki ga elimo ustvariti (npr. lanek, knjiga, poroilo ...), `monosti` pa so dodatne nastavitve, ki vplivajo na videz dokumenta (npr. velikost pisave, velikost papirja ...). V Table ?? so prikazani razlini razredi dokumentov v LaTeXu.

V tem ubeniku bomo obravnavali le razreda `article` in `beamer` (glej [Beamer: Predstavitev v LaTeXu](#), vendar bo veio tega, kar se bomo nauili, mogoe brez teav uporabiti tudi v drugih razredih.

Nekatere od uporabnih `monosti` so `10pt` (prim. `11pt`, `12pt`), `a4paper` (prim. `letterpaper`), `onecolumn` (prim. `twocolumn`). Privzete `monosti` za `article` razred so `[letterpaper, 10pt, oneside, onecolumn, final]`.

Avtor, naslov in datum

Z ukazi `\title`, `\author` in `\date` lahko doloimo naslov, avtorja in datum dokumenta. Ti ukazi se obiajno uporabljajo v preambuli, vendar jih lahko uporabimo tudi v telesu dokumenta. Ti ukazi ne

ustvarijo nesar, dokler ne uporabimo ukaza `\maketitle`, ki ustvari naslovno stran z informacijami, ki smo jih določili z zgornjimi ukazi. Poglejte Example ??.

Example 0.1.1. % Preambula

```
% ...
\author{France Preeren}
\title{Zdravljica}
\date{26.04.1848}
% ...
% Telo
\begin{document}
\maketitle
V tem dokumentu bom napisal ljubezensko pismo Juliji.
\end{document}
```

Paketi in jeziki

LaTeX je zelo prilagodljiv in razirljiv sistem. Osnovni LaTeX ponuja le osnovne funkcije, vendar lahko z uporabo *paketov* dodamo nove funkcije in monosti. LaTeX ima okoli 6000 paketov na voljo. Paketi so zbirke ukazov in okolij, ki jih lahko vključimo v naš dokument z ukazom

```
\usepackage{ime_paketa}.
```

Opozorilo!

Ukaz `\usepackage{ime_paketa}` mora biti v preambuli, torej pred `\begin{document}`.

Eden najpomembnejših paketov je `babel`, ki omogoča uporabo različnih jezikov. Paket naložimo z ukazom

```
\usepackage[jezik]{babel}
```

kjer `jezik` določa jezik, ki ga elimo uporabiti (npr. `slovene`, `english`, `spanish`, `french`, `german`, `italian`, ...). Paket `babel` poskrbi za pravilno oblikovanje besedila glede na izbrani jezik (npr. vezaje, loila, datum, ...).

Opomba!

- V starih različicah LaTeX-a nismo mogli preprosto pisati `, , , á, ü ...`
- Te rke lahko vedno dobimo z ukazi, kot so:
- Alternativno lahko uporabimo naslednje pakete za kodiranje znakov:

```
% Vhodnega dokumenta,
\usepackage[utf8x]{inputenc}
% opcije lahko so [utf8], [utf8x], ali [cp1215]

% izhodnega dokumenta,
\usepackage[T1]{fontenc}
% lahko damo tudi [T2A] za cirilica
```

- Sodobni LaTeX običajno ne potrebuje teh paketov. Ponavadi je dovolj e uporabimo paket `babel`.

Table 2: Posebni znaki v LaTeXu

Znak	Ukaz
%	\%
\$	\\$
&	\&
#	\#
_	_
{ }	\{ \} (vedno v paru)
~	\textasciitilde
^	\textasciicircum
\	\textbackslash

Komentarji, posebni znaki, prazne vrstice in presledki.

Nasvet!

Pri delu v LaTeXu besedila **ne briemo**, temve ga **komentiramo**.

e elimo izpizati simbol %, zapiemo \%. Simbol % je v LaTeXu poseben znak, to pomeni, da ima poseben pomen in ga ne moremo uporabiti kar tako. Drugi posebni znaki v LaTeXu so prikazni v Table ??.

Pogosta napaka!

Pogosta napaka v LaTeXu je, da se pozabi uporabiti ukaz za enega od posebnih simbolov.

Vežaj vs pomiljaj

Vežaj v LaTeXu dobimo z enim samim znakom -. Pogoste uporabe vežaja so:

- sestavljene besede, npr. `informacijsko-komunikacijska tehnologija`: `[informacijsko– komunikacijska tehnologija]{.latex-line}`;
- sklanjanje kratic, npr. v NUK-u: v NUK–u.

Pomiljaj v LaTeXu dobimo z dvema znakama -. Pogoste uporabe pomiljaja so:

- za označevanje intervalov, npr. `strani 10–20`: strani 10—20;
- loevanje besedila, npr. za pojasnila v stavku, npr. `LaTeX – program za oblikovanje besedil` – je zelo prilagodljiv: `LaTeX — program za oblikovanje besedil — je zelo prilagodljiv`.

Narekovaji

LaTeX ima posebna pravila za tiskanje narekovajev:

- Enojne narekovaje dobimo z uporabo znakov ‘ in ’, npr. `‘narekovaj’`: ‘narekovaj’.
- Dvojne narekovaje dobimo z uporabo znakov " in ", npr. `"narekovaji"`: “narekovaji”.
- Za slovenske narekovaje uporabimo ukaza `\glqq` in `\grqq`, npr. `\glqq slovenski narekovaji\grqq`: slovenski narekovaji“.
- e uporabimo `\usepackage[slovene]{babel}`, lahko preprosto napiemo “‘ in ””, npr. `"‘slovenski narekovaji"’`: slovenski narekovaji“.
- Lahko uporabimo tudi "> in "<, npr. `>slovenski narekovaji<`: »slovenski narekovaji« (potreben je tudi paket `babel`).

Pogosta napaka!

Pogosta napaka v LaTeXu je, da uporabljamo naravne narekovaje (" in ') ali pa napano uporabljamo ukaze za narekovaje, npr.

- "narekovaji"
- "narekovaji"

Presledki

- Poljubnotevilo presledkov (ali tabulatorjev) med besedami se obravnava kot en sam presledek.
- Presledek na zaetku vrstice se ignorira.
- Nova vrstica se obravnava kot presledek, e ni prazna.
- Prazna vrstica se obravnava kot konec odstavka in zane nov odstavek.
- Zaporedne prazne vrstice se obravnavajo kot ena sama prazna vrstica.
- Prazen komentar % ni prazna vrstica.

Example 0.1.2.

Nasvet!

Ker se nova vrstica v LaTeXu obravnava kot presledek, razen e je prazna, je dobro, da vsak stavek napiete v eno vrstico. To olaja urejanje besedila in sledenje spremembam v sistemih za nadzor razlic, kot je Git.

Vasih resnino potrebujemo prelom vrstice, ne da bi zaeli nov odstavek. Za to imamo na voljo naslednji dve ukazi.

- Ukaz `\` ustvari prelom vrstice.
- Ukaz `\newline` ustvari prelom vrstice in ignorira vse presledke na zaetku naslednje vrstice.
- Ukaz `\linebreak[arg]` (*arg* jetevilo od 0 do 4, privzeta vrednost je 4.): 0 pomeni, da je prelom vrstice nizke prioritete in se lahko, zaradi drugih okoliin, ignorira, 4 pa pomeni, da je prioritete preloma vrstice visoka.
- Ukaz `\noindent` preprei zamik prve vrstice novega odstavka.

Example 0.1.3.

LaTeX ima tudi ukaze za obravnavanje prelomov strani.

- Ukaz `\pagebreak[arg]` je najbolj primitivni med njimi. Preprosto prekine stran na mestu, kjer se pojavi. Argument *arg* deluje enako kot pri `\linebreak`.
- Ukaz `\newpage` prekine stran in zane novo stran. Prav tako zapolni preostali prostor na strani s praznim prostorom.
- Ukaz `\clearpage` deluje podobno kot `\newpage`, vendar poleg tega tudi izpie vse plavajoe elemente (npr. slike in tabele), ki e niso bili izpisani.
- Ukaz `\cleardoublepage` deluje podobno kot `\clearpage`, vendar zagotavlja, da se nova stran zane na desni strani v dvostranskem tisku.

Nevarnost!

Ukazi za prelome vrstic in strani so namenjeni uporabi v izjemno specifinih okoliinah, vendar **njihova uporaba ni priporoljiva**.

Ne pozabite, da moramo oblikovanje dokumenta prepustiti LaTeXu.

Pisave in velikosti pisave

LaTeX privzeto uporablja pisavo *Computer Modern*, ki jo je zasnoval Donald Knuth. Sodobne razliice LaTeX pogosto uporabljajo Latin Modern, ki je sodobna razliica Computern Modern. e ta pisava ni privzeto naloena, jo lahko (in jo tudi morate) naloiti s ukazom `\usepackage{lmodern}` v preambuli.

LaTeX samodejno izbere pisavo in velikost rk glede na razred dokumenta in izbrane pakete. e elite spremeniti pisavo ali velikost pisave, LaTeX uporablja tirje parametri:

family (druina) Dejanska pisava, ki jo uporablja. Privzeto ima LaTeX tri druine: *roman* oz. *pokonna serifna* pisava (`\textrm`), *sans serif* oz. *gladka* pisava (`\textsf`) in *strojepisna* oz. *enakoprostorna* pisava (`\texttt`).

series (serija) Debelina pisave. Privzeto ima LaTeX dve seriji: *medium* oz. *srednja* pisava (`\textmd`) in *bold face* oz. *krepka* pisava (`\textbf`).

shape (oblika): Naklon pisave. Privzeto ima LaTeX tri oblike: *italic* oz. *poevna* pisava (`\textit`), *slanted* oz. *leea* pisava (`\textsl`) in *small caps* oz. *male velike rke* (`\textsc`). Poleg tega sta na voljo dve virtualni obliki: *pokonna* (`\textup`) in *velika-mala* (`\textulc`). To v resnici nista obliki, ampak uporabniki ukazi. Prvi preklopi nazaj na pokonno pisavo, drugi pa onemogoi male velike rke. Ukaz `\textnormal` je le kombinacija obeh.

size (velikost). To se veinoma nadzira z monostmi ukaza `\documentclass`, vendar o tem ve kasneje.

e elite spremeniti enega od prvih treh parametrov v danem besedilu, preprosto uporabite ukaze, kot je `\ukaz{dano besedilo}`. Npr. z ukazom `\textbf{To je krepko besedilo}` dobimo **[To je krepko besedilo]**`{.latex-line}` in z ukazom `\textit{To je poevno besedilo}` dobimo *[To je poevno besedilo]*`{.latex-line}`.

Pazite!

Lahko zdruimo ve ukazov, prikazanih zgoraj, (npr. `\textit{\textbf{To je krepko in poevno besedilo}}`) proizvajajo ***[To je krepko in poevno besedilo]***`{.latex-line}`; vendar niso vse kombinacije na voljo. LaTeX bo pritoeval, e boste poskusili uporabiti kombinacijo, ki ne obstaja.

Vsi zgornji ukazi imajo razliico *switch*. Namesto da bi prejeli besedilo prek argumenta, trajno spremenijo pisavo, dokler se ta ponovno ne spremeni. V Table ?? so prikazani vsi ukazi in njihove razliice *switch*.

Example 0.1.4.

Poleg zgoraj navedenih ukazov lahko ustvarimo tudi podrtano besedilo (`\underline{besedilo}`) in poudarjeno besedilo (`\emph{besedilo}`). Delovanje ukaza `\emph{besedilo}` je odvisno od konteksta. Obiajno ustvari besedilo, ki je zelo podobno besedilu, ustvarjenemu z ukazom `\textit`, vendar ni namenjeno za pisanje poevna pisava. Poglejte Example ?? za primer.

Example 0.1.5.

Za razliko od veine urejevalnikov besedil WYSIWYG (Word, itd.) LaTeX nima izrecnega naina za doloanje velikosti pisave. Dejanska velikost besedila je odvisna od monosti argumenta `\documentclass` (`10pt`, `11pt` ali `12pt`). To pomeni, da bo navadno besedilo velikosti `10pt` (oz. `11pt` ali `12pt`). Velikost pisave za druge dele besedila bo doloena ustrezno.

e resnino elimo spremeniti velikost pisave za doloen del besedila, uporabimo ukaze v Table ?. Natanneje, ti ukazi niso LaTeX ukazi, ampak TeX ukazi (to pomeni, da niso sodobni ukazi). Zaradi tega je njihova sintaksa nekoliko drugana. Delujejo kot `{\velikost <besedilo>}`, kjer *velikost* je eden od ukazov iz tabele in *besedilo* je besedilo, ki ga elimo spremeniti. Npr. z ukazom `{\Huge To je najveje besedilo, ki ga lahko ustvari LaTeX.}` dobimo stavek:

To je najveje besedilo, ki ga lahko ustvari LaTeX.

V Table ?? podajamo dejansko velikost za vsako od njih ter relativno primerjavo med razlinimi velikostmi.

Mnoge od teh ukazov imajo svojo sodobno razliico v obliki okolja. Namesto da napiete `{\small majhna pisava}`, lahko na primer napiete `\begin{small} majhna pisava \end{small}`.

Opozorilo!

To je res pomembno, zato je vredno ponoviti: Preve igranja s pisavami in velikostmi pisav je v nasprotju s filozofijo LaTeXa. Ne pozabite, da moramo LaTeXu prepustiti oblikovanje, medtem ko se **mi osredotoimo na vsebino**.

Vaje

Vsako poglavje v tej knjigi se kona s povzetkom razlinih vaj ter dodatnimi vajami ali domaimi Vajami. Tukaj je ustrezna vaja za prvo poglavje:

- Exercise ??: Ustvarjanje rauna v Overleafu.
- Exercise ??: Ustvarjanje prve LaTeX datoteke.
- Exercise ??: Dodajanje avtorja, naslova in datuma.
- Exercise ??: Uporaba paketa `babel`.
- Exercise ??: Uporaba komentarjev in posebnih znakov.
- Exercise ??: Uporaba posebnih znakov.
- Exercise ??: Uporaba vezajev, pomiljajev in narekovajev.
- Exercise ??: Uporaba razlinih pisav.
- Exercise ??: Uporaba razlinih velikosti pisav.

0.1.2 Osnovna okolja

Opomba

Vasih, ko vadimo z LaTeXom, se znajdemo v situaciji, ko moramo vstaviti lano besedilo, tj. besedilo, katerega vsebina ni pomembna. LaTeX ima za to poseben paket, in sicer `lipsum`. Ta paket nam omogoča dostop do 150 odstavkov besedila Lorem Ipsum, ki je delek besedila “*De finibus bonorum et malorum*” avtorja Cicero.

Ko pri vsakem paketu, ga morate naloiti z ukazom `\usepackage{lipsum}` v preambuli. Nato lahko vstavite lano besedilo z ukazom `\lipsum[<razpon odstavkov>][<razpon stavkov>]`, kjer sta `<razpon odstavkov>` in `<razpon stavkov>` razpon odstavkov in stavkov, ki jih elimo vstaviti.

Na primer, `\lipsum[1-3]` bo vstavilo prve tri odstavke besedila Lorem Ipsum, medtem ko bo `\lipsum[4][1]` vstavilo samo prvi stavek etrtega odstavka.

Logina struktura dokumenta

LaTeX ima na voljo posebni ukazi za logino strukturo dokumenta. Sintaksa teh ukazov je `\ime_enote{naslov}`, kjer je `ime_enote` ime loginne enote, npr. `section`, medtem ko je `naslov` besedilo naslova, ki ga elimo prikazati v dokumentu.

Dejanske logine enote, ki so na voljo, so odvisne od razreda dokumenta, npr. za razred `article` so na voljo enote `section`, `subsection`, `subsubsection`, `paragraph` in `supargraph`. Razreda `report` in `book` imata na voljo tudi ukazi `\chapter{naslov}`.

Example 0.1.6.

Vse te razrede dokumentov imajo tudi enote `part`, ki ga lahko uporabite za razdelitev dokumenta na ve delov, ne da bi to vplivalo na oteviljenje poglavij, razdelkov itd.

Razmik med odstavki, oteviljenje in velikost pisave naslovov bo samodejno nastavljal LaTeX. To pomeni, da vam ni treba rono oteviljevati naslovov in da se, e na primer, odloite, da boste dodali novo poglavje na zaetek dokumenta, vsa oteviljenje poglavij, razdelkov itd. se bo samodejno posodobilo.

Ukaz `\tableofcontents` natisne kazalo naega dokumenta, ki sledi strukturi, ki smo jo določili z zgoraj opisanimi ukazi. Upotevajte, da je treba ta ukaz uporabiti v telesu datoteke, da se kazalo natisne na ustrezno mesto.

Vsi ukazi za logino strukturo dokumenta imajo tudi “z zvezdico” različico, npr. `\section*{naslov}`, ki ustvari naslov brez oteviljenja in ne vkljuuje naslova v kazalo.

Vasih je naslov naega razdelka zelo dolg in ne elimo, da se tako natisne v kazalu. V tem primeru lahko uporabimo naslednjo sintakso, da natisnemo krajo različico naslova.

```
\section[Kratki naslov za kazalo]{Dolgi in e posebno dologoasen naslov,
ki se izpie na zaetku razdelka
ampak noro bi bilo,
e bi se tako izpisal tudi v kazalu}
```

Poravnava besedila

LaTeX privzeto poravna besedilo z obeh strani, kar pomeni, da prilagodi prostor med besedami, tako da so vse vrstice približno enake doline. V urejevalnikih WYSIWYG se ta nastavek pogosto imenuje *obojestransko poravnano*.

Vasih elimo besedilo poravnati drugače, zato ima LaTeX tri razline okolja: `flushleft`, `flushright` in `center`. Vendar je izvedba tega okolja nekoliko zastarela in povzroča težave z razdeljevanjem besed (kar LaTeX obiajno opravi sam). Sodobne različice teh okolij so bile implementirane v paketu `ragged2e`. Ta paket morate naloiti v preambuli z ukazom `\usepackage{ragged2e}`. Ta paket ponuja okolja `FlushLeft`, `FlushRight` in `Center`, ki so izboljšane različice zgoraj omenjenih okolij.

Njegova uporaba je preprosta in laje razumljiva s pomojo vaje.

Seznami

LaTeX ima tri vrste seznamov: *urejene seznane*, *neurejene seznane* in *opise*. Sintaksa za vse tri vrste seznamov je podobna. Zanemo z okoljem, ki ga elimo uporabiti, nato pa vsak element seznama zanemo z ukazom `\item`. Za urejene seznane uporabimo okolje `enumerate`, za neurejene seznane okolje `itemize`, za opise pa okolje `description`.

Example 0.1.7.

Podobno kot pri loginih enotah, LaTeX samodejno izvede otevilčenje urejenih seznamov. Lahko tudi gnezdit seznane, tj. imeti seznam znotraj seznama.

Example 0.1.8.

Paket `enumitem` (v ang.), nam omogoča prilagoditev videza seznamov. Na primer, lahko spremenimo oznake neurejenih seznamov ali prilagodimo zamik elementov seznama. Za uporabo tega paketa ga moramo naložiti v preambuli z ukazom `\usepackage{enumitem}`. Ko smo naložili paket, lahko uporabimo seznam lastnosti po začetku ustreznega okolja.

Monosti uporabe tega paketa so ogromne in vključujejo prilagajanje oznak, boljši razmik med seznami, seznane v vrstici, različne sloge za opisne seznane in drugo. Tukaj bomo obravnavali le nekaj osnovnih primerov.

Example 0.1.9.

Example 0.1.10.

Seveda, rono nastavljanje lastnosti vsakega seznama je nevarno in v nasprotju s filozofijo LaTeXa. Globalne lastnosti lahko nastavimo na naslednji način.

Example 0.1.11.

Opozorilo!

Dejstvo, da lahko seznane malo prilagodimo, e ne pomeni, da bi to morali storiti. Na splono bi morali izbrati slog in se ga drati. Poleg tega je privzeti slog pogosto dovolj dober za mnoge namene.

Tabele

Tabele v LaTeXu so bile obravnavane na več načinov. LaTeX ima zelo primitivni način obravnavanja navpino poravnanih vsebin, ki jih je mogoče enostavno pretvoriti v tabelo. Obstaja več paketov, ki razirjajo to funkcionalnost. Eden od prvih in najpogostejše uporabljenih je paket `array`. Ta paket morate običajno naložiti, ko delate s tabelami.

Glej tudi

Obstaja veliko paketov, ki na različne načine razirjajo paket `array`. Vsi niso medsebojno združljivi, zato njihovo podrobno raziskovanje presega obseg tega ubenika. Vendar pa so najpogostejše uporabljane funkcionalnosti za tabele zajete v enem od naslednjih dveh virov (v angleščini):

- Tabele — Dokumentacija Overleaf.
- LaTeX wikibooks — tabele.

LaTeX uporablja okolje `tabular` za tiskanje vsebine, ki je poravnana tako navpino kot vodoravno, torej nekaj, kar je zelo podobno tabeli. Sintaksa prva vrstica okolja je

```
\begin{tabular}[pos]{<poravnava stolpcev>}
```

Table 3: Specifikacija stolpca za okolje `tabular`

Znak	Opis
<code>l</code>	Poravnava besedila v stolpcu na levo.
<code>c</code>	Poravnava besedila v stolpcu na sredino.
<code>r</code>	Poravnava besedila v stolpcu na desno.
<code>p{<irina>}</code>	Besedilo v stolpcu bo zavito, e preseee določeno irino.
<code>m{<irina>}</code>	Kot <code>p{<irina>}</code> , vendar bo besedilo navpino poravnano na sredino celice.
<code>b{<irina>}</code>	Kot <code>p{<irina>}</code> , vendar bo besedilo navpino poravnano na dno celice.

kjer je `pos` neobvezna monost, ki določa navpino poravnavo tabele glede na preostali del besedila (privzeto je `c`, tj. sredie, Vendar sta mona tudi `b` (za dno) in `t` (za vrh)). Argument `<poravnava stolpcev>` je niz, ki določajo tevilko in poravnavo stolpcev. Monosti za ta niz so pojasnjene v Table ?? . Nekateri od njih lahko zahtevajo paket `array`.

Example 0.1.12.**Example 0.1.13.****Example 0.1.14.**

Za loevanje stolpcev v tabeli uporabimo znak `&`, za zaetek nove vrstice pa ukaz `\\`. e elimo navpino rto med stolpci, uporabimo znak `|` v nizu `<poravnava stolpcev>`. Za vodoravne rte, ki se raztezajo ez celotno irino tabele, uporabimo ukaz `\hline` na koncu vrstice. Za vodoravne rte, ki se raztezajo ez določene stolpce, uporabimo ukaz `\cline{i-j}`, kjer `i` in `j` določata zaetni in konni stolpec, ez katerega se rta razteza.

Example 0.1.15.

Zgoraj opisani ukazi so e dovolj za izdelavo kompleksnih tabel v LaTeXu. Vendar pa konni rezultat ni nujno lep in izgleda nekoliko zastarel.

Obstaja veliko paketov, ki razirjajo funkcionalnosti okolja `tabular`. Osredotoili se bomo na paket `booktabs`, ki je zasnovan za izdelavo tabel v LaTeXu, ki so primerne za objavo. Ta paket velja za sodoben standard za izdelavo tabel v LaTeXu.

Ta paket ustvarja tabele v skladu z naslednjima dvema konvencijama:

1. Lepe tabele ne potrebujejo navpinih rt.
2. Lepe tabele ne zlorablajo vodoravnih rt, zlasti pa nikoli ne smemo uporabljati dvojnih vodoravnih rt.

Vse, kar je izrecno obravnavano v tej knjigi, je zdruljivo s paketom `booktabs`, vendar lahko nekateri drugi klasini pristopi (zlasti navpine rte na tabelah) povzroijo nepriakovane rezultate.

Najprej moramo naloiti paket z ukazom `\usepackage{booktabs}` v preambuli. Nato lahko uporabimo ukaze `\toprule`, `\midrule` in `\bottomrule` za ustvarjanje vodoravnih rt na vrhu, sredini in dnu tabele.

Example 0.1.16.

Simbol `@{sep}` lahko uporabimo za vstavljanje vsebine *med stolpce*. Tukaj je `sep` niz, ki bo vstavljen med celice ustreznih stolpcev. Ta sintaksa je uporabna za vstavljanje dodatnega prostora med stolpci z uporabo ukaza `\hspace{<irina>}` kot vrednost za niz `sep`. Lahko jo uporabimo tudi za vstavljanje posebnih znakov. Naslednji primer naj pojasni ta koncept.

Example 0.1.17.

Ukaz `\multicolumn{n}{<poravnava>}{besedilo}` združi n stolpcev v en stolpec, ki vsebuje `besedilo` in je poravnan glede na `<poravnava>`. Ta ukaz je uporaben, kadar elimo, da se besedilo razteza ez ve stolpcev, na primer za naslove stolpcev.

Delno vodoravno rto iz stolpca i v stolpec j natisnemo z ukazom `\cmidrule{i-j}`. To je podobno ukazu `\cline`, ki smo ga preuili prej, vendar je del paketa `booktabs`. rta, ki jo dobimo, je tanja in izgleda bolj estetsko.

Vasih potrebujemo, da se celica razteza ez ve vrstic (v nasprotju z ve stolpci). To je mogoe v LaTeXu, ko naloimo paket `multirow`. Ta paket nam omogoja uporabo ukaza `\multirow`, ki ima naslednjo sintakso.

```
\multirow[np]{<tVrstic>}{<irina>}[nGibanje]{besedilo}
```

kjer je `np` navpina poravnava celice glede na preostali del besedila (privzeto je `c`, tj. sredie, vendar sta mona tudi `b` (za dno) in `t` (za vrh)). Argument `tVrstic` jetevilo vrstic, ez katere se celica razteza, `irina` pa `irina` celice (lahko uporabimo `*`, da LaTeX sam dolo `irino` ali `=` za enako kot stolpec, v tem primeru, da je stolpec doloen z `p{<irina>}`, `m{<irina>}` ali `b{<irina>}`). Neobvezni argument `nGibanje` doloa navpino premikanje besedila v celici (privzeto je `Opt`, kar pomeni, da je besedilo navpino poravnano na sredino celice). Nazadnje je `besedilo` vsebina celice.

Za razliko od ukaza `\multicolumn`, ukaz `\multirow` ne nadomesti celic v vrsticah, ki jih pokriva. To pomeni, da moramo v teh vrsticah pustiti prazne celice (tj. uporabiti `&` za loevanje stolpcev, vendar brez vsebine med njimi).

Ko je treba stolpec oblikovati na doloen nain, je neprijetno v vsako celico vstaviti iste ukaze. Poleg tega, e se odloite, da je treba nekaj spremeniti v oblikovanju, bi morali vsako celico urediti posebej. Da bi se temu izognili, paket `array` opredeljuje `>{ukazi}` in `<{ukazi}` specifikacije stolpcev, ki se lahko uporabijo za vstavljanje kode pred in za stolpec.

Example 0.1.18.

eprav je uporaba ukazov `>{...}` in `<{...}` prirona, e imamo samo en tak stolpec, postane hitro neprijetna, ko se v kateri koli tabeli pojavi ve stolpcev istega tipa. V tem primeru je morda zaeleno uporabiti ukaz

```
\newcolumnntype{<ime>}{<definicija>}
```

za definiranje nove vrste stolpca, ki ga lahko uporabimo v okolju `tabular`. Tukaj je `<ime>` ime nove vrste stolpca (ena rka), `<definicija>` pa je definicija vrste stolpca, ki lahko vkljuuje ukaze `>{...}` in `<{...}`.

Example 0.1.19.**Glej tudi!**

Ukazi, opisani v tem razdelku, so dovolj za izdelavo lepih tabel v LaTeXu. Vendar pa obstaja veliko paketov, ki razirjajo funkcionalnosti okolja `tabular`. e bralca zanimajo bolj napredne funkcije, naj si ogleda naslednje vire (v angleini):

- Paket `longtable`. za tabele, ki se raztezajo ez ve strani: [dokumentacija]
- Paket `colortbl` za barvne tabele.
- Paket `tabularx` za tabele, ki se prilagajo irini strani.

Slike

LaTeX sam po sebi ne podpira vkljujevanja slik, vendar pa to omogoja paket `graphicx`. Ta paket moramo naloiti v preambuli z ukazom `\usepackage{graphicx}`. Nato lahko uporabimo ukaz `\includegraphics` za vkljujevanje slik v dokument. Ta ukaz ima naslednjo sintakso.

```
\includegraphics[<lastnosti>]{<pot do slike>}
```

kjer je `<pot do slike>` pot do datoteke s sliko (relativna ali absolutna). Datoteka slike je lahko v razlinih formatih, vendar so najbolj podprti formati `.png`, `.jpeg` in `.pdf`. Lahko izпустite konnico datoteke in LaTeX bo poskusil naslednje (v tem vrstnem redu): `.pdf`, `.png`, `.jpg`, `.mps`, `.jpeg`, `.jbig2`, `.jb2`.

Neobvezni argument `<lastnosti>` je seznam lastnosti, ki jih elimo uporabiti pri vkljujti slike. Najpogostejše uporabljena lastnost sta `width` in `height`, ki določata irino oz. viino slike. e samo ena od teh lastnosti dolo, bo druga samodejno prilagojena, da ohrani razmerje stranic slike.

Example 0.1.20.

Namesto ronega doloanja irine ali viine slike lahko uporabimo tudi monosti `scale` za poveanje ali pomanjanje slike za doloen faktor. Prenos negativnih vrednosti bo zrcalil sliko.

Example 0.1.21.

e elimo sliko zavrteti, lahko uporabimo monost `angle`, ki doloa kot vrtenja v stopinjah (v nasprotni smeri urinega kazalca). Privzeto je slika zavrtana okoli svojega spodnjega levega kota, vendar lahko uporabimo monost `origin`, da spremenimo to toko. Mone vrednosti za `origin` so `c` (sredie), `l` (levo), `r` (desno), `t` (zgoraj), `b` (spodaj) in kombinacije teh vrednosti, kot so `lt` (levo zgoraj), `rb` (desno spodaj) itd.

Example 0.1.22.

Plavajoi elementi

e poskusite napisati dalji tekst z ve tabelami in slikami, boste kmalu opazili problem: vkljuitev teh elementov v obiajni tekstovni tok bo povzročila veliko praznega prostora.

Poglejmo naslednji primer:

Example 0.1.23.

V prvi strani je veliko praznega prostora, ki ga povzroa slika. Da bi se temu izognili, LaTeX ponuja koncept plavajoih elementov, ki vkljuuje slike in tabele. Plavajoi elementi so lahko postavljeni na najbolj primerno mesto v dokumentu, ne da bi motili besedilo.

LaTeX ponuja dve okolji za plavajoe elemente: `figure` za slike in `table` za tabele. Ta okolja imajo podobno sintakso.

```
\begin{figure}[<pozicija>]
%...Vsebina slike, obiajno z ukazom \includegraphics...
\end{figure}
```

```
\begin{table}[<pozicija>]
%... Vsebina tabele, obiajno z okoljem tabular...
\end{table}
```

kjer je `<pozicija>` neobvezna monost, ki doloa, kje naj LaTeX poskua postaviti plavajoi element. Mone vrednosti so v Table ??.

Na primer, `\begin{table}[!hbp]` pomeni, da naj LaTeX poskua postaviti tabelo tukaj, e je mogoe (`h`), sicer na dno strani (`b`), e to ni mogoe, pa na posebno stran samo za plavajoe elemente (`p`); to vse pa lahko naredi tudi, e rezultat ni najlepši moen (!). e ne podamo nobene monosti, LaTeX privzeto uporabi `tbp`.

Z ukazom `\caption{<pojasnilo>}` lahko dodamo pojasnilo plavajoemu elementu. Ta ukaz mora biti znotraj okolja `figure` ali `table`. Oteviljenje in niz "Table" oziroma "Figure" se dodata samodejno. Seveda, e smo v preambuli spremenili jezik dokumenta z ukazom `\usepackage[<jezik>]{babel}`, se bosta spremenila tudi ta niza. Obiajno uporabljamo ukaz `\centering` znotraj okolja `figure` ali `table`, da centriramo vsebino.

Table 4: Pozicije plavajoih elementov

Oznaka	Objekt lahko stoji...
h	tukaj, na mestu, kjer je definiran v besedilu (samo za majhne elemente).
t	na vrhu strani.
b	na dnu strani.
p	na posebni strani, ki vsebuje samo plavajoe elemente.
!	brez upotevanja veine vgrajenih parametrov, ki bi lahko prepreile, da je objekt postavljen na to mesto.

Example 0.1.24.

S ukazi `\listoftables` in `\listoffigures` lahko ustvarimo seznam tabel oziroma seznam slik, podobno kot z ukazom `\tableofcontents` za kazalo vsebine. e uporabljamo dolga pojasnila, lahko podobno kot pri naslovih loginih enot kot dodatni argument zapiemo kratko pojasnilo, ki se bo pojavilo v seznamu tabel oziroma seznamu slik:

```
\caption[<kratko pojasnilo>]{<dolgo pojasnilo>}
```

Vasih elimo vkljuiti ve slik skupaj. To lahko storimo z uporabo paketa `subcaption`. Ta paket nam omogoča uporabo okolja `subfigure` znotraj okolja `figure`. Vsaka podslika lahko ima svoje pojasnilo z ukazom `\caption{<pojasnilo>}`. Celotna figura lahko ima tudi svoje pojasnilo z ukazom `\caption{<pojasnilo>}` znotraj okolja `figure`. Okolje `subfigure` ima podobno sintakso kot okolje `figure`, vendar mora imeti tudi obvezni argument, ki določa irino podslike.

Example 0.1.25.

Ukaz `\centering` znotraj okolja `figure` obravnava obe sliki kot eno samo sliko. Na splono je dobro uporabiti relativne doline za razporeditev slik. V tem primeru določamo, da je vsaka podslika 40 % irine strani. Ukaz `\hfill` med podslikama vstavi vodoravni prostor, ki se razteza, da zapolni prostor med njima. Namesto `\hfill` lahko uporabimo tudi `\hspace{<irina>}`, da vstavimo fiksno irino prostora med podslikama. Podnaslovi se ne prikazujejo pri tiskanju seznama slik (z uporabo ukaza `\listoffigures`), temve le napisi, povezani s sliko. Okolje `subfigure` ima tudi neobvezni argument, ki določa pozicijo podslike glede na slike okoli nje; mnoge vrednosti so `t`, `b` in `c` (za vrh, dno, sredino).

Podobno kot podslike lahko ustvarimo tudi podtabele z uporabo okolja `subtable` znotraj okolja `table`. Uporaba je enaka kot pri podslikah, ampak ni priporočljiva. Za razliko od slik, skoraj ni primerov, ko bi imeli dve tabeli pod istim naslovom bolj berljivost kot pod razlinimi naslovi.

Vaje

- Exercise ??: Ustvarite dokument z ve razdelki in podrazdelki.
- Exercise ??: Ustvarite tabelo z razlinimi poravnkami stolpcev in vrstic.
- Exercise ??: Ustvarite razline vrste seznamov (otevilene, neotevilene, opisne).
- Exercise ??: Ustvarite tabelo s paketom `booktabs`.
- Exercise ??: Uredite tabelo iz prejnje vaje z uporabo ukaza `\multicolumn`.
- Exercise ??: Uredite tabelo iz prejnje vaje z uporabo ukaza `\cmidrule`.
- Exercise ??: Uredite tabelo iz prejnje vaje z uporabo ukaza `\multirow`.
- Exercise ??: Ustvarite dokument z vsaj tremi tabelami in dvema slikama kot plavajoe elemente.

0.1.3 Sklicevanje in literatura

Z LaTeXom lahko oblikujemo ne le dokumente, ki so visoke kakovosti za tiskanje. Vendar pa je danes veliko dokumentov prebrati na zaslonu računalnika ali mobilne naprave. Paket `hyperref` omogoča ustvarjanje dokumentov PDF z aktivnimi povezavami oz. *hiperpovezavami*. To pomeni, da lahko kliknemo na sklic in nas ta pripelje do ustreznega dela dokumenta ali na zunanjo spletno stran. Paket `hyperref` omogoča tudi ustvarjanje kazal, ki so interaktivni in omogočajo enostavno navigacijo po dokumentu.

Za zdaj bomo uporabili paket `hyperref` za sklicevanje na literaturo v našem dokumentu. Seveda, najprej moramo naložiti paket v preambuli našega dokumenta:

```
\usepackage{hyperref}
```

Pazite!

Paket `hyperref` mora biti naložen kot zadnji paket v preambuli, saj lahko povzroči konflikte z drugimi paketi, e ni naložen na koncu.

Sklicevanje

V dokumentu lahko sklicujemo na razline elemente, kot so slike, tabele, enabe, razdelki, itd. Za sklicevanje uporabimo ukaza `\label{<oznaka>}` in `\ref{<oznaka>}`. Ukaz `\label{<oznaka>}` postavimo takoj za element, na katere se elimo sklicevati. Ukaz `\ref{<oznaka>}` pa uporabimo na mestu, kjer elimo vstaviti sklic. Argument `<oznaka>` je poljuben niz znakov, ki ga izberemo sami, vendar mora biti edinstven v celotnem dokumentu.

Example 0.1.26.

Nasvet

Oznake je priporoljivo oblikovati tako, da so opisne ampak kratke. Na primer, za razdelek o sklicevanju, `sec:sklicevanje`, je boljša oznaka kot `s1` ali `razdelek1`.

Dobra praksa je tudi, da uporabljamo predpone za razline vrste elementov, kot so `sec:` za razdelke, `fig:` za slike, `tab:` za tabele in `eq:` za enabe.

Opomba

Ukaz `\ref` vrne le tevilko elementa, na primer tevilko razdelka ali slike. Potrebno je, da rono dodamo besedilo, kot je “Razdelek”, “Slika” ali “Tabela”, pred tevilko.

Obstajajo paketi, kot je `cleveref`, ki omogočajo samodejno dodajanje besedila pred tevilko. Na alost, paket `cleveref` ni podpira slovenskega jezika, in eprav je mogoe definirati nove jezike, je trenutno slovenski jezik teko implementirati zaradi sklanjatev (prim. glej Slika 1” v na Sliki 1”).

e uporabimo ukaz `\ref{<oznaka>}` z razdelki “z zvezdico” (npr. `\section*{...}`), ki niso otevileni, bo ukaz vrnil ni ali nepriakovano vrednost, saj ti razdelki nimajo tevilke. Zato je priporoljivo, da se na razdelke “z zvezdico” ne sklicujemo z ukazom `\ref`. Mona alternativa je uporaba ukaza `\nameref{<oznaka>}`, ki vrne ime razdelka.

Ukaz `\pageref{<oznaka>}` vrne tevilko strani, na kateri se nahaja element z dano oznako. Ta ukaz lahko uporabimo z razdelki “z zvezdico”, saj nam ni treba pridobiti tevilke razdelka, ampak le tevilko strani.

Za plavajoe elemente, kot so slike in tabele, je priporoljivo, da oznako dodamo znotraj okolja `figure` ali `table`, takoj po ukazu `\caption{...}`. To zagotavlja, da se oznaka nanaa na pravilno tevilko slike ali tabele.

Example 0.1.27.

Oznaujmo in sklicujmo lahko vse, emur LaTeX dodeli tevilko. Na primer, toke v seznamih.

Example 0.1.28.**Literartura****Klasini nain**

Za vkljuitev literature v dokument LaTeX, lahko uporabimo okolje `thebibliography`. To je le poseben seznam, kjer vsaka referenca dobi svojo oznako z ukazom `\bibitem{<oznaka>}`. Nato se na referenco sklicujemo z ukazom `\cite{<oznaka>}`.

Example 0.1.29.

Ta nain je enostaven in hiter, vendar ima nekaj pomanjkljivosti:

- Rono moramo vnaati vse podatke o referencah, kar je lahko zamudno in nagnjeno k napakam.
- Ni standardiziranega formata za vnos referenc, kar lahko vodi do neenotnosti v slogu citiranja.
- Enostavne spremembe sloga citiranja so teavne, saj moramo rono prilagoditi vsako referenco.
- Ni lonega/struturnega oznaevanja, ki bi nakazovalo, kateri del bibliografskega vnosa je avtor, naslov, leto izdaje itd.
- Sortiranje referenc je rono in lahko vodi do napak. Urejanje (npr. po letu izdaje namesto po priimku avtorja) je prav tako rono in zamudno.
- Na splono to ni pristop, ki bi bil znailen za LaTeX.

Zaradi teh razlogov je priporoljivo uporabiti bolj napredne metode za upravljanje literature v LaTeX dokumentih, kot je BibTeX ali/in BibLaTeX.

BibLaTeX nain

BibTeX je orodje za upravljanje bibliografskih podatkov, ki deluje skupaj z LaTeXom. Omogoa nam, da shranimo vse nae reference v loeno datoteko `.bib`, kar olaja upravljanje in ponovno uporabo referenc v razlinih dokumentih. BibLaTeX je sodobnejša alternativa BibTeX-u, ki ponuja ve funkcionalnosti in prilagodljivosti.

Uporaba BibLaTeX-a vkljuuje naslednje korake:

1. Ustvarimo datoteko z bibliografskimi podatki z raziritvijo `.bib`. V tej datoteki so reference shranjene v posebnem formatu.
2. V preambulo naega LaTeX dokumenta vkljuimo paket `biblatex` in doloimo slog citiranja ter datoteko z bibliografskimi podatki.

Datoteka `.bib` je sestavljena iz vnosa, kjer vsak vnos predstavlja eno referenco. Vsak vnos ima svoj edinstven klju, ki ga uporabimo za sklicevanje na to referenco v naem dokumentu. Vsak vnos vsebuje razlina polja, kot so avtor, naslov, leto izdaje, itd. in izgledajo nekako takole:

```
@<vrsta_Vnosa>{<klju>,
<polje1>      = {vrednost1},
<polje2>      = {vrednost2},
<polje3>      = {vrednost3},
<poljeN>      = {vrednostN},
}
```

kjer je `<vrsta_Vnosa>` lahko `book`, `article`, `inproceedings`, `misc`, itd., odvisno od vrste reference. `<klju>` je edinstven identifikator za ta vnos, ki ga bomo uporabili v ukazu `\cite{<klju>}` v naem LaTeX dokumentu. `<polje1>`, `<polje2>`, itd. so razlina polja, ki vsebujejo informacije o referenci, kot so avtor, naslov, leto izdaje, itd. Npr. za knjigo *Harry Potter in kamen modrosti* bi bil vnos videti takole:

```
@book{hp1,
author    = {J. K. Rowling},
title     = {Harry Potter in kamen modrosti},
publisher = {Zaloba EPTA},
year      = {1997}
}
```

Polja so odvisna od vrste vnosa in slog bibliografije, ki ga uporabljamo (ve o tem kasneje). Za knjige so obiajna polja `author`, `title`, `publisher` in `year`. Za lanke v revijah bi lahko uporabili polja, kot so `journal`, `volume`, `number` in `pages`. Za spletne vire je pogosto uporabno polje `url` in `note`. Celoten seznam vrst vnosov in polj najdete v dokumentaciji paketa BibLaTeX ali tem lanku na wikibooks.

Dobra novica je, da obiajno ni potrebno rono pisati datoteke `.bib`, saj obstajajo tevilna orodja in spletne strani, ki lahko samodejno generirajo vnose v pravilnem formatu. Nekatera orodja, ki jih lahko uporabite za ustvarjanje in upravljanje datotek `.bib`, vkljuujejo:

- Google Scholar: omogoa izvoz posameznih referenc v format `.bib`.
- COBISS: slovenski sistem za iskanje in izposajo knjininih gradiv, ki omogoa izvoz bibliografskih podatkov v format `.bib`.
- ZoteroBib: spletno orodje za hitro ustvarjanje bibliografij, ki podpira izvoz v format `.bib`.
- MathSciNet: spletna baza podatkov za matematiko.
- DBLP: baza podatkov za raunalnitvo.

Ko smo pripravili nao bazo podatkov referenc (tj. nao datoteko `.bib`, recimo `literatura.bib`), moramo v naem LaTeX dokumentu naloiti paket `biblatex` in povezati datoteko z bibliografskimi podatki. To storimo z naslednjimi ukazi v preambuli:

```
\usepackage{biblatex} % Naloi paket biblatex
\addbibresource{literatura.bib} % Povee datoteko z bibliografskimi podatki, recimo literatura.
```

V besedilu naega dokumenta se na reference sklicujemo z ukazom `\cite{<klju>}`, kjer je `<klju>` klju vnosa v nai datoteki `.bib`. Parameter `<klju>` lahko je tudi seznam kljuev, loenih z vejico, e elimo sklicevati na ve referenc hkrati.

Na koncu dokumenta (ali, kjerkoli e elimo natisniti seznam referenc), dodamo ukaz `\printbibliography`, ki natisne seznam vseh referenc, na katere smo se sklicevali v besedilu. Upotevajte, da eprav nismo uporabili vseh referenc iz datoteke `.bib`, bo `\printbibliography` natisnil le tiste, na katere smo se sklicevali z ukazom `\cite{<klju>}`.

Z uporabo BibLaTeX lahko na nadzorovan nain prilagodimo videz naih citatov in sklicev. To storimo z izbiro sloga citiranja, ki ga doloimo kot monost pri nalaganju paketa `biblatex`. Na primer,

```
\usepackage[style=alphabetic]{biblatex}
```

naloi slog `alphabetic`, ki uporablja rke iz imen avtorjev in leto izdaje za oznaevanje citatov v besedilu. Privzeto je slog `numeric`, ki uporablja tevilke za oznaevanje citatov. Obstaja veliko razlinih slogov citiranja, ki jih lahko uporabimo, odvisno od naih potreb in preferenc. Obseen seznam slogov najdete v dokumentaciji paketa BibLaTeX ali na Overleaf.

Pazite!

Slog citatov lahko spreminjamo neodvisno, to pomeni, da lahko spreminjamo, kaj ukaz `\cite` natisne v naem besedilu in kako se natisnejo oznake, ko natisnemo bibliografijo, vendar je dobro ohraniti enak slog.

e damo monost `sorting=ynt` razvrsti bibliografijo po letu izdaje (`y`), nato po imenu avtorja (`n`) in nato po naslovu (`t`). Privzeto je `sorting=nty`, kar pomeni, da se najprej razvrsti po imenu avtorja, nato po naslovu in nato po letu izdaje.

Celoten seznam monosti za razvrnanje najdete v dokumentaciji paketa BibLaTeX ali na BibLaTeX cheat sheet.

Table 5: Naslov bibliografije v kazalu.

Vrednost	Opis
<code>bibliography</code>	privzeto, kot je razdelek »z zvezdico«.
<code>bibintoc</code>	kot razdelek z zvezdico, vendar se bo pojavil v kazalu.
<code>subbibliography</code>	bo bibliografija padla za eno stopnjo v hierarhiji (<code>\section*</code> namesto <code>\chapter*</code> , <code>\subsection*</code> namesto <code>\section*</code> in tako naprej).
<code>bibnumbered</code>	naraven razdelek (brez zvezdice), torej s tevilko in v kazalu.
<code>subbibintoc</code>	podobno kot <code>subbibliography</code> , vendar bo prikazana v kazalu.
<code>subbibnumbered</code>	podobno kot <code>subbibliography</code> , vendar brez zvezdice.
<code>none</code>	ne bo natisnil naslova.

Kot smo e omenili, ukaz `\printbibliography` natisne seznam vseh referenc, na katere smo se sklicevali v besedilu. e elimo natisniti reference, na katere se nismo sklicevali, lahko uporabimo ukaz `\nocite{<klju>}`. Ta ukaz nam omogoča, da vkljuimo določene vnose v bibliografijo, tudi e se nanje nismo sklicevali v besedilu. e elimo vkljuiti vse vnose iz nae datoteke `.bib`, lahko uporabimo ukaz `\nocite{*}`.

Ukaz `\printbibliography` lahko sprejme razline monosti, ki nadzorujejo videz bibliografije. Na primer, lahko uporabimo monost `title`, da spremenimo naslov bibliografije, kot je prikazano spodaj:

Example 0.1.30.

Referenca lahko organizirate z uporabo filtrov. Na primer, lahko natisnemo samo knjige z uporabo monosti `type=book`:

Example 0.1.31.

Na voljo so filtri `type`, `keywords` in `category` in lahko uporabimo tudi njihove negacije. Z ukazom `\defbibfilter{<ime_filtra>}` lahko definiramo tudi svoje filtre.

Example 0.1.32.

Example 0.1.33.

Privzeto se bibliografija ne prikae v kazalu vsebine. e elimo, da se prikae, lahko uporabimo monost `heading`. Razlien vrednosti za monost `heading` so v Table ??'.

Example 0.1.34.

Example 0.1.35.

Example 0.1.36.

Example 0.1.37.

Example 0.1.38.

Example 0.1.39.

Example 0.1.40.

Vaje

- Exercise ??: Sklicevanje z uporabo paketa `hyperref`.
- Exercise ??: Sklicevanje na razline elemente v dokumentu.
- Exercise ??: Uporaba BibLaTeX-a za upravljanje literature.

0.1.4 Matematika v LaTeXu

LaTeX je e posebej primeren za pisanje matematinih izrazov in ena izmed njegovih glavnih prednosti je prav podpora za matematine simbole in formule. LaTeX je e sam po sebi zelo zmogljiv. Vendar pa obstaja tudi veliko paketov, ki e dodatno razirijo njegove zmogljivosti na podroju matematike.

Standardni paket za matematiko so:

- **amsmath**: paket, ki ga je razvila Amerika matematina druba (American Mathematical Society) in ponuja tevilne izboljšave za pisanje matematinih izrazov.
- **amssymb**: paket, ki ponuja dodatne matematine simbole.
- **amsfonts**: paket, ki omogoča uporabo dodatnih matematinih pisav.
- **amsthm**: paket za definiranje in oblikovanje matematinih izrekov, trditve, definicij itd.

Ti paketi so pogosto vključeni v predloge dokumentov, vendar jih je mogoe vključiti tudi rono z ukazom `\usepackage{amsmath, amssymb, amsfonts, amsthm}` v preambuli vaega LaTeX dokumenta.

V sodoben LaTeX dokument je priporoljivo vključiti paket **mathtools**, ki je raziritev paketa **amsmath** in ponuja dodatne funkcije in izboljšave za pisanje matematinih izrazov.

Pomembno!

V tem poglavju bomo predpostavljali, da so ti paketi e vključeni v va dokument. Lahko jih vključite z naslednjim ukazom v preambuli vaega LaTeX dokumenta:

```
\usepackage{mathtools} % mathtools vkljuuje tudi amsmath
\usepackage{amssymb, amsfonts, amsthm}
```

Osnovni matematini izrazi

LaTeX omogoča pisanje matematinih izrazov na dva naina: znotraj besedila (*vrstini nain*; ang. *inline mode*) in v loenih vrsticah (*prikazen nain*; ang. *display mode*).

Matematika v vrstinem nainu se vstavlja z uporabo znakov `$...$` ali `\(...\)`. Na primer, izraz za Pitagorov izrek lahko zapiemo kot `$a^2 + b^2 = c^2$` (ali `\(a^2 + b^2 = c^2\)`), ki je natisnjen kot $a^2 + b^2 = c^2$.

Za prikaz matematinih izrazov v loenih vrsticah (in osredotoeno poravnano) se uporabljajo znaki `\[...\]` ali okolje `equation*` (sta enakomerna). Na primer, e je $ax^2 + bx + c = c$ kvadratna enaba, potem je reitev te enabe:

```
\[x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2} \]
```

kar je natisnjeno kot

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2}. \quad (1)$$

Ta izraz v vrstinem nainu bi bil zapisan kot `$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2}$`, kar je natisnjeno kot $x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2}$.

Opomba

Matematika v vrstinem nainu zapisana z `$...$` vasih se teje za zastarelo. Priporoljivo je uporabljati `\(...\)`. Vendar, v praksi veinoma avtorjev uporabijo `$...$`, ker je hitreje za tipkanje in dobi se enak rezultat.

Podobno je tudi pri prikaznem nainu, kjer je priporoljivo uporabljati `\[...\]` (ali okolja `equation*`) namesto `$$ ... $$`. Vendar je `$$...$$` resnino zastarelo in **se ne sme uporabljati**.

Matematine izraze v prikaznem nainu je mogoe tudi oteviliti z uporabo okolja `equation` (brez zvezdice). Rezultat je podoben kot zgoraj, vendar bo enaba otevilena. Seveda, lahko sklicujemo na to enabo v besedilu z uporabo ukazov `\label{<oznaka>}` in `\ref{<oznaka>}`:

Example 0.1.41.

Za enabe lahko tudi uporabimo ukaze `\eqref{<oznaka>}` in `\tag{<opis>}`, ki omogoata bolj prilagojeno sklicevanje in oznaevanje enab:

Example 0.1.42.

Example 0.1.43.

Ukazi in okolja za pisanje matematinih aktivirajo matematini nain, kjer so nekateri znaki in ukazi drugani kot v obiajnem besedilnem nainu. Na primer, presledki v matematinem nainu nimajo pomena, medtem ko so v besedilnem nainu pomembni:

Vse rke so v matematinem nainu obravnavane kot spremenljivke in so zato natisnjene v poevni pisavi (italics). Razmik med rkami je veji kot v obiajnem besedilu.

Zaradi zgoraj navedenih razlogov moramo matematino besedilo **vedno** zapisovati v matematinem nainu (v vrstici ali prikazano).

e elimo v matematinem nainu uporabiti obiajno besedilo, lahko uporabimo ukaz `\text{...}`: why?

Stavljenje matematinih formul

Monosti za matematine simbole je ogromno, zato bomo prikazali le nekaj primerov. Seznam primerov tukaj je namenjen prikazu nekaterih najpogostejih primerov uporabe, vendar nikakor ni izrpen.

Pri sestavljanju matematinih formul vam bodo morda v pomo naslednji viri.

- Comprehensive LaTeX Symbol List: Obseen seznam LaTeX simbolov, ki vkljuuje simbole iz razlinih paketov **Skoraj 500 strani**.
- Detexify: Orodje, ki vam omogoaa, da nariete simbol, ki ga iete, in orodje vam bo predlagalo ustrezne LaTeX ukaze za ta simbol.
- [Seznam osnovnih matematinih simbolov](#){target=“_blank”}: Kratek seznam pogosto uporabljenih LaTeX matematinih simbolov.

Izreki, dokazi, definicije, itd..

Ko sestavljanju matematinih besedil je pogosto potrebno vkljuiti izreke, dokaze, definicije in podobne strukture (ki jih bomo zaradi enostavnosti imenovali “teoremi”). Paket `amsthm`, ponuja enostaven nain za ustvarjanje teh struktur z uporabo okolij.

Najprej moramo definirati nova okolja za nae teoreme v preambuli dokumenta. To storimo z uporabo ukaza

```
\newtheorem{<ime_okolja>}[<drugi_teorem>]{<Napis>}[<razdelek>]
```

kjer je `<ime_okolja>` ime novega okolja in `<Napis>` je besedilo, ki se bo prikazalo pred tevilko teorema. Neobvezna argumenta `<drugi_teorem>` in `<razdelek>` doloata, kako se bodo teoremi tevilili (in lahko uporabimo samo ena od njih ali nobene). Uporabimo `<drugi_teorem>`, e elimo, da se novi teoremi tevilijo skupaj z obstojeim teoremom, kjer je `<drugi_teorem>` ime okolja. e elimo, da se teviljenje teoremov ponastavi v vsakem razdelku, uporabimo `<razdelek>`, kjer je `<razdelek>` ime tevca razdelkov (npr. `section` ali `subsection`). e ne uporabimo nobenega od teh dveh argumentov, se bodo teoremi tevilili neodvisno skozi celoten dokument (1, 2, 3, ...). Ukaz `\newtheorem*{<ime_okolja>}{<Naslov>}` definira razliico okolja, ki ne bo imela tevilke.

Po definiciji okolja lahko uporabimo novo okolje v naem dokumentu z uporabo standardnega LaTeX okolja sintakse:

```
\begin{<ime_okolja>}[<Naslov>]
Vsebina teorema.
\end{<ime_okolja>}
```

Neobvezni argument `<Naslov>` omogoča dodajanje naslova teorema, ki bo prikazan poleg tevilke teorema.

Example 0.1.44.

V LaTeXu obstajajo tri razline sloge teoremov:

- `plain`: privzeti slog, krepko naslov in poevno telo.
- `definition`: krepko naslov in pokonno telo.
- `remark`: poevno naslov in pokonno telo.

Za nastavitve sloga teorema uporabimo ukaz

```
\theoremstyle{<slog>}
```

pred definicijo okolja teorema. Vsak teorem, ki je definiran po tem ukazu, bo uporabil določen slog.

Example 0.1.45.

Paket `amsthm` ponuja tudi okolje `proof` za pisanje dokazov. Okolje samodejno doda besedo “Dokaz” na začetek in simbol kvadrata na konec dokaza.

Example 0.1.46.

Ukaz `\qedhere` lahko uporabimo znotraj okolja `proof`, da postavimo simbol kvadrata na določeno mesto, običajno na konec enabe.

Example 0.1.47.

Vaje

- Exercise ??: Napišite LaTeX kodo za osnovne matematične izraze.
- Exercise ??: Napišite LaTeX kodo za matematične izraze z uporabo okolji `align` in `align*` in `aligned`.
- Exercise ??: Napišite LaTeX kodo, da reproducira Maxwellove enabe.
- Exercise ??: Napišite LaTeX kodo za izreke in dokaze.

0.1.5 LaTeX: Razno

V tej poglavju so zbrani razlini nasveti in triki za delo z LaTeX-om, ki ne sodijo v druge poglavja.

Obseneji projekti

Za obseneje projekte, kot so diplomske naloge, je priporoljivo razdeliti dokument na ve datotek in jih vkljuiti v glavno datoteko z ukazom `\input{}` ali `\include{}`. To omogoa laje upravljanje in urejanje posameznih delov dokumenta.

Ukaz `\include{<pot do datoteke>}` vkljui vsebino navedene datoteke in samodejno zane novo stran pred in po vkljuitvi.

Ukaz `\includeonly{<ime_datoteke1>, <ime_datoteke2>, ...}` omogoa vkljuitev samo doloenih datotek med kompilacijo, kar je uporabno za hitreje pregledovanje doloenih delov dokumenta.

Pazite!

- Ukaz `\include{<ime_datoteke>}` lahko uporabljamo samo v **telesu glavne datoteke** ne znotraj drugih vkluenih datotek.
- Ukaz `\includeonly{<ime_datoteke1>, <ime_datoteke2>, ...}` mora biti postavljen v preambulo glavne datoteke. V telesu dokumenta morajo biti ukaze `\include{}` za vse datoteke, ki jih elimo vkljuiti.
- Ukaz `\include{}` vedno zane novo stran, zato ni primeren za vkljuevanje manjih delov besedila. Poleg tega, ta ukaz ne deluje znotraj okolij, kot so `\begin{figure}...\end{figure}` ali `\begin{table}...\end{table}`.

Ukaz `\include` je malo omejevalen in ima posebno uporabo: vkljuiti ve poglavje (`chapter`) v velik projekt. Ta ukaz vedno zane novo stran, to je koristno ko uporabljamo ukaz `\includeonly{}`, ker se prelomi strani ne bodo premaknili, e bomo vkljuili samo doloene datoteke.

Vasih je bolj smiselno uporabiti ukaz `\input{<pot do datoteke>}`, ki vkljui vsebino navedene datoteke brez preloma strani. Ukaz `\input{}` lahko uporabljamo kjerkoli v dokumentu, tudi znotraj okolij, kot so `\begin{figure}...\end{figure}` ali `\begin{table}...\end{table}`.

Ukaz `\input{}` lahko uporabimo tudi za vkljuitev datotek z ukazi LaTeX-a, kot so definicije makrov ali nastavitve paketov. Pogosta uporaba je vkljuitev datoteke, recimo, `preambula.tex`, ki vsebuje vse nastavitve in pakete, ki jih elimo uporabiti v naem dokumentu.

Pazite!

eprav lahko uporabimo ukaz `\input` za naloitev paketov v nao datoteko, mora vrstica, ki vsebuje ukaz `\documentclass`, **vedno** biti v glavni datoteki, pred vsemi ukazi `\input`.

Barve

Za uporabo barv v LaTeX dokumentih uporabimo paket `xcolor`. Paket omogoa uporabo treh ukazov za barvanje besedila ali ozadij:

- `\textcolor{<model>}{<barva>}{<besedilo>}`: Barva besedilo z doloeno barvo.
- `\color{<model>}{<barva>}`: Spremeni barvo besedila od tega mesta naprej. To je razliica *switch* ukaza `\textcolor`, kot je ukaz `\bfseries` za krepko pisavo (prim. `\textbf{}`).
- `\mathcolor{<model>}{<barva>}{<matematini_izraz>}`: Barva matematini izraz z doloeno barvo. Ne pozabite, da ukaze za *navadno* besedilo obiajno ne delujejo v matematinem nainu.

Za zdaj se ne skrbite za neobvezni parameter `<model>`. Ta parameter nadzira, kaj se priakuje za argument `<barva>`. e ni doloen, se privzeto uporablja model `named`, ki omogoa uporabo vnaprej doloenih imen barv, to pomeni, barve pokliemo z imeni.

Barve, ki so vnaprej doloene v paketu `xcolor`, so prikazane v Figure ??.

	red
	green
	blue
	cyan
	magenta
	yellow
	black
	gray
	white
	darkgray
	lightgray
	brown
	lime
	olive
	orange
	pink
	purple
	teal
	violet

Seznam določenih barv lahko razirimo z uporabo monosti, ki jih ponuja paket `xcolor`. Npr z monostjo `dvipsnames` lahko uporabimo dodatne barve prikazane v Figure ??.



Figure 4: Dodatne barve, ki jih določa paket `xcolor` z monostjo `dvipsnames`.

Druge monosti so `svgnames`, `x11names`. Monost `svgnames` določa 151 barv, ki so definirane v SVG standardu, medtem ko monost `x11names` določa 317 barv, ki so bile prvotno definirane za X11 Window System. Celoten seznam barv je na voljo v dokumentaciji paketa `xcolor`.

Drugi način pridobivanja novih barv je z meanjem dveh ali več barv. Sintaksa za meanje dveh barv je naslednja:

```
<barva1>!<odstotno>!<barva2>
```

Rezultat je barva, ki je sestavljena iz `<odstotno>` odstotkov `<barva1>` in $(100 - \text{odstotno})$ odstotkov `<barva2>`. Na primer, `red!40!blue` ustvari vijolino barvo, ki je meanica 40% rdeč in 60% modre.

Meanje lahko vključuje tudi več kot dve barvi. V tem primeru, meanje je izvedeno zaporedno. Na primer, `red!50!green!30!blue` najprej zmeša 50% rdeč in 50% zelene, nato pa rezultat zmeša z 70% modre in 30% prej dobljene barve.

Vasih moramo biti natančnejši pri barvah, ki jih uporabljamo. Za to lahko uporabimo izbirni argument `model`. Najlažje ga razumemo prek modela `Gray`. V tem modelu določimo odtenek sive barve z vrednostjo med 0 in 15, kjer 0 predstavlja rno in 15 belo.

Podobno lahko uporabimo model `grey`, kjer vrednosti segajo od 0 do 1, pri čemer 0 predstavlja rno in 1 belo.

V Table ?? so prikazani nekateri barvni modeli, ki jih podpira paket `xcolor`.

Opozorilo!

Pri meanju barv v različnih modelih lahko dobimo nepriakovane rezultate. Na primer, da izrazi `<barva1>!25!<barva2>` in `<barva2>!75!<barva1>` ne dajeta vedno iste barve, če sta `<barva1>` in `<barva2>` definirani v različnih barvnih modelih. Zato je priporočljivo, da pri meanju barv vedno uporabljamo barve iz istega modela. Alternativno v poglavje 6 dokumentacije paketa `xcolor` najdemo več informacij o spremenjenju med barvnimi modeli.

Z uporabo ukaza `\definecolor{ime_barve}{model}{parametri}` (v preambuli) lahko definiramo svoje barve. Na primer, z ukazom `\definecolor{ULrdeca}{RGB}{226,25,30}` definiramo barvo z imenom `ULrdeca`, ki je rdeča barva Univerze v Ljubljani.

Doslej smo obravnavali ukaze za barvanje besedila. Mono je tudi spremeniti barbo ozadja strani z ukazom

Table 6: Najpogostejši modeli barv v paketu xcolor

Model
Osnovne barve/komponente
Razpon parametrov
Primer
rgb
Rdea, zelena, modra
$[0, 1]^3$
<code>\textcolor[rgb]{1,0,0}{Rdea}</code>
RGB
Rdea, zelena, modra
$\{0, \dots, 255\}^3$
<code>\textcolor[RGB]{255,0,0}{Rdea}</code>
HTML
Rdea, zelena, modra
estnajstiki zapis
<code>\textcolor[HTML]{FF0000}{Rdea}</code>
cmY
Cijan, magenta, rumena
$[0, 1]^3$
<code>\textcolor[cmY]{0,1,1}{Rdea}</code>
cmYk
Cijan, magenta, rumena, rna
$[0, 1]^4$
<code>\textcolor[cmYk]{0,1,1,0}{Rdea}</code>
Gray
Odteneke sive
$\{0, \dots, 15\}$
<code>\textcolor[Gray]{0}{rna}</code>
gray
Odteneke sive
$[0, 1]$
<code>\textcolor[gray]{0}{rna}</code>
wave - Valovna dolina (nm) - $[380, 780]$ - <code>\textcolor[wave]{700}{Rdea}</code>

`\pagecolor[<model>]{<barva>}`.

Ta ukaz je različica *switch*, to pomeni da spremeni barvo celotne strani od mesta, kjer je uporabljen, naprej. e elimo vrniti nazaj privzeto barvo ozadja (prozoren), uporabimo ukaz `\nopagecolor`.

V izrazu za barvo lahko uporabimo znak - (minus), da dobimo nasprotno barvo. Na primer, `-white` predstavlja rno barvo.

e elimo spremeniti barvo samo določenega obmoja besedila, lahko uporabimo ukaz `\colorbox[<model>]{<barva>}{<besedilo>}`. Ta ukaz obdaja besedilo z ozadjem določene barve. Ukaz `\fcolorbox[<model>]{<barva_okvirja>}{<barva_ozadja>}{<besedilo>}` obdaja besedilo z okvirjem določene barve in ozadjem določene barve.

Ukazi in okolja po meri.

LaTeX omogoča ustvarjanje lastnih ukazov in okolij, kar lahko olajša pisanje ponavljajočih se struktur v dokumentu. Klasini `nain` za definiranje novega ukaza je z uporabo ukaza `\newcommand{<ime_ukaza>}[<tevilko_argumentov>]{<definicija>}` in novega okolja z uporabo ukaza `\newenvironment{<ime_okolja>}[<tevilko_argumentov>]{<zaetek_definicije>}{<konec_definicije>}` eprav ti ukazi delujejo povsem normalno za osnovne definicije novih ukazov in okolij, bodo postopoma postali zastareli. Tega pristopa tukaj ne bomo obravnavali, vendar e bralca zanima, priporočamo branje Overleafovih navodil za nove ukaze `{target=_blank}` in nova okolja `{target=_blank}` (v angleini).

Sodobnejši in bolj zmogljiv `nain` za definiranje novih ukazov in okolij je z uporabo ukaza `\NewDocumentCommand` in `\NewDocumentEnvironment`. Ta pristop omogoča bolj fleksibilno določanje argumentov in podpira različne vrste argumentov, kot so obvezni, neobvezni, in ve vrst argumentov.

Ukaz `\NewDocumentCommand` ima naslednjo sintakso:

`\NewDocumentCommand{<ime_ukaza>}{<vrsta_argumentov>}{<definicija>}`

- **<ime_ukaza>**: Ime novega ukaza, ki ga elimo definirati (vključno z `\`).
- **<vrsta_argumentov>**: Niz, ki določa vrste argumentov, ki jih ukaz sprejema. To bomo poasi obravnavali v nadaljevanju.
- **<definicija>**: Telo ukaza, kjer lahko uporabimo argumente znotraj ukaza.

Najlajše ukaz, da lahko definiramo je brez argumentov, to pomeni z praznim nizom za `<vrsta_argumentov>`.

Ta `nain` je zelo uporaben, ko obstajajo del besedila ali strukture, ki jih pogosto uporabljamo v dokumentu. Seveda, e se kdajkoli odloimo spremeniti to besedilo ali strukturo, moramo to storiti samo na enem mestu: v definiciji ukaza.

Pogosta uporaba novih ukazov je pri definiciji matematičnih izrazov, ki jih pogosto uporabljamo. Kot smo videli v prejšnjih poglavjih, nekatere vir (npr. Standard ISO 80000-2) priporočajo, da se za matematične konstante in posebne funkcije uporablja rimska pisava (upravljeno). Seveda, ni smiselno vsaki pisati `\mathrm{\pi}` za konstantno π ali `\mathrm{e}` za Eulerjevotevilo. Namesto tega lahko definiramo nove ukaze za te konstante:

Nasvet

Ko definiramo nove ukaze, ki se bodo uporabljali v matematičnem `nainu`, je priporočljivo, da ne vključimo nobenega simbola, ki označuje matematični `nain`. Na primer, pri definiciji ukaza za konstantno π , je bolje definirati ukaz `\kpi` kot `\mathrm{\pi}` namesto `$_\mathrm{\pi}$`.

e elimo definirati nov ukaz, ki sprejema argumente, moramo določiti vrsto intevilo argumentov v nizu `<vrsta_argumentov>`. Nekateri najpogostejši tipi argumentov so prikazani v Table ??.

Najlajše tip argumenta je `m`, ki predstavlja obvezni argument. Na primer, lahko definiramo ukaz za barvanje besedila z določenim in barvo. Pri definiciji ukaza, uporabljamo znak `#1` za sklicevanje na prvi argument.

Table 7: Najpogostejši tipi argumentov v `\NewDocumentCommand`

Tip argumenta	Opis
m	Obvezni argument, ki lahko vsebuje ve vrstic.
O{<privzeta_vrednost>}	Neobvezni argument, ki lahko vsebuje ve vrstic. e ni podan, ima privzeto vrednost <privzeta_vrednost>.
o	Neobvezni argument. e ni podan, natisne posebni niz <code>-NoValue-</code> . Obiajno v definicij uporaba pogojev za preverjanje, ali je bil argument podan kot je ukaz <code>\IfNoValueTF</code> .
D<levo><desno>{<privzeta_vrednost>}	Neobvezni argument, ki je obdan z <levo> in <desno>. e ni podan, ima privzeto vrednost <privzeta_vrednost>. Obiajno se uporablja znak < in >, za <levo> in <desno>. Deluje kot o, vendar s posebnimi loevalniki.
d<levo><desno>	Podobno kot D, vendar brez privzete vrednosti. Deluje kot o, vendar s posebnimi loevalniki.
s	Preklopni argument (switch). e je prisoten, je vrednost <code>\BooleanTrue</code> , sicer <code>\BooleanFalse</code> . Uporabno, ko definiramo ukaze ‘z zvezdico’ ali ‘brez zvezdice’.

Seveda, lahko definiramo ukaze z ve argumenti. Dovolj je da dodamo ve tipov argumentov v niz `<vrsta_argumentov>` in uporabimo `#2`, `#3`, ... za sklicevanje na druge argumente.

e elimo definirati ukaz z neobveznimi argumenti, lahko uporabimo tip `O{<privzeta_vrednost>}`. Ne pozabite, da neobvezni argumenti damo znotraj oklepaj `[]` pri klicu ukaza.

Lahko seveda kombiniramo razline tipe argumentov. Na primer, lahko definiramo ukaz z enim obve

```
::::{grid} 2 2 2 2
```

```
:::{card} LaTeX
```

```
“‘latex
V preambuli:
\NewDocumentCommand{\Zap}{0{n}m}
{#2_{1}, \dots, #2_{#1}}
% V telesu dokumenta:
Nekatere zaporedje imajo veliko elementov:
$\Zap[1000]{a}$,
za veino pa sploh ne vemo,
koliko jih je:
$\Zap{b}$.
```

```
“‘
```

```
:::
```

```
:::{card} PDF
```

```
“‘{figure} ./img/05_eg -ArgsMO -1.png
:label: fig:eg -ArgsMO -1
“‘
```

...

Upotevajte, da je vrstni red argumentov pomemben. `e` je ukaz `\ukaz` definiran z vrstnim redom argumentov `0{}`m, ga moramo klicati z obveznim argumentom najprej, nato pa neobveznim argumentom (e ga elimo podati), to pomeni `\ukaz[<neobvezni>]{<obvezni>}` je pravilen klic, medtem ko `\ukaz{<obvezni>}[<neobvezni>]` ni pravilen klic. Dobra praksa je, da najprej definiramo vse neobvezne argumente, nato pa obvezne argumente.

Vasih elimo da ukaz deluje drugače, `e` je neobvezni argument podan ali ne. V tem primeru lahko uporabimo tip argumenta `o` in ukaz `\IfValueTF` za preverjanje, ali je bil argument podan.

Namesto ukaza `\IfValueTF`, lahko uporabimo tudi ukaz `\IfValueT`, `e` elimo izvesti dejanje samo, `e` je bil argument podan, ali ukaz `\IfValueF`, `e` elimo izvesti dejanje samo, `e` argument ni bil podan. Lahko tudi uporabimo ukaze `\IfNoValueTF`, `\IfNoValueT`, in `\IfNoValueF`, ki delujejo nasprotno.

Ko elimo da ukaz sprejme neobvezni argument z določenimi loevalniki, lahko uporabimo tip argumenta `D<levo><desno>{<privzeta_vrednost>}`. Obiajno se uporabljata znaka `<` in `>` kot loevalnika.

`e` namesto `D<levo><desno>{<privzeta_vrednost>}` uporabimo `d<levo><desno>`, dobimo podoben uinek, vendar brez privzete vrednosti (tono tako, kot ko uporabljamo `o`). V tem primeru moramo vedno preveriti, ali je bil argument podan z uporabo ukaza `\IfValueTF` in podobnih.

Nazadnje, lahko tudi uporabljamo znak `s` za niz `<vrsta_argumentov>`, da definiramo preklopni argument oz. različni ukaz *z zvezdico*. Pri definiciji ukaza, lahko uporabimo ukaz `\IfBooleanTF`, da preverimo, ali je bil preklopni argument podan.

Lahko tudi definiramo nova okolja z uporabo ukaza `\NewDocumentEnvironment`, ki ima podobno sintakso kot `\NewDocumentCommand`.

```
\NewDocumentEnvironment{<ime_okolja>}{<vrsta_argumentov>}
{<zaetek_definicije>}
{<konec_definicije>}
```

- `<ime_okolja>`: Ime novega okolja, ki ga elimo definirati.
- `<vrsta_argumentov>`: Niz, ki določa vrste argumentov, ki jih okolje sprejema. To deluje skoraj enako kot pri ukazu `\NewDocumentCommand`.
- `<zaetek_definicije>`: Koda za zaetek okolja, kjer lahko uporabimo argumente znotraj okolja.
- `<konec_definicije>`: Koda za konec okolja.

Naslednji primeri naj bi pojasnili uporabo tega ukaza.

Example 0.1.48.

Lahko tudi definiramo okolja z argumenti. Na primer, lahko definiramo okolje `kraljIme`, ki sprejme en obvezni argument za ime kralja.

Example 0.1.49.

Tukaj imamo problem, ker elimo imeti razline naslove in zaključke za kralja in kraljico. To lahko reimo z uporabo preklopnega argumenta `s`. Moramo v definiciji okolja uporabiti ukaz `\IfBooleanTF`, da preverimo, ali je bil preklopni argument podan; to lahko storimo v obeh delih definicije okolja.

Pazite, za razliko od nekaterih drugih LaTeX okoljih (npr. `align`), ko uporabljamo okolje *z zvezdico*, zvezdica gre zunaj oklepajev imenja okolja, t.j. `\begin{kralij}*{Ime}` namesto `\begin{kralij}{Ime}`. Zvezdica ni potrebna na koncu okolja, t.j. `\end{kralij}` je pravilen klic tudi za okolje *z zvezdico*.

Example 0.1.50.

Vaje

- Exercise ??: Vaje o ukazih `\input{}` in `\include{}`.
- Exercise ??: Vaja o meanju barv.
- Exercise ??: Vaje o definiciji novih ukazov z `\NewDocumentCommand`.

0.1.6 Beamer: Predstavitve v LaTeXu

Beamer je LaTeX raziritev za izdelavo drsnic ("slajdov"). Za izdelavo predstavitev z Beamerjem uporabimo razred `beamer` namesto `article`. Nato uporabimo okolja `frame` za vsak posamezen diapozitiv.

Glej tudi: Beamer uporabniki vodi.

V tem poglavju bomo predstavili osnovne koncepte in ukaze za ustvarjanje predstavitev z Beamerjem. Za podrobneje informacije in napredne funkcije si oglejte uradno Beamer user guide.

Opozorilo

Razred `beamer` uporaba paket `babel` za jezike malo drugače kot običajni razredi dokumentov (kot so `article` ali `report`). V tem razredu je priporočljivo, da je jezik dokumenta določen kot monost razreda dokumenta (npr. `\documentclass[slovene]{beamer}`) in da se paket `babel` naloži brez monosti jezika (npr. `\usepackage{babel}`).

Ukaz `\frametitle{}` določi naslov drsnice. Lahko tudi uporabimo ukaz `framesubtitle{}` za podnaslov drsnice. Kraja monost je, da naslov oz. podnaslov drsnice podamo kot monost okolja `frame`; Example ??.

Example 0.1.51.

Pomembne drsnice

Podobno kot pri linkih lahko tudi pri Beamer predstavitev ustvarimo naslovni diapozitiv z uporabo ukazov `\title{}`, `\author{}` in `\date{}` v preambuli dokumenta.

Zdaj ukaz `\maketitle` v telesu dokumenta ustvari naslovni diapozitiv. Ukaz `\maketitle` je enakomerno kot

```
\begin{frame}
\titlepage
\end{frame}
```

Običajno ukaz `\date` nastavimo na ime konference ali dogodek, kjer bo predstavitev prikazana. V razredu `beamer` lahko tudi uporabljamo ukaze `\institute{}`, `\subtitle{}` in `\titlegraphic{}` za dodajanje dodatnih informacij na naslovno drsnico; pogledajte Example ??.

Example 0.1.52.

Nekatere teme Beamerja (glej Teme in barvne sheme) dodajo informacije o avtorju in datumu na vsako drsnico. Če je vsebina teh polj predolga, ne bo dovolj prostora; v tem primeru lahko dodamo neobvezni kratek argument v ukaze (npr. `\author[<kratek>]{<dolg>}`), ki se bo prikazal v glavi ali nogi drsnice, pogledajte Example ??.

Example 0.1.53.

Ukaz `\titlegraphic{}` doda sliko na naslovno drsnico. Če elimo dodati logotip na vse drsnice, lahko uporabimo ukaz `\logo{}` v preambuli dokumenta, pogledajte Example ??.

Upotevajte, da je pozicija logotipa odvisna od uporabljene teme. Poleg tega, nekatere teme ne podpirajo logotipov/grafikov na naslovni drsnici.

Example 0.1.54.

V razredu `beamer` lahko tudi uporabimo ukaze za *logine strukture* kot so `\section{}` in `\subsection{}` za organizacijo predstavitev. Ti ukazi ne ustvarijo drsnic, vendar jih lahko uporabimo za generiranje kazala vsebine. To lahko storimo z uporabo ukaza `\tableofcontents` v okolju `frame`, pogledajte Example ??.

Upotevajte, da razdelek *Literatura* ni v kazalo vključen, ker je definiran z ukazom `\section*{}`.

Example 0.1.55.

e elimo kazalo prikazati postopoma, lahko k ukazom `\tableofcontents` dodamo monost `pausesections`. Poglejte Example ??.

Example 0.1.56.

Ukaze kot `\section` ne ustvarijo drsnic, vendar lahko z ukazom `\sectionpage` znotraj okolja `frame` ustvarimo drsnico, ki prikazuje naslov razdelka.

Druga monost je ponatis kazala na zaetku vsakega poglavja, da bralce spomnimo, kje smo. To lahko storimo z uporabo ukazov `\AtBeginSection[]` in `\tableofcontents[currentsection]` pogledajte Example ??.

e elimo, da se v kazalu prikae samo razdelek (in ne podrazdelek), dodamo ukaz `\tableofcontents` opcijo `hidesubsections`.

Example 0.1.57.

Poleg ukazov `\section` in `\subsection`, Beamer podpira tudi ukaz `\part{}` za veje logine enote. Vsak del predstavitev (definiran z `\part{}`) lahko zane z drsnico, ki prikazuje naslov dela z uporabo ukaza `\partpage` v okolju `frame`. Poleg tega, ima vsak del lahko svoje kazalo vsebine; pogledajte Example ??.

Example 0.1.58.**Specifikacije prekrivanja**

Izhodna datoteka Beamer je obiajno v formatu PDF kar je v principu statina datoteka. Beamer pa omogoča ustvarjanje dinaminih uinkov z uporabo *specifikacij prekrivanja* (ang. *overlay specifications*).

Predstavitev Beamer sestoji iz serie drsnic, vsaka drsnica pa je lahko razdeljena na ve slojev oz. stran (ang. *slides*). V izhodni PDF datoteki vsak sloj ustreza eni strani. Specifikacije prekrivanja so ukazi, ki določajo, kateri deli vsebine so prikazani na doloenem sloju.

Najlaji nain za uporabo specifikacij prekrivanja je z uporabo ukaza `\pause`, ki ustvari prekinitev na trenutni toki v drsnici. e dodamo ukaz `\pause` v okolje `frame`, prvi sloj drsnice prikae vsebino do prva uporaba ukaza `\pause`, drugi sloj prikae vsebino do druge uporabe ukaza `\pause` in tako naprej. Poglejte Example ??.

Example 0.1.59.

Ukaz `\pause` je enostaven za uporabo, vendar ponuja omejene monosti prilagajanja. Za bolj natanno kontrolo nad tem, kdaj se posamezni deli vsebine prikaejo, lahko uporabimo *specifikacije prekrivanja* znotraj ukazov kot so `\textbf`, `\textit`, `\textcolor` in druge (poglejte Table ??).

Sintaksa za specifikacije prekrivanja je naslednja:

```
\ukaz<spec_prek>{<besedilo>}
```

Kjer `\ukaz` predstavlja ukaz, ki ga elimo nadzorovati (npr. `\textbf`), `spec_prek` pa določa, na katerih slojih bo ukaz uporabljen. Na primer, `\textbf<2>{<besedilo>}` bo prikazalo `<besedilo>` krepko le na drugem sloju drsnice (ostali sloji bodo prikazali `<besedilo>` v obiajnem slogu); pogledajte Example ??.

Example 0.1.60.

Nekatere ukaze, kot so `\item` v okolju `itemize` in `enumerate`, imajo vgrajeno podporo za specifikacije prekrivanja; pogledajte Example ??.

Example 0.1.61.

Ukaz `\includegraphics` podpira tudi specifikacije prekrivanja, kar omogoča prikazovanje razlinih slik na razlinih slojih drsnice. To je uporabno za simulacijo animacij z zaporednim prikazovanjem slik; pogledajte Example ??.

Example 0.1.62.

`stick.zip` vsebuje slike, uporabljene v zgornjem primeru.

Naslednje tri ukaze delujejo zelo podobno. Prikažejo `<besedilo>` v skladu z `<spec_prek>`. Razlika med njimi je v tem, kako se obnašajo, ko besedilo ni prikazano.

- `\only<spec_prek>{<besedilo>}`: Prikaže `<besedilo>` samo na slojih, določenih z `<spec_prek>`. Na drugih slojih `<besedilo>` ni prisotno (ne zavzame prostora).
- `\visible<spec_prek>{<besedilo>}`: Prikaže `<besedilo>` na slojih, določenih z `<spec_prek>`, in na drugih slojih ohrani prostor, ki ga `<besedilo>` zavzema (vendar je **nevidno**).
- `\uncover<spec_prek>{<besedilo>}`: Prikaže `<besedilo>` na slojih, določenih z `<spec_prek>`, in na drugih slojih ohrani prostor, ki ga `<besedilo>` zavzema (vendar je **prozoren**).

Razlika med `\visible` in `\uncover` je odvisna od uporabe *prosojnosti* (ang. *transparency*). To se nadzoruje z ukazom `\setbeamercovered{<monost>}` v preambuli dokumenta. Monosti vključujejo:

- `invisible`: Besedilo je popolnoma nevidno (privzeta monost).
- `transparent=<neprosojnost>`: Besedilo je prikazano z določeno stopnjo prosojnosti: `<neprosojnost>` je vrednost med 0 (popolnoma prozorno) in 100 (popolnoma neprozorno); privzeta vrednost je 15.
- `dynamic`: Vse prekrita besede postanejo precej prosojne, vendar na dinamičen način. Daljši čas potreben za odkritje besedila, morda je prosojnost. Poglejte Example ??.

Example 0.1.63.

Obstajajo še drugi ukazi za specifikacije prekrivanja, kot so:

- `\invisible<spec_prek>{<besedilo>}`: Tono nasprotje ukaza `\visible`. Prikaže `<besedilo>` na slojih, določenih z `<spec_prek>`, in na drugih slojih besedilo ni prisotno.
- `\alt<spec_prek>{<besedilo1>}{<besedilo2>}`: Prikaže `<besedilo1>` na slojih določenih z `<spec_prek>`, in `<besedilo2>` na ostalih slojih.
- `\temporal<spec_prek>{<besedilo1>}{<besedilo2>}{<besedilo3>}`: Prikaže `<besedilo1>` pred prvi sloji določenimi z `<spec_prek>`, `<besedilo2>` na slojih določenih z `<spec_prek>`, in `<besedilo3>` po zadnji sloji določenih z `<spec_prek>`.

V Example ?? so prikazani ti ukazi v akciji.

Example 0.1.64.

Nekatere okolje v Beamerju podpirajo specifikacije prekrivanja. Sintaksa je običajno `\begin{okolja}<spec_prek>` in rezultat je, da se celotno okolje prikaže samo na slojih določenih z `<spec_prek>`.

Vsak od ukazov `\only`, `\alt`, `\visible`, `\uncover` in `\invisible` ima svojo različico okolja, in sicer, `onlyenv`, `altenv`, `visibleenv`, `uncoverenv` in `invisibleenv`. Vsi razen `altenv` ima sintakso

```
\begin{okolja}<spec_prek>
  <vsebina>
\end{okolja}
```

in delujejo podobno kot njihovi ukazni ekvivalenti. Okolje `altenv` ima sintakso

```
\begin{altenv}<spec_prek>
{<begin_privzeto>}{<end_privzeto>}
{<begin_alternativno>}{<end_alternativno>}
  <vsebina>
\end{altenv}
```

V določenih slojih določenih z `<spec_prek>`, se `<vsebina>` obdani med `<begin_privzeto>` in `<end_privzeto>`, medtem ko se v ostalih slojih prikaže vsebina obdana z `<begin_alternativno>` in `<end_alternativno>`.

Lahko uporabimo ukaze kot je `\only` za dinamično spremeniti vsebino drsnice, npr.

```
\only<1>{Besedilo za prvi sloj.}
\only<2>{Besedilo za drugi sloj, ki je lahko dalje. e je besedilo dalje, se okvir prilagodi}
\only<3>{Tretji sloj ima spet kraje besedilo.}
```

Teava pri tem pristopu je, da lahko pride do nadlenih razlik v viini elementov, kar lahko povzroci, da se celoten okvir »trese« od sloja do sloja.

Za reitev tega problema lahko uporabimo okolje `overlayarea`, ki ima naslednjo sintakso:

```
\begin{overlayarea}{<irina>}{<viina>}
<vsebina>
\end{overlayarea}
```

To okolje ustvari obmoje `<irina>` krat `<viina>`, v katerem se vsebina lahko dinamino spreminja brez vpliva na viino okvirja drsnice; pogledjte Example ??.

Example 0.1.65.

Vsebina predstavitve

Seznami Navadna okolja za sezname, kot so `itemize`, `enumerate` in `description`, so podprta v Beamerju.

Seveda lahko uporabimo tudi ukaz `\pause` ali specifikacije prekrivanja kot prikazano v Example ?? za postopno prikazovanje elementov seznama. Druga monost je uporaba specifikacije [`<+>`] v okolju seznama, kar samodejno prikae vsak element na naslednjem sloju; pogledjte Example ??.

V tem primeru tudi pokaemo uporabo ukaza `\vspace{<viina>}` za ustvarjanje navpinega prostora med seznami in ukaza `\fill`, ki potisne vsebino navzdol do dna okvirja drsnice.

Example 0.1.66.

e je seznam predolg, da bi ga lahko prikazali na eni drsnici. V tem primeru lahko uporabimo pristop, ki ga prikazuje Example ??.

Example 0.1.67.

Stolpci Beamer omogoja ustvarjanje stolpcev z uporabo okolja `columns`. Vsak stolpec je definiran z okoljem `column`, ki ima naslednjo sintakso:

```
\begin{columns}
\begin{column}<spec_prek>{<irina>}
<vsebina stolpca 1>
\end{column}
\begin{column}<spec_prek>{<irina>}
<vsebina stolpca 2>
\end{column}
...
\end{columns}
```

Kjer `<irina>` doloa irino stolpca (lahko je absolutna vrednost, npr. `5cm`, ali relativna vrednost, npr. `0.5\textwidth`), `<spec_prek>` pa so opsijske specifikacije prekrivanja, ki določajo, na katerih slojih bo stolpec prikazan. Pogledjte Example ??.

Example 0.1.68.

Ta pristop lahko uporabimo za ustvarjanje drsnic z besedilom na eni strani in sliko na drugi strani; pogledjte Example ??.

Example 0.1.69.

Tabele Na splono veina ukazov, ki smo jih opisali v poglavju [Osnovna okolja](#) za ustvarjanje tabel v LaTeX-u deluje tudi v Beamerju, dokler naložimo ustrezne pakete (npr. `booktabs`, `array`, `multrow`).

Poleg tega lahko uporabimo specifikacije prekrivanja za postopno prikazovanje vrstic ali stolpcev v tabeli; pogledajte Example ??.

Example 0.1.70.

e je tabela preiroka ali predolga za drsnico, lahko uporabimo okolje `resizebox` iz paketa `graphicx`. Sintaksa je `\resizebox{<irina>}{<višina>}{<vsebina>}`, kjer lahko uporabimo `!` za samodejno prilagoditev višine ali irine glede na drugo dimenzijo. Pogledajte Example ??.

Example 0.1.71.

Bloki Beamer ponuja okolje `block` za ustvarjanje poudarjenih vsebinskih blokov znotraj drsnic. Sintaksa je naslednja:

```
\begin{block}{<naslov bloka>}
  <vsebina bloka>
\end{block}
```

Poleg tega Beamer ponuja posebne vrste blokov, kot so `exampleblock` in `alertblock`, ki imajo razline sloge za poudarjanje vsebine. Slog bloka je odvisen od izbrane teme Beamerja. Pogledajte Example ??.

Example 0.1.72.

Okolje za teoreme, kot so `theorem`, `lemma`, `definition`, `proof`, itd. so prav tako podprta v Beamerju in imajo slog, ki je podobno kot bloki. Pogledajte Example ??.

Example 0.1.73.

Lahko tudi definiramo svoje matematične bloke z uporabo ukaza `\newtheorem` tono tako, kot v običajnem LaTeXu.

Teme in barvne sheme

Beamer ponuja razline teme in barvne sheme za prilagoditev videza predstavitve. Teme določajo splono postavitev in slog drsnic, in barvno paleto.

Obstajajo trije parametri za prilagoditev videza predstavitve v Beamerju:

- Tema barv (ang. `color theme`): Določa barvno shemo predstavitve.
- Tema pisave (ang. `font theme`): Določa slog pisave.
- Notranje teme (ang. `inner theme`): Določa slog notranjih elementov, kot so sezname, bloki, kazalo, itd.
- Zunanje teme (ang. `outer theme`): Določa slog zunanjih elementov, kot so glave, noge, okvirji drsnic, itd.

Poleg teh tiri parametrov lahko izberemo tudi celotno temo, ki združuje vse te vidike v eno samo temo. Večinoma asa, je dovolj, da izberemo samo temo, saj bo ta samodejno nastavila ustrezne barvne sheme, pisave in notranje ter zunanje teme in e elimo, lahko dodatno prilagodimo posamezne parametre.

Teme se naložijo z ukazom `\usetheme{<ime_teme>}`, medtem ko se posamezni parametri naložijo z ukazi

- `\usecolortheme{<ime_barvne_sheme>}`
- `\usefonttheme{<ime_pisave>}`
- `\useinnertheme{<ime_notranje_teme>}`
- `\useoutertheme{<ime_zunanje_teme>}`

tevilno monosti za te ukaze je ogromno in nemogoče je prikazati vse relevantne primere. Lahko pa ponudimo zunanje vire za raziskovanje razlinih tem.

- Beamer Theme Matrix: Interaktivno orodje za raziskovanje razlinih kombinacij tem in barvnih shem v Beamerju.
- Beamer Theme Gallery: Galerija razlinih tem Beamer, tem pisave in tem barv.
- Beamer User Guide: Del III vodia za uporabnike Beamerja vsebuje podrobne informacije o temah in barvnih shemah.
- Overleaf beamer templates: Sodobne teme Beamer na Overleafu.

Vaje

0.2 GeoGebra

Table 8: Primerjava razlinih aplikacij Geogebra

aplikacije / funkcije	Scientific	Graphing	Geometry	3D	CAS	Suit	Classic
Numerini izrauni Funkcije Operacije z ulomki Grafino prikazo- vanje Drsniki Vektorji in matrike Tabele Geometrijske konstruk- cije 3D prika- zovanje Verjetnost in statis- tika Odводи in integrali Reevanje enab Simbolni izrauni Spreadsheets							

0.2.1 GeoGebra: Uvod

Kaj je Geogebra?

Geogebra je brezplačno dinamično matematično orodje, ki omogoča raziskovanje in vizualizacijo različnih matematičnih konceptov.

Trenutno, Geogebra je popoln paket za učenje in poučevanje matematike. Ponuja različne aplikacije, ki pokrivajo področja, kot so geometrija, algebra, statistika in kalkulus. Vse te aplikacije je mogoče namestiti lokalno ali dostopati do njih prek spleta:

- Scientific Calculator: Za izraune z ulomki, statistiko in osnovnimi matematičnimi funkcijami.
- Graphing Calculator: Za vizualizacijo enab in funkcij z interaktivnimi grafikoni.
- Geometry: Za dinamične geometrijske koncepte in konstrukcije v ravnini.
- 3D Calculator: Za raziskovanje tridimenzionalnih oblik in površin.
- CAS Calculator: Za simbolne izraune in reevanje enab.
- Calculator suit: Kombinacija različnih orodij za širok spekter matematičnih nalog. Idealna za uence in študente.
- Geogebra Classic: Vse-v-enem aplikacija, ki združuje vse zgoraj omenjene funkcije. Idealna za avtorije unih gradiv.

Različne možnosti, ki jih ponuja vsako od orodij, so navedene v Table ??.

Mi bomo v nadaljevanju uporabljali predvsem Geogebra Classic, saj omogoča uporabo vseh orodij na enem mestu.

Kako zanemo z Geogebro?

Geogebra lahko uporabimo na dva naina: prek spletnega brskalnika ali z namestitvijo aplikacije na računalnik (ali telefon/tablet).

V vsakem primeru bomo uporabili račun Geogebra, ki nam omogoča shranjevanje naših delovnih zvezkov v oblak in dostop do njih od kjerkoli.

Zdaj imate dve možnosti za začetek uporabe Geogebre:

1. Uporaba spletnega brskalnika:

- Obiščite Geogebra Classic.
- Kliknite na gumb  *Menu* v zgornjem levem kotu.
- Prijavite se s svojim računom z klikom na  *Sign in*.

2. Namestitev aplikacije:

- Prenesite aplikacijo Geogebra Classic za vaš operacijski sistem z uradne strani za prenos.
- Namestite aplikacijo in jo odprite.
- Prijavite se s svojim računom z klikom na gumb  *Menu* v zgornjem levem kotu in nato na  *Sign in*.

Ko odprete Geogebro in se prijavite, boste videli glavno delovno okolje. Privzeto je prikazi 

Graf (ang. 

Graphing perspective), ki omogoča risanje funkcij in geometrijskih objektov. Lahko jo vidite tudi spodaj:

Opomba.



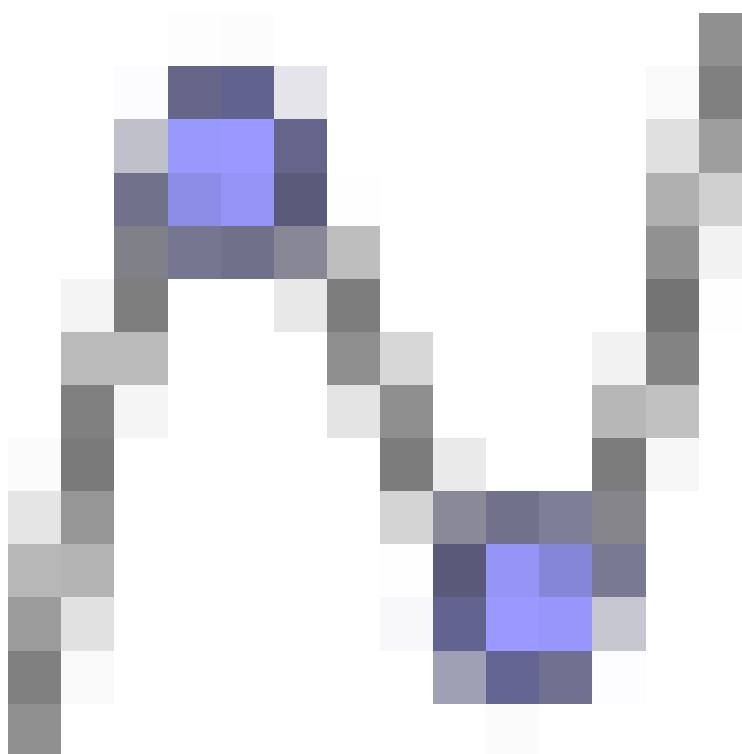
Graphing perspective je eden izmed več prikazov, ki jih ponuja Geogebra. Vsaka aplikacija Geogebre je sestavljena iz različnih prikazov, tako da lahko ustvarimo gradiva za različne aplikacije. Različne prikaze lahko izberemo s klikom na gumb 

Menu v zgornjem levem kotu in nato izbiramo možnost  *Perspectives*.

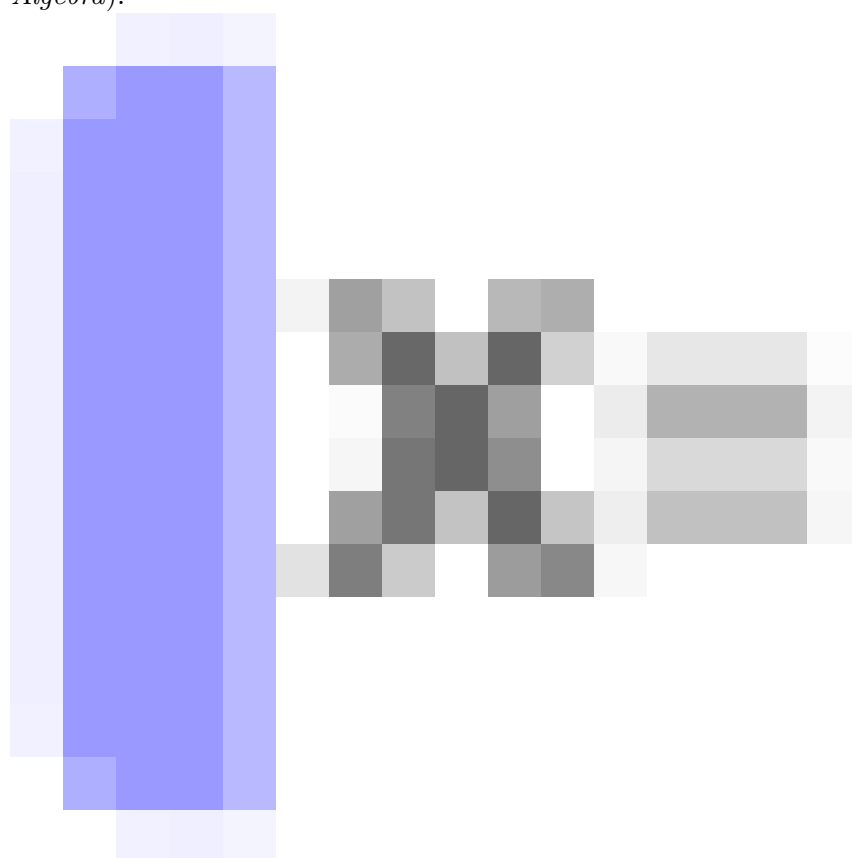
Vsak prikaz ponuja različne *pogled* (ang. *views*). Lahko jih dodamo ali odstranimo glede na naše potrebe s klikom na gumb 

Menu v zgornjem levem kotu in nato izbiramo možnost  *View*. Različni pogledi vključujejo:

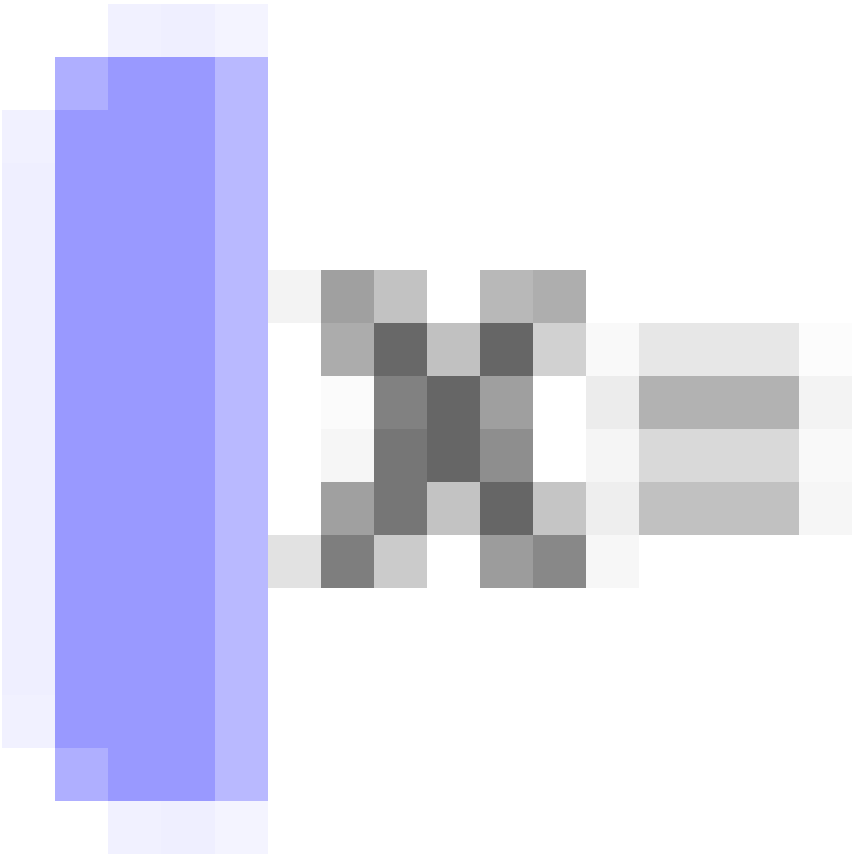
- 



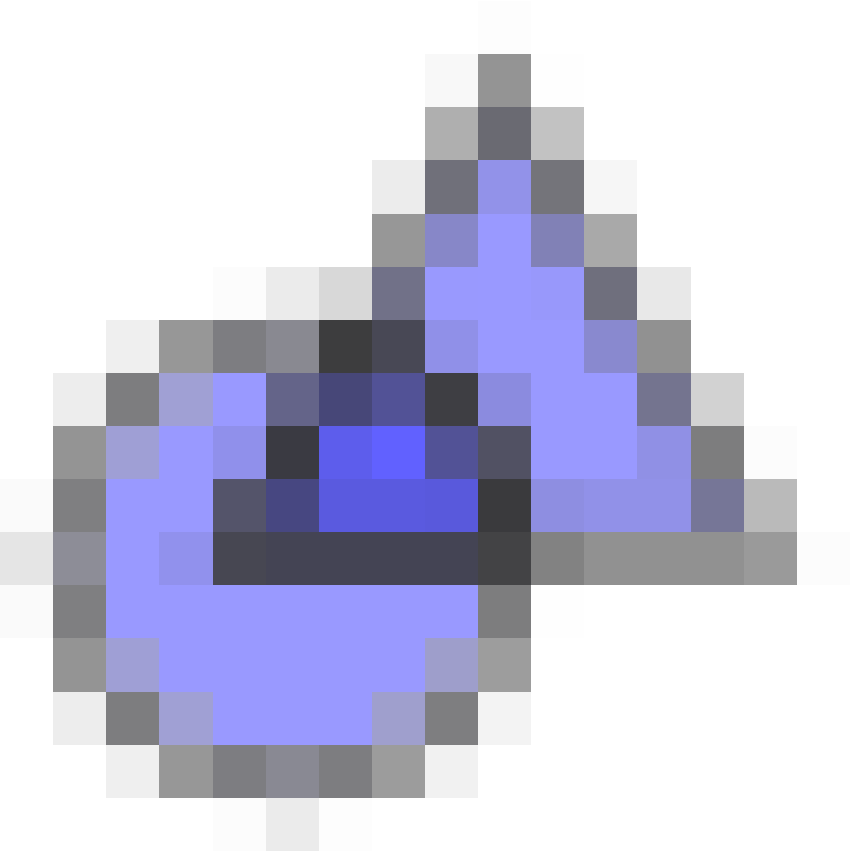
*Algebrsko okno (ang.
Algebra).*



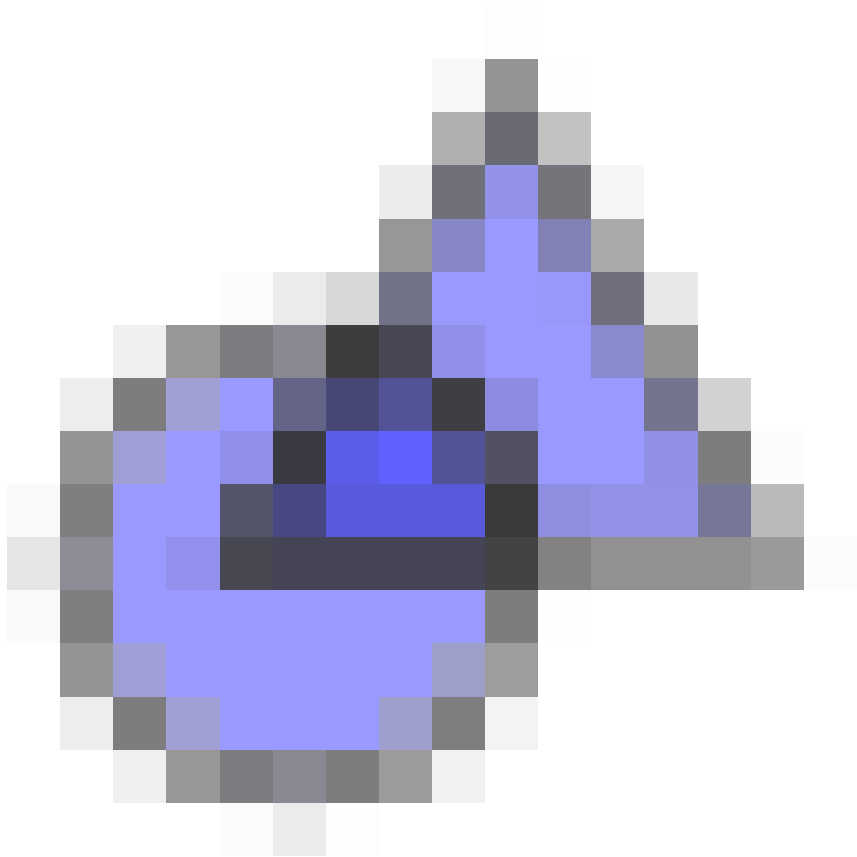
•



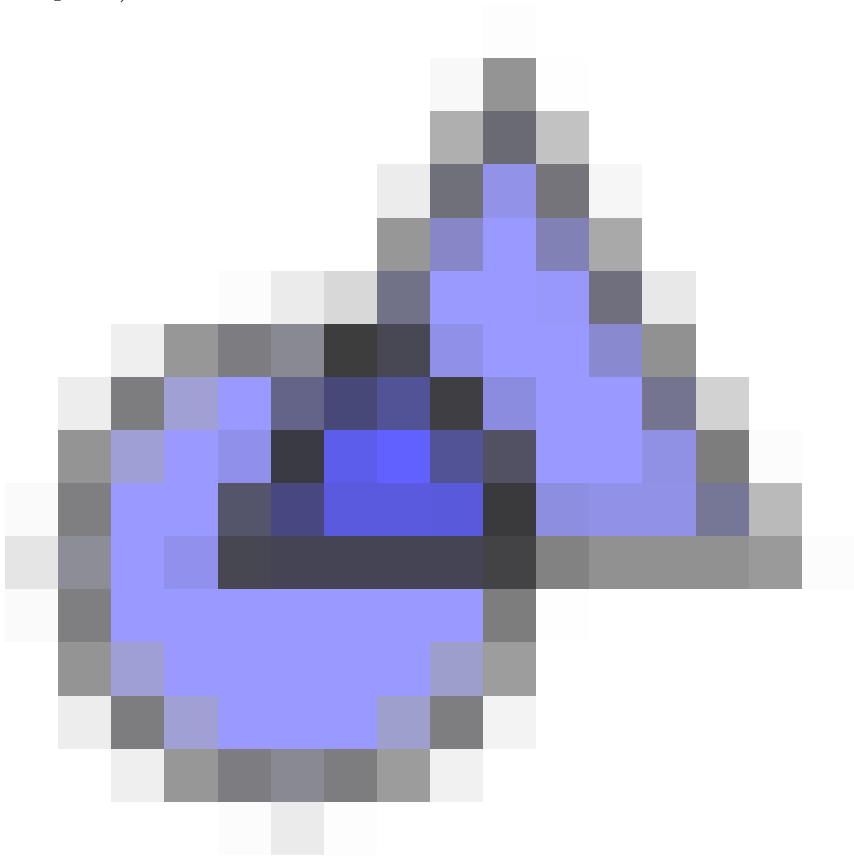
Simbolno raunanje (ang. CAS).



•



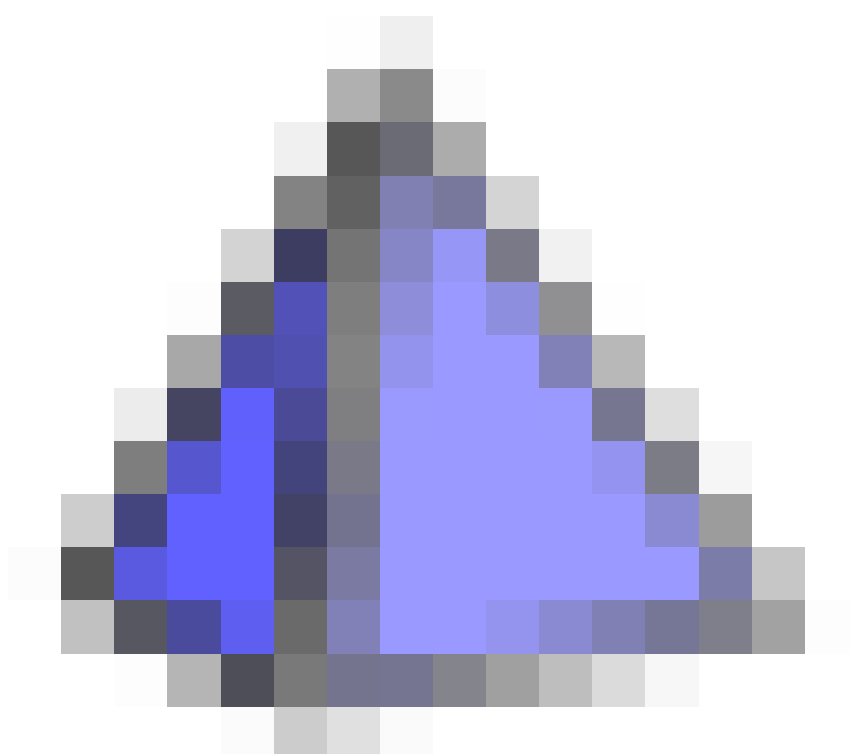
Risalna povrina (ang. Graphics).

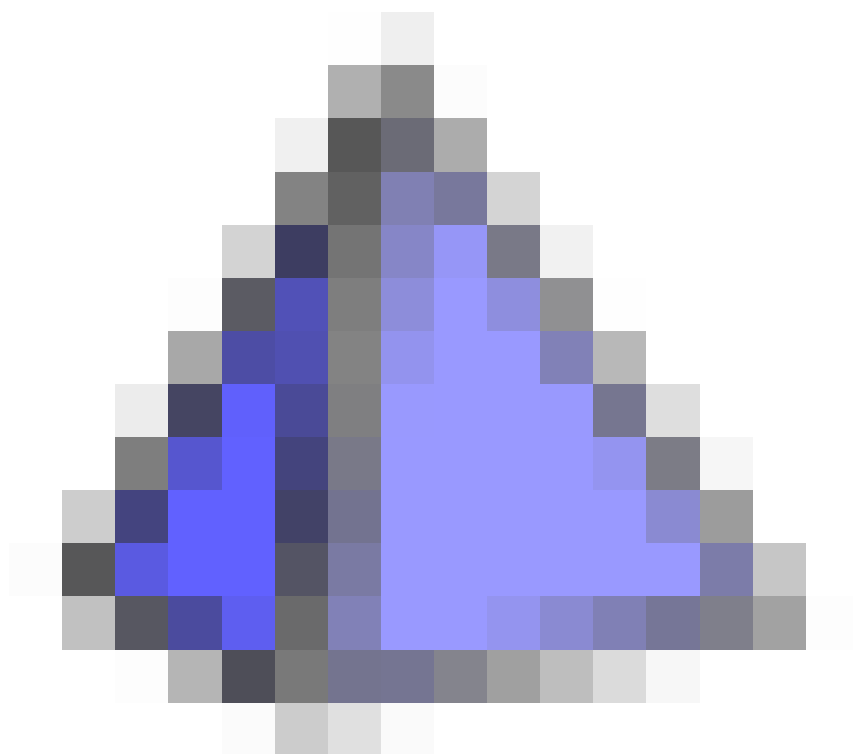


•

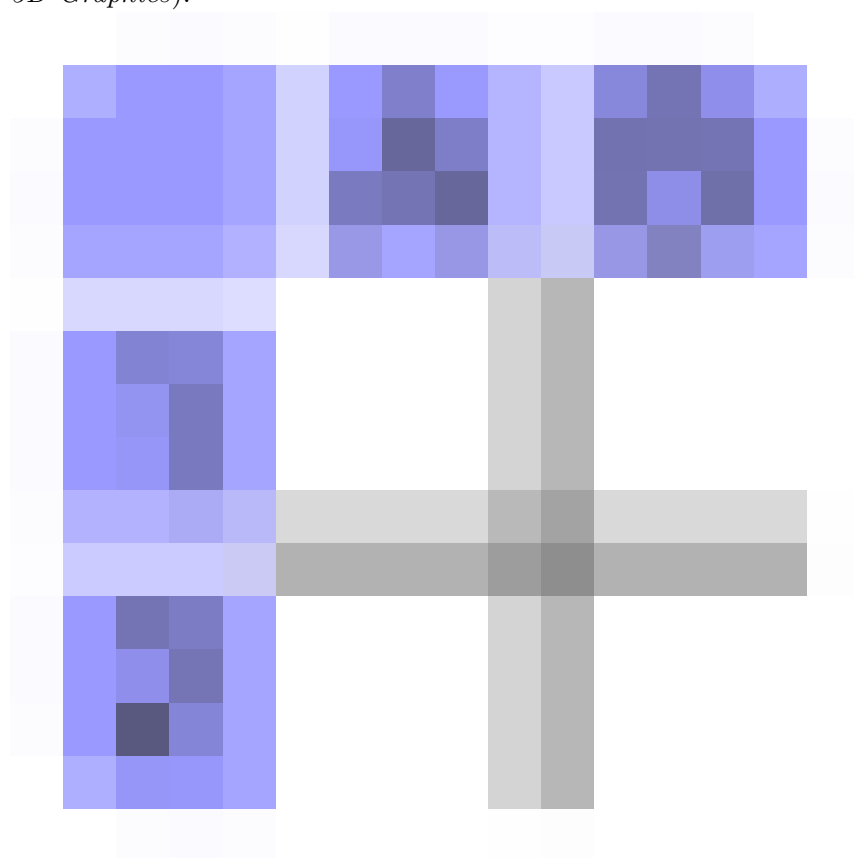


*Grafika 2 (ang.
Graphics 2).*

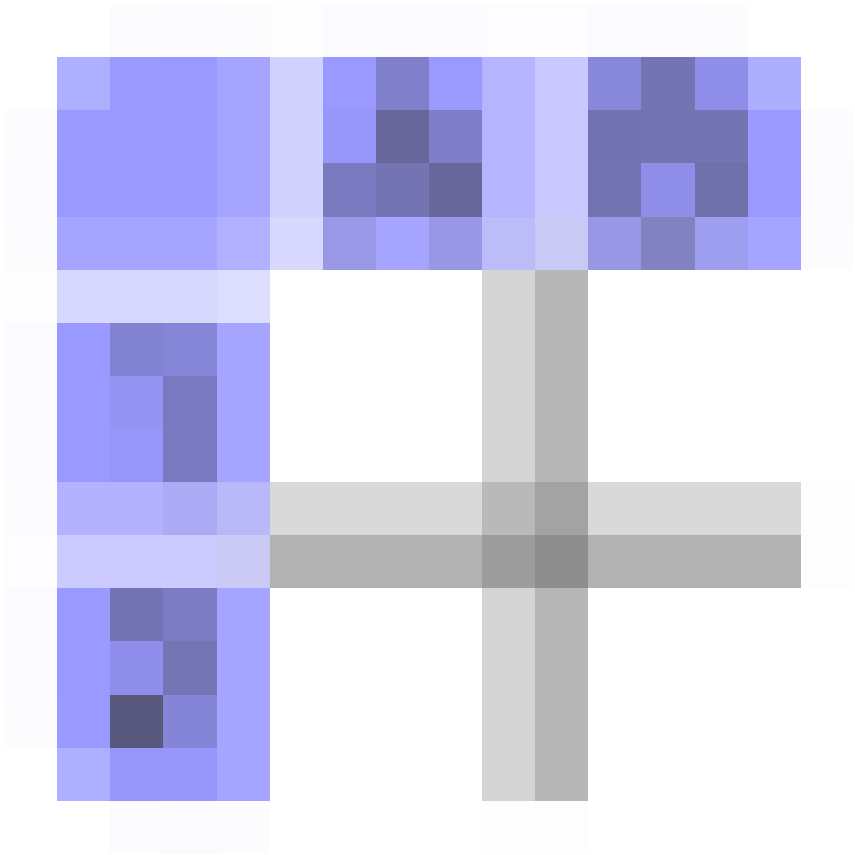




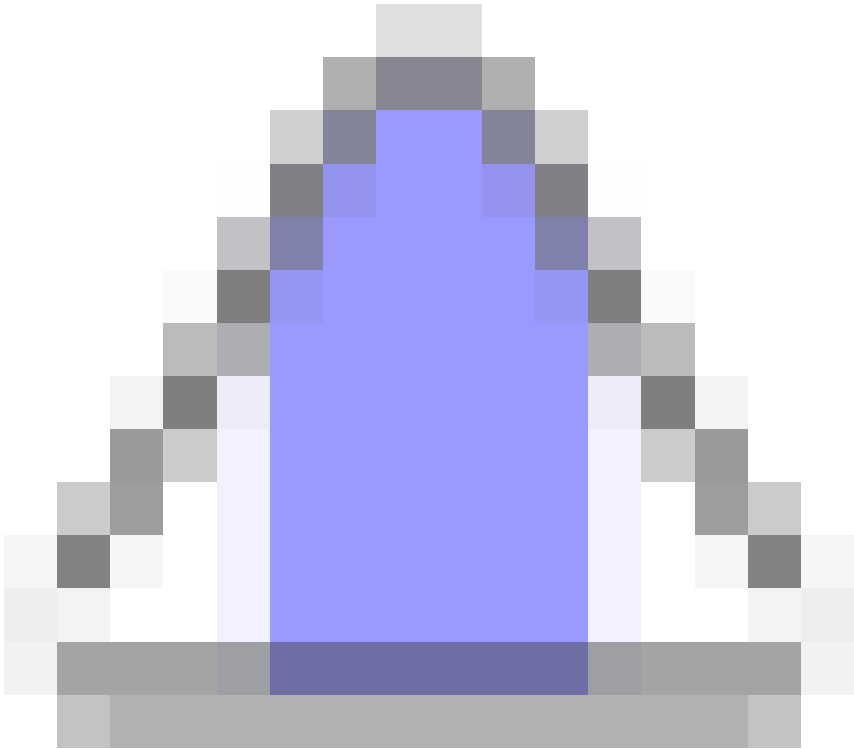
*3d grafika (ang.
3D Graphics).*



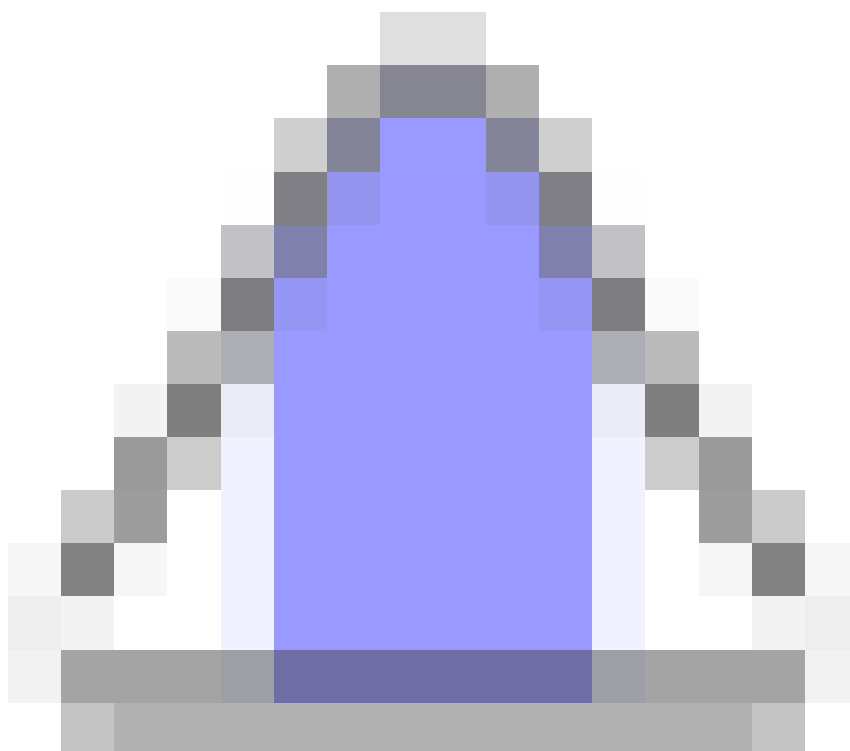
•



*Preglednica (ang.
Spreadsheet).*



•



Raunanje verjetnosti (ang.
Probability Calculator).

Drugi vnosi nad gumbom 

Menu omogoajo dostop do razlinih orodij in nastavitev Geogebre, kot so:

-  *Datoteka* (ang.  *File*): Ustvarjanje, odpiranje, shranjevanje in izvoz datotek.
-  *Uredi* (ang.  *Edit*): Razveljavi, ponovi, kopiraj, prilepi in druge osnovne funkcije urejanja.
-  *Nastavitve* (ang.  *Settings*): Prilagajanje nastavitev Geogebre, kot so jezika, velikosti pisave in drugih monosti.

Orodarne (ang. *Toolbar*):



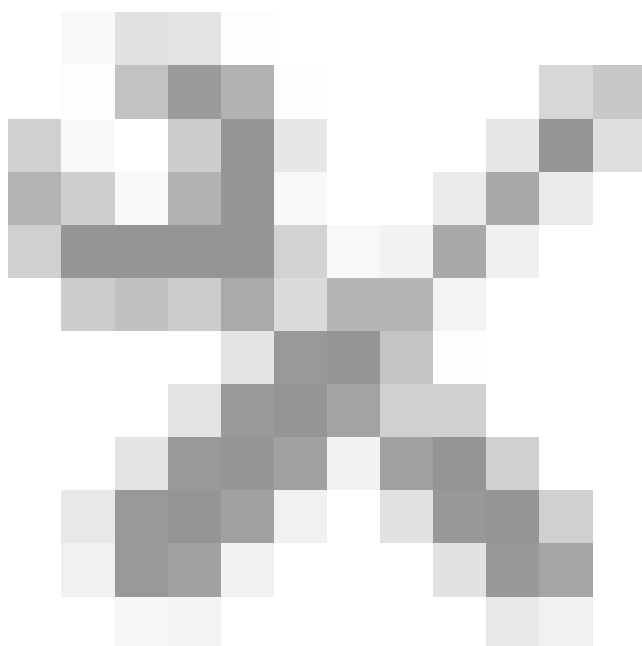
Slog vrstica (ang. *Stylebar*):



Razveljavi (ang. *Undo*):



Ponovno naredi (ang. *Redo*):



- *Orodja* (ang. 
Tools): Dostop do razlinih orodij za risanje in konstrukcije.
-  *Pomo* (ang. 
Help and Feedback): Dostop do dokumentacije, vadnic in podpore skupnosti Geogebra.
-  *Raun* (ang. 
Account): Prijava in odjava iz rauna Geogebra.

Opomba

Geogebra se lahko uporablja v slovenskem jeziku, vmesnik je preveden, prav tako pa tudi ukazi, al pa ni dokumentacije v slovenskem jeziku za ukaze (ki jih uporabljamo za ustvarjanje gradiva). Zaradi tega bomo vmesnik ohranili v angleini, vendar upotevajte, da e elite Geogebro uporabljati z otroki ali uenci, lahko jezik spremenite v meniju 

Settings.

Poleg gumba 

Menu, pomembni del orodjarne so tudi:

Orodje

Geogebra ponuja širok nabor orodij za ustvarjanje in manipulacijo matematičnih objektov. Orodja so dostopna prek orodjarne (ang. *Toolbar*), ki se nahaja običajno na vrhu zaslona. Orodjarna je odvisna od izbranega prikaza in ponuja različna orodja glede na kontekst. Za zdaj, mi bomo raziskali nekaj osnovnih orodij, ki so na voljo v privzetem prikazu .

Graphing in v prikazu .

Graphics, in sicer:



V nadelevanju opisujemo nekatere najpogostejše uporabljena orodja, celoten seznam orodij pa najdete v dokumentaciji Geogebre.

Seveda, najlažji način za spoznavanje orodij je, da jih preizkusite sami v Geogebri!

Vrstica za zamenjavo sloga



Vrstica za zamenjavo sloga (ang. *Stylebar*) omogoča hitro spreminjanje grafičnih lastnosti izbranih objektov, kot so barva, debelina črte, slog črte in polnilo. Ko izberete objekt v grafinem oknu, se vrstica za zamenjavo sloga prikaže pod orodjarno in ponuja različne možnosti za prilagajanje videza izbranega objekta. Možnosti so odvisne od vrste izbranega objekta, e ni objekta izbranega, vrstica za zamenjavo sloga prikaže možnosti sloga za pogled.

V vrstici za zamenjavo sloga se nahajajo samo osnovne možnosti, za dostop do naprednih možnosti sloga pa lahko kliknete na gumb .

Nastavitve (ang. *Settings*), ki odpre pogovorno okno z več možnostmi za prilagajanje sloga izbranega objekta. e ni izbranega objekta, gumb .

Nastavitve odpre pogovorno okno z možnostmi za prilagajanje sloga pogleda.

Ukaze

GeoGebra poleg grafičnih orodij ponuja tudi algebrske vnose in ukaze. Vsako orodje ima ustrezen ukaz, zato ga je mogoče uporabljati brez miške.

Nasvet

Dokumentacija za ukaze je v angleščini, zato je priporočljivo, da imate ukaze uporabite v angleščini. Ve o ukazih in njihovi uporabi najdete v dokumentaciji Geogebre.

Opomba

Vsaka orodja ima svoj ukaz, obratno pa ne velja, obstajajo več ukazov kot orodij. Veina uporabnikov Geogebra lahko brez težav dela z grafinimi orodji, vendar moramo avtorji znati uporabljati ukaze. Tako bomo lahko delali hitreje in natančneje.

e hitro uporabimo ukaz, lahko ga vnesemo v *Vrstico za vnos* (ang. *Input Bar*), ki se običajno nahaja na dnu zaslona. Alternativno, lahko ukaze vnesemo tudi neposredno v grafino okno ali v .

Algebrsko okno (ang. *Algebra View*). Oboje lahko najdemo v meni ju .

Views.

Nalaga način za spoznavanje ukazov je, da jih preizkusite sami v Geogebri!

Nasvet

Ko uporabite ukazi v Geogebri, je pomembno, da upoštevate naslednje nasvete:

- Toke so definirane z velikimi rkami (npr. A, B, C), medtem ko so premice, daljice in kronice definirane z malimi rkami (npr. a, b, c).
- Pri vnosu koordinat tok uporabite oklepaje in vejice, npr. (2,3) za toko z x koordinato 2 in y koordinato 3.
- Ukaz $v = (1, 2)$ ne definira toke, ampak vektor z komponentama 1 in 2.
- Uporabite za samodejno dokonjanje imen ukazov in objektov.
- Uporabite in za navigacijo po zgodovini ukazov v vrstici za vnos.
- Z gumbom  *Help* (sl. *Pomo*) (na desno od vrstice za vnos) lahko dostopate do dokumentacije Geogebre za ukaze.

V Exercise ?? ste se nauili

- Ustvariti geometrijske objekte z uporabo ukazov v vrstici za vnos.
- Meriti kotov in dolin in z uporabo orodje.
- Prilagoditi besedilo z uporabo LaTeX formul.
- Nastavitev **Starting point** za pozicioniranje besedila.

Geogebra nam omogoča vizualizacijo osnovnih geometrijskih konstrukcij in iskanje vzorcev v njih. Na primer, pri konstrukciji v Exercise ?? lahko opazimo, da je trikotnik ABC vedno enakostraničen, ne samo za izbiro toka $A = (0, 0)$ in $C = (4, 0)$, ampak za katerokoli izbiro toka A in C . To lahko vidimo, e povleemo toki A in C z orodjem .

Move. Opazimo da so dolini stranici AB , BC in CA vedno enaki. V Geogebri je to znano kot **test vleenja** (ang. *dragging test*), ki nam pomaga pri odkrivanju in dokazovanju geometrijskih lastnosti.

V prejšnji konstrukciji smo uporabili **test vleenja** (ang. *dragging test*), ki nam pomaga pri odkrivanju in dokazovanju, da je trikotnik ABC vedno enakostraničen. S testom vleenja lahko tudi vidimo, da so vsi koti v enakostraničnem trikotniku enaki in merijo 60° . To sledi iz dejstva, da je vsota notranjih kotov v trikotniku vedno 180° in da so vsi trikotniki enakostranični.

Algebrsko okno, proste toke vs. odvisne toke in kontrolni okvirek

Vrstica za vnos ni edini način za interakcijo z Geogebro z uporabo ukazov. Lahko uporabimo tudi  *Algebrsko okno* (ang. *Algebra View*), ki prikazuje seznam vseh objektov v konstrukciji skupaj z njihovimi lastnostmi in vrednostmi.

Upotevajte, da so toke A in B **proste toke** (ang. *free points*), ker jih lahko premikamo kjerkoli v grafinem oknu. Vse ostale toke v konstrukciji so **odvisne toke** (ang. *dependent points*), ker so določene z drugimi objekti v konstrukciji in se premikajo glede na spremembe teh objektov. Proste toke so običajno modre barve, medtem ko so odvisne toke sive barve. Seveda, to lahko spremenimo z uporabo vrstice za zamenjavo sloga ali v nastavitvah objekta, in torej barva ni zanesljiv indikator vrste toke. V algebrskem oknu lahko vidimo katere objekte so proste in katere odvisne, saj so uporabimo gump *Sort by* in torej *Dependency* v vrstico za zamenjavo sloga algebrskega okna.

Kot smo videli, lahko uporabimo orodje .

Show/Hide Object za prikaz ali skrivanje objektov v grafinem oknu. To lahko storimo tudi v algebrskem oknu, kjer lahko kliknemo na krog levo od imena objekta za prikaz ali skrivanje objekta. Poleg tega, lahko uporabimo tudi kontrolni okvirek (ang. *checkbox*), ki ga lahko ustvarimo z orodjem



Check Box ali z ukazom `CheckBox()`. Kontrolni okvirek vstavirte *Check Box* v grafino okno in booleano vrednost v algebrskem oknu. Nato lahko uporabimo to vrednost za nadzor vidljivosti drugih objektov v konstrukciji.

Izvoz konstrukcij kot slike.

Lahko izvozimo naše konstrukcije iz Geogebre kot slike v različnih formatih, kot so PNG, SVG ali PDF. Za izvoz konstrukcije kot slike, sledite tem korakom:

1. Pojdite na meni 

File in izberite 

Download as....

2. V pogovornem oknu izberite eleni format slike (npr. `.png`, `.svg` ali `.pdf`).
3. Posebni format je `PGF/TikZ`, ki omogoča izvoz konstrukcije kot kode, ki jo lahko uporabimo v LaTeX dokumentih.

Vaje

- Exercise ??: Ustvarjanje rauna Geogebra.
- Exercise ??: Vaje za spoznavanje orodij Geogebre.
- Exercise ??: Vaje za uporabo vrstice za zamenjavo sloga.
- Exercise ??: Vaje za spoznavanje ukazov Geogebre.
- Exercise ??: Vaje za dokazovanje lastnosti trikotnika z uporabo Geogebre.
- Exercise ??: Vaje za dokazovanje vsote notranjih kotov v trikotniku z uporabo Geogebre.
- Exercise ??: Vaje za ustvarjanje petkotnika in zvezde z uporabo Geogebre.
- Exercise ??: Vaje za uporabo kontrolnega okvirka za nadzor vidljivosti objektov v Geogebri.

0.2.2 Dinamina geometrija z GeoGebro

V poglavju [GeoGebra: Uvod](#) smo spoznali osnove dela z GeoGebro. V tem poglavju bomo raziskali, kako ustvariti animacije in dinamine konstrukcije z uporabo GeoGebre.

Toka na objektu

Najlajše nain za ustvarjanje dinamine konstrukcije je uporaba toke, ki je vezana na drug objekt. Toko lahko postavimo na premico, kronico ali katerikoli drug objekt, kar omogoča, da se toka premika znotraj omejitve tega objekta.

Ko ustvarimo toko na krivulji (kot so premice, kronice, graf funkcij itd.), lahko to toko animiramo. To naredimo tako, da izberemo toko in kliknemo na gumb .

Play v algebrskem oknu ali uporabimo ukaz `StartAnimation(<toka>)`. S tem se bo toka zaela premikati po objektu, na katerem je bila ustvarjena.

Na alost GeoGebra ne omogoča animacije tok na daljih objektih, kot so mnogokotniki. Vendar pa lahko ustvarimo drsnike, ki nam omogočajo nadzor nad poloajem toke na taknih objektih s pomojo parametra.

Drsniki

V resnini, drsniki so samo orodja za spreminjanje vrednosti spremenljivk. Vendar pa so zelo uporabni pri ustvarjanju dinaminih konstrukcij, saj nam omogočajo, da spreminjamo vrednosti in opazujemo, kako se konstrukcija spreminja.

Drsniki imajo tri glavne lastnosti: **Min** (minimalna vrednost), **Max** (maksimalna vrednost) in **Increment** (prirastek). Te lastnosti določajo obseg vrednosti, ki jih drsnik lahko zavzame, in kako hitro se vrednost spreminja, ko premikamo drsnik.

Za ustvarjanje drsnika lahko uporabimo orodje .

Slider (Drsnik) in dobimo okno za nastavitve lastnosti drsnika. Drsnike lahko ustvarimo tudi z vnosom imena spremenljivke v vnosno vrstico, na primer $a=5$, kar ustvari spremenljivko z imenom **a** in zaetno vrednostjo 5. Nato lahko kliknemo na tri pike poleg imena spremenljivke v algebrskem oknu in nato *Create slider (Ustvari drsnik)*. To ustvari drsnik z privzetimi lastnostmi (**Min**=-5, **Max**=5, **Increment**=0.1), ki jih lahko spremenimo z dvojnimi klikom na drsnik (v grafinem pogledu) ali z desnim klikom na ime spremenljivke v algebrskem oknu in izbiro *Settings (Nastavitve)*. V oknu z nastavitvami lahko spremenimo lastnosti drsnika, kot so **Min**, **Max** in **Increment**, in tudi druge lastnosti, kot so barva, slog in vidnost.

Drsniki lahko tudi uporabimo za ustvarjanje animacij. To naredimo tako, da izberemo drsnik in kliknemo na gumb .

Play v algebrskem oknu ali uporabimo ukaz `StartAnimation(<drsnik>)`. S tem se bo vrednost drsnika zaela spreminjati samodejno z določenim prirastkom, kar omogoča ustvarjanje animacij v konstrukciji.

Zaporedja

Ukaz `Sequence()` omogoča ustvarjanje zaporedij objektov v GeoGebri. Splona oblika ukaza je `Sequence(<izraz>, <spremenljivka>, <zaetek>, <konec>, <korak>)`, kjer **<izraz>** je izraz (običajno ukaz GeoGebre), ki ga elimo ponoviti, **<spremenljivka>** je spremenljivka, ki se spreminja v zaporedju, **<zaetek>** in **<konec>** določata obseg vrednosti za spremenljivko, in **<korak>** določa, za koliko se spremenljivka poveča v vsakem koraku. Variantne ukaza so:

- `Sequence(<konec>)` - ustvari zaporedje tevil od 1 do **<konec>**.
- `Sequence(<zaetek>, <konec>)` - ustvari zaporedje tevil od **<zaetek>** do **<konec>**.
- `Sequence(<izraz>, <spremenljivka>, <zaetek>, <konec>)` - ustvari zaporedje z privzetim korakom 1.

Ukaz `Sequence()` ustvari zaporedje objektov, e hoemo izbrati posamezne objekte iz zaporedja, lahko uporabimo ukaz `Element(<zaporedje>, <indeks>)`, kjer **<zaporedje>** je zaporedje objektov in **<indeks>** je indeks objekta, ki ga elimo izbrati (indeksi se zanejo pri 1).

e hoemo izgraditi ve objektov (namesto zaporedje objektov), lahko uporabimo naslednje nain:

- Ustvarimo zaporedje objektov z ukazom `Sequence()`, npr. `kroznice = Sequence(Circle((0,0), i), i, 1, 5)`, ki ustvari zaporedje kronic s polmeri od 1 do 5.
- Nato uporabimo ukazi `Sequence()` in `Element()` za ustvarjanje zaporedja ukazov, npr. `kroU=Sequence("K_{"+i+"} = Element(kroznice, "+i+")", i, 1, 5)`, ki ustvari zaporedje ukazov za vsako kronico v zaporedju z imenom `krogU`. V tem primeru ustvarimo ukaze `K_{1} = Element(kroznice, 1)`, `K_{2} = Element(kroznice, 2)`, itd.
- Na koncu uporabimo ukaz `Execute()` za izvedbo vsakega ukaza.

Ta nain lahko uporabimo za ustvarjanje zaporedje ukazov, da ni nujno ustvariti objekte. Na primer z ukazom `barv=Sequence("SetDynamicColor(L_{"+i+"}, random(), random(), random(), 0.5)", i, 1, 40)`, ustvarimo zaporedje ukazov za nastavitve nakljune barve za vsako daljico, nato pa z ukazom `Execute(barv)` izvedemo te ukaze in tako nastavimo barve za vse daljice naenkrat. *Pozkusite to sami!*

Zaporedje in drsniki postanejo zelo mona orodja, ko jih kombiniramo skupaj. Na primer, lahko ustvarimo drsnik, ki doloatevilo objektov v zaporedju, in nato uporabimo ta drsnik za ustvarjanje dinamne konstrukcije, ki se spreminja glede na vrednost drsnika. Ta nain bomo raziskali v naslednjih vajah.

Vaje

- Exercise ?? - Ustvarite dinamini paralelogram in opazujte, kako se spreminja njegova površina.
- Exercise ?? - Ustvarite premico z drsniki za naklon in preseie z osjo y ter opazujte, kako se spreminja premica.
- Exercise ?? - Ustvarite dinamino konstrukcijo, ki prikazuje veljavnost trikotne neenakosti s pomojo drsnikov in dinaminih barv.
- Exercise ?? - Ustvarite animacijo z uporabo drsnikov za premikanje tok v ravnini.
- Exercise ?? - Ustvarite dinamino konstrukcijo za setevanje dveh celih tevil z uporabo drsnikov.
- Exercise ?? - Ustvarite zaporedje tok na kronici z uporabo ukaza `Sequence()`.
- Exercise ?? - Uporabite ukaz `Sequence()` za ustvarjanje vzorca premic v GeoGebri.
- Exercise ?? - Uporabite ukazi `Element()` in `Execute()` za ustvarjanje posamezni objektov iz zaporedje.

0.2.3 Analiza z GeoGebro

Sledovi (traces in loci)

Sledovi (ang. *traces*) in loci (ang. *loci*) so uporabna orodja v GeoGebri za analizo geometrijskih in funkcijskih lastnosti. Sledovi omogoajo sledenje poti toke, ki se premika glede na druge objekte, medtem ko loci prikazujejo množico tok, ki izpolnjujejo določene pogoje. Razlika med sledovi in loci je v tem, da sledovi prikazujejo pot posamezne toke, medtem ko loci prikazujejo celotno množico tok, ki izpolnjujejo določene pogoje. Najlažje je, da to ponazorimo z naslednjo konstrukcijo:

Glej tudi

Peucellier-Lipkinov mehanizem je opisan tudi na Wikipediji (ang.). To je mehanizem, ki pretvarja krono gibanje v ravno linijsko gibanje. Uporablja se lahko za risanje ravnih rt brez uporabe ravnila, poleg tega, lahko tudi uporabimo za risanje kronic brez uporabe kompasa.

Lahko tudi uporabimo orodja 

Image (Slika) za uvoz slike make in nato sledimo poti določene toke na sliki. To je prikazano v naslednjem konstrukciji:

e elite, lahko to konstrukcijo odprete v GeoGebri tukaj. **Poskusite sami!**, e elite, lahko uporabite moje slike: wall.png in cat.png.

Funkcije

GeoGebra ima veliko vnaprejšnjih določenih funkcij, ki jih lahko uporabimo za analizo. Nekatere izmed teh funkcij vključujejo: **sin()** (sinus), **cos()** (kosinus), **tan()** (tangens), **exp()** (eksponentna funkcija), **ln()** (naravni logaritem) in e ve. (celoten seznam (v ang.) tukaj) Te funkcije lahko uporabimo za ustvarjanje grafov in izvajanje razlinih analiz.

Za uporaba funkcije v GeoGebri, preprosto vnesite ime funkcije, sledeno z oklepaji, ki vsebujejo argumente funkcije. Na primer, za risanje grafa funkcije $f(x) = \sin(x)$, vnesite **f(x) = sin(x)** v vhodno vrstico in pritisnite .

Lahko tudi ustvarimo funkcije, ki so odvisne od drugih objektov v GeoGebri. Na primer, z ukazom **g(x) = ax³ + bx² + cx + d** lahko ustvarimo funkcijo, ki je odvisna od spremenljivk **a**, **b**, **c** in **d**. Te spremenljivke lahko nato nastavimo na določene vrednosti ali jih povežemo z drsniki za dinamično prilagajanje funkcije.

Nekatera pomembna orodja za delo s funkcijami so:

- Ukaz **Roots(<funkcija>, <min x>, <max x>)** najde nile funkcije na intervalu $[\min x, \max x]$.
- Ukaz **Max(<funkcija>, <min x>, <max x>)** najde maksimum funkcije na določenem intervalu.
- Ukaz **Min(<funkcija>, <min x>, <max x>)** najde minimum funkcije na intervalu $[\min x, \max x]$.

e hitro elimo najti nekatere lastnosti funkcije, lahko uporabimo orodje 

Function Inspector (Lastnosti funkcije). To orodje nam omogoja hitro analizo funkcije in izračun njenih lastnosti, kot so nile, ekstrema, infleksijske toke in e ve.

Ko delamo s funkcijami v LaTeXu, je lahko koristno, da uredimo funkcije, s katerimi delamo. Za to, lahko uporabimo orodja 

Input box (Vhodno polje). Ko ustvarimo vhodno polje, dobimo okno kjer določimo objekt, ki ga elimo prikazati in urejati *Linked Object (Povezani objekt)* in *Caption (Naslov)*, ki bo prikazan poleg vhodnega polja.

Odводи in integrali

Polarni krivulje

GeoGebra omogoja tudi risanje in analizo polarnih krivulj. Polarne krivulje so definirane z enabami v polarnih koordinatah, kjer je vsaka toka določena z razdaljo od izhodia (polmer) in kotom glede na pozitivno smer osi x .

Poleg tega, lahko prikaemo polarno omreje v grafinem oknu z klikom na 

Vrstica za zamenjavo sloga, nato na 

Show grid (Omreje) in izberemo monost 

Polar Grid (Polarno omreje).

Parametrine krivulje

Polarne krivulje so posebna vrsta parametrinih krivulj, kjer je polmer funkcija kota. Vendar pa lahko ustvarimo tudi splone parametrine krivulje v GeoGebri z uporabo ukaza **Curve**. Ta ukaz omogoča definiranje krivulje z uporabo dveh funkcij, ki določata x in y koordinate glede na parameter t . Na primer, ukaz `Curve((cos(t), sin(t)), t, 0, 2*π)` ustvari enoto kronico.

Vaje

- Exercise ?? Ustvarite Peaucellier-Lipkinov mehanizem v GeoGebri in analizirajte sled toke F .
- Exercise ?? Kako se premika maka, ko se lestev zdrsne po tleh?
- Exercise ?? Ustvarite funkcijo in analizirajte njene lastnosti z orodji GeoGebre.
- Exercise ?? Ustvarite konstrukcijo v GeoGebri, ki omogoča prikaz definicije odvoda funkcije.
- Exercise ?? V konstrukciji iz prejnje vaje, dodajte monost za prikaz grafa odvoda funkcije.
- Exercise ?? Ustvarite konstrukcijo v GeoGebri, ki omogoča prikaz določenega integrala funkcije med tokama a in b .
- Exercise ?? Ustvarite polarne krivulje in analizirajte njihove lastnosti z orodji GeoGebre.
- Exercise ?? Ustvarite datoteko GeoGebra, ki prikazuje parametrino krivuljo.
- Exercise ?? Ustvarite datoteko GeoGebra, ki prikazuje Taylorjevo približek funkcije okoli toke a .

0.2.4 Tridimenzionalna geometrija z GeoGebro

GeoGebra 3D omogoča delo s tridimenzionalnimi objekti in njihovo vizualizacijo. V tem poglavju bomo raziskali osnovne funkcije GeoGebre 3D ter kako jih uporabiti za ustvarjanje in analizo tridimenzionalnih geometrijskih oblik.

Za delo v GeoGebri 3D moramo najprej preklopiti na 3D pogled. To storimo tako, da iz menija izberemo “Perspectives” in nato “3D Graphics”. Prikaz “3D Graphics” vključuje tridimenzionalno koordinatno mrežo, ki nam omogoča ustvarjanje in manipulacijo 3D objektov, in tudi algebrsko okno, kjer lahko vidimo seznam vseh ustvarjenih objektov in njihovih lastnosti, podoben kot v 2D pogledu.

Toke v 3D prostoru ustvarimo z orodjem .

Point (Toka). Kliknemo na orodje in nato v 3D prostoru izberemo lokacijo, kjer elimo postaviti toko. Za to, moramo najprej določiti koordinate x in y v ravnini (sivi ravnini na 3D prikazu), nato pa določimo višino z tako, da povlečemo toko navzgor ali navzdol. Ko ustvarimo toko, se njene koordinate prikažejo v algebrskem oknu. Alternativno lahko toko ustvarimo tudi z vnosom njenih koordinat v algebrsko okno, na primer $A = (2, 3, 4)$.

Podobno, za premikanje tok v 3D prostoru uporabimo orodje .

Move (Premakni). Nato lahko preklapljate med dvema načinoma s ponavljajočim klikom na toko, medtem ko je orodje aktivno: premikanje po ravnini (vzporedno v sivo ravnino) in premikanje navpino (v smeri osi z).

Najlajše način za učenje orodje je, da jih raziskujete.

Take in krogle

Vasih je teko vse videti v 3D prostoru. Priporočamo, da eksperimentirate z različnimi pogledi in orodji za navigacijo v GeoGebri 3D. Vse najdete v .

Styling bar (Vrstice za spremenite sloga pogleda). Čeelite rono spremeniti pogled, uporabite orodje .

Rotate View (Zasukaj pogled) ali orodje .

Translate View (Premakni pogled).

Premice in ravnine

Poliedri in drugi 3D objekti

Rotacijska ploskev

Drugi način za prikaz rotacijske ploskve je z orodjem .

Rotate around Line (Vrtenje okoli premice).

Funkcije z ve spremenljivkami.

Dodatne vaje

Vaje

- Exercise ??: Ustvarite dve krogli v GeoGebri 3D in raziskujte njihovo presečino.
- Exercise ??: Kako izmeriti razdaljo medo tokama v GeoGebri 3D?
- Exercise ??: Ustvarite kocko in ravnino v GeoGebri 3D ter poiščite njihovo presečino.
- Exercise ??: Ustvarite razline poliedre in 3D oblike v GeoGebri 3D.
- Exercise ??: Ustvarite rotacijske ploskve v GeoGebri 3D z vrtenjem daljic in kronic okoli osi.
- Exercise ??: Ustvarite rotacijske ploskve z vrtenjem grafov funkcij okoli osi v GeoGebri 3D.
- Exercise ??: Razvijte funkcije z ve spremenljivkami v GeoGebri 3D.
- Exercise ??: Ustvarite konstrukcijo v GeoGebri 3D, ki prikazuje nastanek stonice kot presečino dvojnega stoca in ravnine.
- Exercise ??: Ustvarite konstrukcijo v GeoGebri 3D, ki prikazuje pet platonskih teles.

0.3 SageMath

0.3.1 Uvod v SageMath

Kaj je SageMath?

SageMath (na kratko Sage) je **brezplačen** odprtokodni programski sistem za matematiko. Lahko si ga predstavljate kot Python na matematičnih steroidih. Poslanstvo sistema SageMath je ustvariti učinkovito brezplačno odprtokodno alternativo sistemom Magma, Maple, Mathematica in Matlab.

Glej tudi

SageMath se pogosto uporablja in na spletu je na voljo veliko gradiva. Navajam nekaj najuporabnejih virov.

- SageMath Documentation: <https://doc.sagemath.org/>
- SageMath Reference Manual: <https://doc.sagemath.org/html/en/reference/index.html>
- A tour of SageMath: https://doc.sagemath.org/html/en/a_tour_of_sage/
- Introductory SageMath tutorial: <https://doc.sagemath.org/html/en/prep/Intro-Tutorial.html>
- SageMath tutorial: <https://doc.sagemath.org/html/en/tutorial/index.html>
- SageMath thematic tutorial: https://doc.sagemath.org/html/en/thematic_tutorials/index.html
- SageMath repository of interacts (interaktivni programi): <https://wiki.sagemath.org/interact>
- SageMath quickref cards (reference za pomoč): <https://wiki.sagemath.org/quickref>

Kako uporabljamo SageMath?

SageMath lahko uporabljate na več načinov, vsak od njih ponuja različno raven nadzora in različne funkcije. Glavni načini uporabe so:

- **SageCell** <https://sagecell.sagemath.org/> Zelo uporabno za enostavne in hitre naloge.
- **Lokalna namestitve**. Ni priporočljivo za začetnike. Če nimate izkušenj z namestitvijo programske opreme, je to lahko nekoliko zapleteno. Poleg tega skoraj nikoli ni potrebna. Edini dober razlog za lokalno namestitve programa SageMath je, da nimate dostopa do interneta.
- **Interaktivni programi**. Podobno kot apleti GeoGebra. Za uporabo interaktivnih programov ni treba imeti predznanja. Za njihovo ustvarjanje pa potrebujete napredno poznavanje programa SageMath, ki presega vsebine tega predmeta.
- **CoCalc.com** <https://cocalc.com/>. S preprostimi besedami, CoCalc je za SageMath, kar je Overleaf za LaTeX.

SageMath na CoCalcu

Obstajajo različne načini za uporabo SageMath, najpogostejše so:

- Na ukazni vrstici (terminalu). Ta način je primeren za napredne uporabnike in ni priporočljiv za začetnike. V tem primeru, SageMath deluje podobno kot običajen programski jezik, ukaze napišemo v terminal in dobimo rezultate. Ukazna vrstica običajno izgleda kot Figure ??.
- Scripting: Kodo napišemo v besedilno datoteko in jo nato zaženemo v ukazni vrstici. Ta način je prav tako primeren za napredne uporabnike in ni priporočljiv za začetnike, saj zahteva poznavanje osnov programiranja.
- **Interaktivno v zvezku (Jupyter Notebook)**. Ta način je najbolj primeren za začetnike in je tisti, ki ga bomo uporabljali v tem predmetu. SageMath lahko uporabljamo v zvezkih Jupyter, ki omogočajo kombiniranje besedila, matematičnih izrazov in kode v enem dokumentu. Zvezki so zelo uporabni za učenje, raziskovanje in dokumentiranje matematičnih izračunov. V tem predmetu bomo uporabljali CoCalc, spletno platformo, ki omogoča uporabo SageMath v zvezkih Jupyter brez potrebe po lokalni namestitvi.

V belenici Jupyter imamo dve vrsti celic: *Code* (koda) in *Text* (besedilo) (glej Figure ??).

```

amontero — iPython: Users/amontero — Python · sage-python ~/sage/sage/src/bin/sage-ipython ~...
Last login: Sun Dec 15 18:48:42 on ttys005
You have mail.
sage
SageMath version 10.4.beta8, Release Date: 2024-06-01
Using Python 3.12.5. Type "help()" for help.
Warning: this is a prerelease version, and it may be unstable.
sage: 

```

Figure 5: Ukazna vrstica SageMath

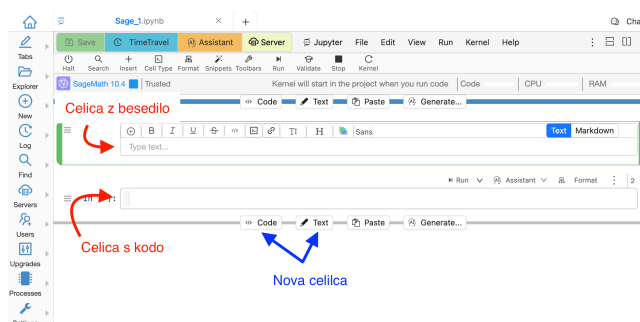


Figure 6: Celice v Jupyter

V celici z besedilo lahko uporabite *Markdown* ali standardne besedilo. Priporočamo, da uporabite *Markdown*, saj omogoča boljše oblikovanje besedila in vključevanje matematičnih izrazov.

Glej tudi

Markdown je preprost jezik za označevanje besedila, ki omogoča enostavno oblikovanje besedila. Pristop je podoben LaTeX-u, vendar je veliko enostavnejši za uporabo, na alost, pa tudi manj zmogljiv.

Osnovne značilnosti Markdowna vključujejo:

- Naslovi: Uporabite **# Naslov** za naslove. Ve # pomeni nivo raven naslova.
- Krepko in leee besedilo: Uporabite ****krepko**** za **krepko** besedilo in *_leee_* za *leee* besedilo.
- Seznami: Uporabite - ali * za neurejene sezname in tevilke za urejene sezname.
- Povezave: Uporabite [besedilo](URL) za ustvarjanje povezav.
- V Jupyter zvezkih lahko vključite tudi matematične izraze z uporabo LaTeX sintakse, na primer $x^2 + y^2 = z^2$ za inline izraze in $E = mc^2$ za prikazne izraze.

Za več informacij o Markdownu si oglejte naslednje vire: Markdown osnovna sintaksa

Ta ubenik je napisan v Myst Markdown, ki je raziritev Markdowna.

V celici z kodo vnesemo ukaze SageMath. Za izvedbo kode pritisnite + ali kliknite na gumb "Run" v orodni vrstici. Rezultati se bodo prikazali neposredno pod celico z kodo.

SageMath kot kalkulator

SageMath lahko uporabljamo kot kalkulator za osnovne aritmetične operacije, kot so seštevanje, odštevanje, množenje in deljenje. Poleg tega lahko uporabljamo tudi bolj zapletene matematične funkcije in operacije.

Z nekaj manjšimi izjemami Sage uporablja programski jezik Python, zato so osnovne aritmetične operacije in druge osnove programiranja v SageMath zelo podobne tistim v Pythonu.

Poglejte naslednje primere. e bolj pa jih preizkusite sami v svojem CoCalc zvezku!

1+1

2

(1 + 2 * (3 + 5)) * 2

34

Simbol `*` pomeni množenje, ki se ne sme izpustiti, tudi v izrazih, kot je $2x$: `2*x`. Za združevanje simbolov uporabljamo oklepaj `( )` nikoli `[ ]` ali `{ }`.

`2**3` # `**` pomeni potenciranje, kot v Pythonu

8

`2^3` # ampak `^` pomeni tudi potenciranje, za razliko od Pythona

8

`-3^2` # pazite!

-9

Pri vzeto SageMath poskua privzeto vrniti izraz, ki je im bolj natanen. To je $10/3$ vrne racionalnotevilo $10/3$ (in ne decimalno približek, kot je 3,33333). e elimo tevilni približek, lahko enemu od tevil preprosto dodamo decimalno piko.

20.0 / 6

3.333333333333333

Kvadratne korene lahko izraunamo z ukazom `sqrt()`. Upotevajte, da e je argument celotevilo, SageMath interpretira rezultat kot simbolnotevilo.

`sqrt(2)`

`sqrt(2)`

`sqrt(8)`

`2*sqrt(2)`

Seveda, lahko dobimo tevilni približek z ukazom `sqrt(8.0)`. Na splono lahko dobimo numerine približke izrazov z (enakovrednimi) ukazi `n()`, `N()`, `numerical_approx()`. Vsi zgornji ukazi imajo neobvezen argument `digits` za doloitev natannosti približke.

`n(sqrt(8), digits=50)`

2.8284271247461900976033774484193961571393437507539

V Table ?? so povzete osnovne aritmetine operacije v SageMath-u.

Table 9: Osnovne aritmetine operacije v SageMath-u

Seštevanje ($a + b$)	<code>a+b</code>
Odštevanje ($a - b$)	<code>a-b</code>
Množenje (ab)	<code>a*b</code>
Deljenje ($\frac{a}{b}$)	<code>a/b</code>
Potenciranje (a^b)	<code>a**b</code> ali <code>a^b</code>
Kvadratni koren ($\sqrt{a}$)	<code>sqrt(a)</code>
Sploni koren ($\sqrt[n]{a}$)	<code>a^(1/n)</code>

Operacije s celimi tevil

S simboli `//` in `%` dobimo (celotevilski) kvocient oziroma ostanek dveh celih tevil

$$20 = 6 \cdot 3 + 2 \quad (2)$$

```
20 // 6 # celotevilski kvocient
```

```
3
```

```
20 % 6 # ostanek
```

```
2
```

```
divmod(20,6) # vrne kvocient in ostanek hkrati kot tuple
```

```
(3, 2)
```

Izračunamo lahko tudi fakultete in binomske koeficiente:

$$3! = 6 \quad (3)$$

```
factorial(3)
```

```
6
```

$$\binom{4}{2} = \frac{4!}{(4-2)!2!} = 6 \quad (4)$$

```
binomial(4,2)
```

```
6
```

tevilu lahko faktoriziramo kot produkt pratevil:

```
factor(factorial(7))
```

```
2^4 * 3^2 * 5 * 7
```

Lahko uporabimo naslednji nain tudi

```
(factorial(7)).factor()
```

```
2^4 * 3^2 * 5 * 7
```

e elimo vedeti, ali je tevilu pratevilu, lahko uporabimo ukaz `is_prime()`:

```
is_prime(29)
```

```
True
```

```
is_prime(65537)
```

```
True
```

Delitelje tevila lahko dobimo z ukazom:

```
12.divisors()
```

```
[1, 2, 3, 4, 6, 12]
```

ali

```
divisors(12)
```

```
[1, 2, 3, 4, 6, 12]
```

e samo elimo delitelje, ki so pratevila, uporabimo ukaz `prime_divisors()`:

```
120.prime_divisors()
```

```
[2, 3, 5]
```

ali

```
120.prime_factors()
```

```
[2, 3, 5]
```

Izračunamo lahko *najveji skupni delitelj* in *najmanjši skupni vkratnik* dveh celih tevil z ukazoma `gcd()` in `lcm()`:

```
gcd(120,48)
```

```
24
```

$$D(120, 48) = 24 \quad (5)$$

```
lcm(9,15)
```

```
45
```

$$v(9, 15) = 45 \quad (6)$$

Matematine konstante in funkcije

SageMath vključuje številne matematične konstante in funkcije, ki jih lahko uporabimo v svojih izračunih. Nekatero izmed najbolj znanih konstant so:

- π : `pi`
- e : `e`
- ϕ (zlato rez): `golden_ratio`
- i (imaginarna enota): `I`

Pogoste funkcije so tudi vključene, na primer:

- Trigonometrične funkcije: `sin()`, `cos()`, itd.
- Eksponentna funkcija: `exp()`
- Logaritemska funkcija: `log()`

```
sin(pi)
```

```
0
```

```
cos(pi/4)
```

```
1/2*sqrt(2)
```

Sage ima poseben ukaz `latex()` za prikaz matematičnih izrazov v LaTeX obliki. Na primer:

```
latex(cos(pi/4))
```

```
\frac{1}{2} \sqrt{2}
```

e elimo da izhod je predstavljen v LaTeX-u, uporabimo arobni ukaz `%display latex`. e elimo, da se izhod prikaže kot običajen besedilni izhod, uporabimo arobni ukaz `%display text`.

Poglejmo e primere:

```
ln(e)
```

```
1
```

V sage `log()` pomeni naraven logaritem, torej logaritem pri osnovni e :

```
log(e) == ln(e)
```

```
1 == 1
```

e elimo logaritem pri drugi osnovi, na primer 10, uporabimo drugi argument:

```
log(1000, 10)
```

```
3
```

Nekateri viri namesto e raje pišejo `exp` in SageMath meni, da je to v redu.

```
exp(3) - e^3
```

```
0
```

V SageMath lahko delamo tudi z kompleksnimi števili. Imaginarno število i je označeno z `i` in z `I`. Ker pa se `i` pogosto uporablja pri programiranju, je dobra praksa, da se označi z `I`.

```
arg(1+I)
```

`1/4*pi`

`latex(1+i)`

`i + 1`

Ukaza `max()` in `min()` vrneti najveje in najmanjetevilo v seznamu tevil.

`max([1,2,3])`

3

Seznam lahko damo z oklepaji `[ ]` ali brez njih.

`min(1,2,3)`

1

Z ukazom `floor()` dobimo najveje celotevilo, ki ni večje od danega tevila, z ukazom `ceil()` pa najmanje celotevilo, ki ni manjše od danega tevila.

`floor(2.1)`

2

`ceil(2.1)`

3

Pazite:

`floor(1/(2.1 - 2))`

9

Ampak

$$\left\lfloor \frac{1}{2.1 - 2} \right\rfloor = \left\lfloor \frac{1}{0.1} \right\rfloor = 10 \quad (7)$$

Kaj se je zgodilo?

Raunalniki shranjujejo realna tevila v binarni obliki, mi pa smo navajeni uporabljati desetiko predstavitev.tevilo 2.1 v desetikem zapisu je precej preprosto in kratko, vendar je po pretvorbi v binarni zapis $10.\overline{00011} = 10.0001100110011\dots$

Ker raunalniki ne morejo shraniti neskonnega tevila tevil, se totevilo nekje zaokroi, kar povzroči manjše napake, ki smo jo videli. V SageMathu pa so racionalna tevila (ulomki) natanna, zato te napake pri zaokroevanju nikoli ne bomo videli.

`floor(1/ ( (21/10) - 2) )`

10

Pripisovanje, enakost, enable in neenable.

SageMath uporablja simbol `=` za **pripisovanje vrednosti spremenljivkam**.

```
a = 5
```

```
a
```

```
5
```

Za primerjavo uporablja `==`, `!=`, `<`, `>`, `<=`, `>=`.

```
2==2
```

```
True
```

```
2!=2
```

```
False
```

```
2 < 2
```

```
False
```

```
a <= 5
```

```
True
```

Seveda, lahko uporabimo `=` za shraniti rezultat izrauna v spremenljivko:

```
b = 2 + 3 * 4
```

```
b
```

```
14
```

Warning

Pazite: `=` ni enako kot `==`. Prvi je za pripisovanje, drugi pa za primerjavo. Pogosta napaka je uporaba izraza `a = b` za primerjavo `a` in `b`, kar v resnici pomeni, da se vrednost `b` pripise `a`.

Rezultat izrauna se ne natisne samodejno, ko je pripisan spremenljivki. Čeelite videti vrednost spremenljivke, jo morate izrecno navesti v celici ali uporabiti ukaz `print()`. Poglejte primeri:

```
y = 1 + 2; print(y)
```

```
y = 3 * y + 1; print(y)
```

```
y = 3 * y + 1; print(y)
```

```
y = 3 * y + 1; print(y)
```

```
3
```

```
10
```

```
31
```

```
94
```

Znak `;` je uporabno za loevanje več ukazov v eni celici.

Naeloma nam ni ne prepreuje, da bi ponovno definirali obstoječe spremenljivke.

```
pi= -I/2
```

```
exp(2*pi*I)
```

e

Z ukazom `restore()` lahko obnovimo privzeto vrednost spremenljivke.

```
restore()
```

```
y+pi
```

```
pi + 94
```

Ukaz `reset()` pa ponastavi celoten delovni prostor in izbriše vse spremenljivke.

```
reset()
```

```
y
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[72], line 1
    - - - -> 1 y
```

```
NameError: name 'y' is not defined
```

Opazite tudi, da SageMath vasih razlikuje med simbolnimi izrauni in tevili:

```
e^(2*pi*I)
```

```
1
```

```
e^(2*pi*I) == 1
```

```
1 == 1
```

SageMath zgornjega izraza ne razume ko primerjavo, temve kot enabo. V tem primeru, ne moremo jo reiti, ampak lahko preverimo, ali je enaba resnina z uporabo ukaza `bool()`:

```
bool(e^(2*pi*I) == 1)
```

```
True
```

Enabe in neenakosti lahko reujemo z ukazoma `solve()`:

```
solve(3*x - 2 == 5, x)
```

```
[x == (7/3)]
```

rka `x` v SageMath-u predstavlja simbolino spremenljivko in lahko jo uporabimo v enabah in izrazih:

```
solve( 2*x - 5 >= 17, x)
```

```
[[x >= 11]]
```

```
solve( x^2 + x == 6, x)
```

```
[x == -3, x == 2]
```

```
solve(2*x^2 - x + 1 == 0, x)
```

```
[x == -1/4*I*sqrt(7) + 1/4, x == 1/4*I*sqrt(7) + 1/4]
```

Ukaz `solve()` poskua reiti enabe brez numerinih priblikov, e je le mogoe. Ko tega ni mono, bo vrnil reitev v simbolni obliki.

```
solve(sin(x)==x,x)
```

```
[x == sin(x)]
```

Za iskanje numerinega priblika reitev lahko uporabimo ukaz `find_root()`; ta ukaz zahteva zaetni interval, v katerem iemo reitev.

```
find_root(sin(x) == x, -pi/2 , pi/2)
```

```
0.0
```

Ob zaetku seje SageMath eno simbolno spremenljivko, in sicer `x`, ki se lahko uporablja v enabah in izrazih. e elimo uporabiti druge spremenljivke, jih moramo najprej definirati z ukazom `var()`:

```
y = var("y")
```

Lahko pa definiramo ve spremenljivk hkrati:

```
phi, theta, rho = var("phi theta rho")
```

Imena spremenljivk lahko vsebujejo rke, tevilke in podrtaje, vendar se ne smejo zaeti s tevilko in presledkov ne smejo vsebovati. Npr. `x_1`, `var123`, in `my_variable` so veljavna imena spremenljivk, medtem ko `1var`, `my variable`, in `var-2` niso veljavna imena spremenljivk.

```
x1, x2 = var("x1 x2")
```

Poskus uporabe neveljavnega imena spremenljivke bo povzroil napako `NameError`:

```
u
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[85], line 1
- - - -> 1 u
```

```
NameError: name 'u' is not defined
```

Simbolno spremenljivko lahko izbriemo s ukazom `restore()`:

```
restore('phi')
```

```
phi
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[87], line 1
- - - -> 1 phi
```

```
NameError: name 'phi' is not defined
```

Majhne sisteme linearnih enab lahko reimo z ukazom `solve()`, dokler so spremenljivke definirane.

```
x, y, z = var("x y z")
```

```
solve( [3*x - y == 2, -2*x - y == 1 ], x,y)
```

```
[[x == (1/5), y == (-7/5)]]
```

```
solve( [ 2*x - y == -1 , 2*x - y == 2],x,y)
```

```
[]
```

```
solve( [ 2*x + y == -1 , -4*x - 2*y == 2],x,y)
```

```
[[x == -1/2*r1 - 1/2, y == r1]]
```

Vrednost `r1` označuje, da obstaja prosta spremenljivka, ki določa mnogo reitev. Če je v prostih spremenljivkah, bo rezultat vseboval več parametrov `r1`, `r2`, itd.

```
solve([ 2*x + 3*y + 5*z == 1, 4*x + 6*y + 10*z == 2, 6*x + 9*y + 15*z == 3], x,y,z)
```

```
[[x == -5/2*r2 - 3/2*r3 + 1/2, y == r3, z == r2]]
```

Ukaz `solve()` je lahko pri velikih enačbah zelo počasen. V takih primerih je bolje uporabiti matrike (poglej [Linearna Algebra s SageMath](#))

Reševanje neenakosti z več spremenljivkami je mono, ampak lahko rezultati postanejo zelo zapleteni in težko berljivi.

```
solve([ x - y >=2, x+y <=3], x,y)
```

```
[[x == (5/2), y == (1/2)], [x == -y + 3, y < (1/2)], [x == y + 2, y < (1/2)], [y + 2 < x, x <
```

0.3.2 Linearna Algebra s SageMath

Vektroji

Vektorje v SageMath lahko ustvarimo na ve nainov. Najenostavnejši je z uporabo ukaza `vector(<seznam>)`, kjer je `<seznam>` seznam tevil, ki predstavljajo komponente vektorja.

```
v=vector([1,2]) # pazite, argument mora biti seznam ali tuple
v
```

```
(1, 2)
```

Do koordinat vektorja a lahko dostopamo kot do kateregakoli seznama v SageMath

```
print(v[0])
print(v[1])
```

```
1
2
```

Pazite!

Indeksi v SageMath (in Pythonu) se zanejo z 0, tako da je prva komponenta vektorja `v[0]`, druga pa `v[1]`.

```
v[2]
```

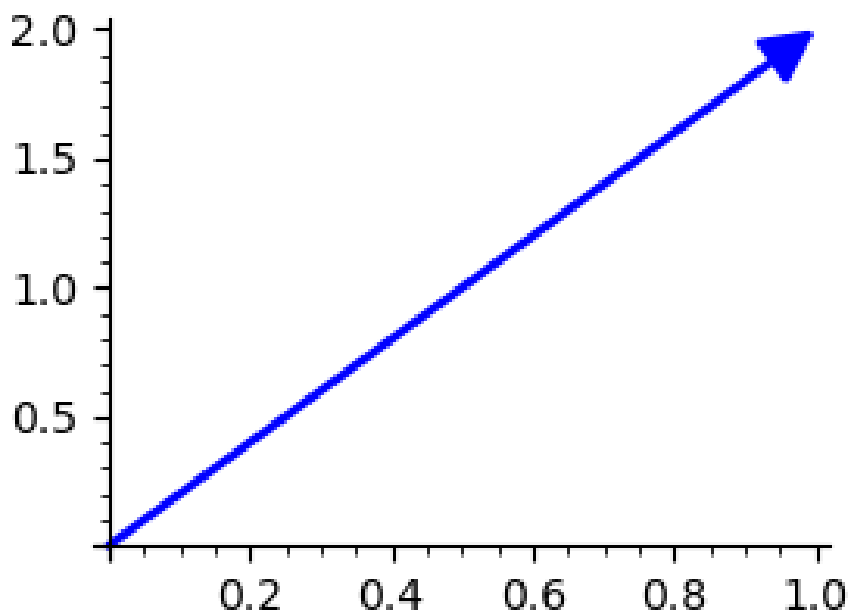
```
-----
IndexError                                Traceback (most recent call last)
Cell In[3], line 1
- - - -> 1 v[Integer(2)]
```

```
File /Applications/SageMath -10 -5.app/Contents/Frameworks/Sage.framework/Versions/Current/ven
 1905     n += d
 1906 if n < 0 or n >= d:
-> 1907     raise IndexError("vector index out of range")
 1908 return self.get_unsafe(n)
 1909
```

```
IndexError: vector index out of range
```

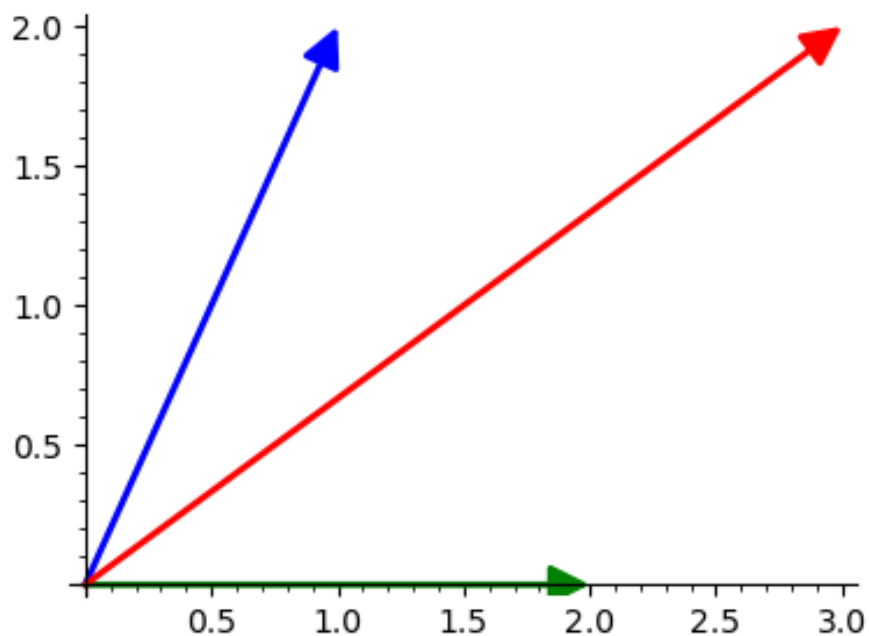
Grafini prikaz vektorjev v ravnini lahko dobimo z metodo `plot()`:

```
v.plot(figsize=3)
```



Vektorje lahko setevamo in odtevamo, prav tako jih lahko mnoimo s skalarjem:

```
v1 = vector([1,2]) #modro
v2 = vector([1,0]) #ni prikazen
v= 2*v2 #zeleno
w = v1 + v #rdece
plots = v1.plot(color='blue') + v.plot(color='green') + w.plot(color='red')
show(plots, figsize=4)
```



Lahko tudi izračunamo skalarni produkt dveh vektorjev:

```
v1*v2
```

```
1
```

$$v_1 \cdot v_2 = (1, 2) \cdot (1, 0) = 1 \cdot 1 + 2 \cdot 0 = 1$$

(8)

ali dolino oz. normo vektorja:

```
print("$$ |v_1|= " + latex(v1.norm()) + "$$")
```

```
$$ |v_1|= \sqrt{5} $$
```

$$|v_1| = \sqrt{5} \quad (9)$$

Zgornje ukaze lahko uporabimo za zapis funkcije v SageMath, ki določi, ali sta vektorja pravokotna:

```
def sta_pravokotna(v,u): # definira funkcijo z dvema argumentoma u,v
    return v*u==0        # vrne, ali je skalarni produkt argumentov enak ni ali ne
```

```
sta_pravokotna(v1,v2)
```

```
False
```

```
v3=vector([0,1])
sta_pravokotna(v2,v3)
```

```
True
```

Veina teh ukazov in funkcij deluje tudi v več dimenzijah:

```
u1=vector([1,0,0])
u2=vector([0,1,0])
u3=vector([1,2,3])
u1+u2
```

```
(1, 1, 0)
```

$$u_1 + u_2 = (1, 0, 0) + (0, 1, 0) = (1 + 0, 0 + 1, 0 + 0) = (1, 1, 0) \quad (10)$$

```
print(u2*u3)
print(u3.norm())
```

```
2
```

```
sqrt(14)
```

$$u_2 \cdot u_3 = (0, 1, 0) \cdot (1, 2, 3) = 0 \cdot 1 + 1 \cdot 2 + 0 \cdot 3 = 2 \quad (11)$$

$$|u_3| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{14} \quad (12)$$

Tudi risanje!

```
u3.plot()
```

Vektorski prostori

SageMath razume ve standardnih tevilskih sistemov:

```
print(ZZ) #cela tevila
print(QQ) #racionalna tevila
print(RR) #raalna tevila
print(CC) #kompleksna tevila
print(SR) #simbolini kolobar
```

Integer Ring

Rational Field

Real Field with 53 bits of precision

Complex Field with 53 bits of precision

Symbolic Ring

Vsak od njih vsebuje vedno ve splonih elementov:

```
print(3 in ZZ, 1/2 in ZZ)
print(3 in QQ, 1/2 in QQ, pi in QQ)
print(3 in RR, 1/2 in RR, pi in RR, I in RR)
print(3 in CC, 1/2 in CC, pi in CC, I in CC, x in CC)
print(3 in SR, 1/2 in SR, pi in SR, I in SR, x in SR)
```

True False

True True False

True True True False

True True True True False

True True True True True

$$3 \in \mathbb{Z}, \quad \frac{1}{2} \notin \mathbb{Z} \quad (13)$$

$$3 \in \mathbb{Q}, \quad \frac{1}{2} \in \mathbb{Q}, \quad \pi \notin \mathbb{Q} \quad (14)$$

$$3 \in \mathbb{R}, \quad \frac{1}{2} \in \mathbb{R}, \quad \pi \in \mathbb{R}, \quad i \notin \mathbb{R} \quad (15)$$

$$3 \in \mathbb{C}, \quad \frac{1}{2} \in \mathbb{C}, \quad \pi \in \mathbb{C}, \quad i \in \mathbb{C}, \quad x \notin \mathbb{C} \quad (16)$$

$$3 \in SR, \quad \frac{1}{2} \in SR, \quad \pi \in SR, \quad i \in SR, \quad x \in SR \quad (17)$$

Ustvarimo lahko vektorske prostore nad katerim koli poljem: $\mathbb{Q}$, $\mathbb{R}$ ali $\mathbb{C}$, vendar, kot smo e videli, SageMath najbolje deluje z racionalnimi tevili.

Za ustvarjanje vektorskega prostora uporabimo ukaz `VectorSpace(<polje>, <dimenzija>)`, kjer je `<polje>` polje nad katerim definiramo vektorski prostor, `<dimenzija>` pa dimenzija vektorskega prostora.

```
V = VectorSpace(QQ, 3) #vektorski prostor dimenzije 3 nad racionalnimi tevili
v = V([1,2,3]) # (1,2,3) kot vector v prostoru V
```

Pazite!

Vasih SageMath ugiba, kaj smo mislili:

```
w = vector(RR, [1,2,3]) #w kot vector nad RR
print(w == v)
print(w in V)
```

```
True
True
```

vasih pa ne:

```
w=vector(CC,[1,2,3]) #w kot vector nad CC
print(w==v)
print(w in V)
```

```
True
False
```

Izračunamo lahko osnovo za vektorski prostor:

```
V.basis()
```

```
[
(1, 0, 0),
(0, 1, 0),
(0, 0, 1)
]
```

Izračunamo lahko podprostor prostora V , ki ga generira množica vektorjev:

```
u1= V([1,0,0]); u2= V([1,1,0]);
W=V.span([u1,u2])
W
```

```
Vector space of degree 3 and dimension 2 over Rational Field
Basis matrix:
[1 0 0]
[0 1 0]
```

```
u3=V([0,3,0]) # vektor (1,2,3) v V
u4=V([0,0,1]) # vektor (0,0,1) v V
(u3 in W, u4 in W) # preveri, ali sta vektorja v podprostoru W

(True, False)
```

Matrike: izdelava in osnovne operacije

Matrike v SageMath lahko ustvarimo na več načinov. Najenostavnejši je z uporabo ukaza `matrix(<seznam seznamov>)`, kjer je `<seznam seznamov>` seznam vrstic matrike, vsaka vrstica pa je predstavljena kot seznam števil.

```
matrix([[1,2],[3,4]])
```

```
[1 2]
[3 4]
```

Lahko tudi najprej določimo dimenzije in nato podamo vse vnose na seznamu.

```
M= matrix(4,2, [1,2,3,4,5,6,7,8]) # argumenti so st_vrstic,
M                                # st_stolpcev, seznam vnosov
```

```
[1 2]
[3 4]
[5 6]
[7 8]
```

e je matrika, ki jo elimo ustvariti, kvadratna, lahko v argumentih izpustimotevilo stolpcev:

```
matrix(2, [1,2,3,4])
```

```
[1 2]
[3 4]
```

Privzeto SageMath konstruira matriko nad najmanjim vesoljem, ki vsebuje koordinate:

```
print(parent(matrix(2, [1,2,3,4])))
print(parent(matrix(2, [1,2/1,3,4])))
print(parent(matrix(2, [x,x^2,x -1,x^3])))
```

```
Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
Full MatrixSpace of 2 by 2 dense matrices over Rational Field
Full MatrixSpace of 2 by 2 dense matrices over Symbolic Ring
```

Lahko pa tudi določimo polje, nad katerim elimo delati:

```
matrix(QQ,2, [1.1,1.2,1.3,1.4])
```

```
[11/10  6/5]
[13/10  7/5]
```

Nekatere standardne matrike imajo svoj ukaz:

```
identity_matrix(3) # enotska matrika velikosti 3x3
```

```
[1 0 0]
[0 1 0]
[0 0 1]
```

Matrike, katerih vsi vnosi so enaki 0:

```
print("Vse nile:")
print(zero_matrix(3,2), "\n")
print("Vse nile, kvadratna:")
print(zero_matrix(2), "\n")
```

```
Vse nile:
[0 0]
[0 0]
[0 0]
```

```
Vse nile, kvadratna:
[0 0]
[0 0]
```

ali vsi vnosi enaki 1:

```
print("Vse 1:")
print(ones_matrix(3,2),"\n")
print("Vse 1, kvadratna:")
print(ones_matrix(3), "\n")
```

Vse 1:

```
[1 1]
[1 1]
[1 1]
```

Vse 1, kvadratna:

```
[1 1 1]
[1 1 1]
[1 1 1]
```

Matrike lahko setevamo in odtevamo, prav tako jih lahko mnoimo s skalarjem:

```
A=matrix(2,[1,1,0,1])
B=matrix(2,[1,0,1,1])
print(A, "\n") # znak "\n" je prelom vrstice
print(B)
```

```
[1 1]
[0 1]
```

```
[1 0]
[1 1]
```

```
print(A+B, "\n")
print(A -B, "\n")
print(A*B, "\n")
print(A^3)
```

```
[2 1]
[1 2]
```

```
[ 0  1]
[-1  0]
```

```
[2 1]
[1 1]
```

```
[1 3]
[0 1]
```

e je A kvadratna matrika, lahko izračunamo tudi njeno inverzno matriko A^{-1} z ukazom `A.inverse()`, ali z ukazom `A^(-1)`:

```
print(A.inverse(), "\n")
A.inverse()==A^(-1)
```

```
[ 1 -1]
[ 0  1]
```

True

Pogost primer uporabe: Piemo dokument v LaTeXu in moramo izračunati inverzno velike matrike, npr.

$$\begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 2 & 3 & 4 & 5 & 6 \\ 3 & 4 & 7 & 8 & 9 \\ 4 & 5 & 6 & 10 & 11 \\ 5 & 6 & 7 & 8 & 12 \end{pmatrix} \quad (18)$$

Nekaj minut lahko porabimo za roni izračun njeno inverzno (in tvegajte, da bomo naredili neumnih napake), ali nekaj sekund za vnos matrike v SageMath.

```
A=matrix(ZZ,5, [1,1,1,1,1,2,3,4,5,6,3,4,7,8,9,4,5,6,10,11,5,6,7,8,12])
A^(-1)
```

```
[ 5/6 -13/6  1/2  1/3  1/3]
[ 2/3   8/3  -1 -1/3 -1/3]
[ 1/6  -1/6  1/2 -1/3  0]
[ 1/3   0    0  1/3 -1/3]
[ -1  -1/3   0    0  1/3]
```

Pridobimo lahko celo različico LaTeXa in jo vrnemo v naš dokument:

```
latex(A^(-1))
```

```
\left(\begin{array}{rrrrr}
\frac{5}{6} & -\frac{13}{6} & \frac{1}{2} & \frac{1}{3} & \frac{1}{3} \\
\frac{2}{3} & \frac{8}{3} & -1 & -\frac{1}{3} & -\frac{1}{3} \\
\frac{1}{6} & -\frac{1}{6} & \frac{1}{2} & -\frac{1}{3} & 0 \\
\frac{1}{3} & 0 & 0 & \frac{1}{3} & -\frac{1}{3} \\
-1 & -\frac{1}{3} & 0 & 0 & \frac{1}{3}
\end{array}\right)
```

$$\begin{pmatrix} \frac{5}{6} & -\frac{13}{6} & \frac{1}{2} & \frac{1}{3} & \frac{1}{3} \\ \frac{2}{3} & \frac{8}{3} & -1 & -\frac{1}{3} & -\frac{1}{3} \\ \frac{1}{6} & -\frac{1}{6} & \frac{1}{2} & -\frac{1}{3} & 0 \\ \frac{1}{3} & 0 & 0 & \frac{1}{3} & -\frac{1}{3} \\ -1 & -\frac{1}{3} & 0 & 0 & \frac{1}{3} \end{pmatrix} \quad (19)$$

Seveda, ko smo se nauili, okolja `array` ni najbolje za matrike, vendar ni teko prilagoditi izhod LaTeXa za okolje `bmatrix`, `pmatrix` ali kateregakoli drugega, ki ga elite uporabiti.

S metodo `is_invertible()` lahko preverimo, ali je matrika obrnljiva:

```
A.is_invertible()
```

```
False
```

...vendar je rezultat odvisen od tega, kje je matrika definirana.

```
A.parent()
```

```
Full MatrixSpace of 5 by 5 dense matrices over Integer Ring
```

```

B=matrix(QQ,5, [1,1,1,1,1,2,3,4,5,6,3,4,7,8,9,4,5,6,10,11,5,6,7,8,12])
print(A==B)
print(B.parent())
print(A.is_invertible())
print(B.is_invertible())

```

```

True
Full MatrixSpace of 5 by 5 dense matrices over Rational Field
False
True

```

Z metodama `det()` in `trace()` izraunamo determinanto oziroma sled matrike.

```

print("Determinanta A:", A.det())
print("Sled A:", A.trace())

```

```

Determinanta A: 18
Sled A: 33

```

$$A = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 2 & 3 & 4 & 5 & 6 \\ 3 & 4 & 7 & 8 & 9 \\ 4 & 5 & 6 & 10 & 11 \\ 5 & 6 & 7 & 8 & 12 \end{pmatrix} \quad (20)$$

$$\det(A) = 1 \quad (21)$$

$$\text{tr}(A) = 1 + 3 + 7 + 10 + 12 = 33 \quad (22)$$

Upravljanje vnosov matrik in reevanje sistemov linearnih enab

SageMath ima razline metode za dostop do vnosov matrike.

```

M = matrix(QQ, [[1,2,3],[4,5,6],[7,8,9]])
M

```

```

[1 2 3]
[4 5 6]
[7 8 9]

```

$$M = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \quad (23)$$

Metodi `nrows()` in `ncols()` vrmetatevilo vrstic oziroma stolpcev matrike:

```

print(M.nrows())
print(M.ncols())

```

```

3
3

```

Z metodama `rows()` in `columns()` dobimo seznam vrstic oziroma stolpcev matrike:

```

M.rows()

```

```
[(1, 2, 3), (4, 5, 6), (7, 8, 9)]
```

```
M.columns()
```

```
[(1, 4, 7), (2, 5, 8), (3, 6, 9)]
```

Metodi `row(i)` in `column(i)` vrneta i -to vrstico oziroma stolpec. `M.row(i)` je enako `M.rows()[i]`.

```
M.row(1) == M.rows()[1]
```

```
True
```

Metoda `diagonal()` vrne seznam z vnosi na diagonalni. To deluje tudi pri matrikah, ki niso kvadratne.

```
M.diagonal()
```

```
[1, 5, 9]
```

Ta metoda deluje tudi pri matrikah, ki niso kvadratne:

```
N=matrix(QQ, [[1,2,3],[4,5,6]])
```

```
N.diagonal()
```

```
[1, 5]
```

SageMath nam omogoča tudi izdelavo novih matrik iz vrstinih in/ali stolpnih vektorjev.

```
print(M.matrix_from_columns([0,2]), "\n")
print(M.matrix_from_rows([0,1]), "\n")
print(M.matrix_from_rows_and_columns([0,2],[0,2]))
```

```
[1 3]
```

```
[4 6]
```

```
[7 9]
```

```
[1 2 3]
```

```
[4 5 6]
```

```
[1 3]
```

```
[7 9]
```

Metoda `matrix_from_rows_and_columns()` vrne matriko, katere vnosi so v izbranih vrsticah in izbranih stolpcih.

Uporabimo lahko tudi imenovane osnovne operacije v matrikah. Npr. z metodo `rescale_row(i, k)` pomnožimo i -to vrstico z k :

```
M.rescale_row(1, -1/4) # ne pozabite, da indeksi zanejo z 0
```

```
M
```

```
[ 1 2 3]
```

```
[-1 -5/4 -3/2]
```

```
[ 7 8 9]
```

Podobno, z metodo `rescale_column(j, k)` pomnoimo j -ti stolpec z k :

```
M.rescale_col(2, -1/3); M
```

```
[ 1  2 -1]
[ -1 -5/4 1/2]
[ 7  8 -3]
```

Upotevajte, da te metode **spreminjajo matriko na mestu**. Vse, kar smo storili, lahko prekliemo.

```
M.rescale_row(1, -4)
M.rescale_col(2, -3)
M
```

```
[1 2 3]
[4 5 6]
[7 8 9]
```

Vekratnik vrstice ali stolpca lahko dodamo drugi vrstici ali stolpcu z uporabo metode `add_multiple_of_row()`. Naslednji ukaz vzame -4 krat vrstico 0 in jo doda vrstici 1.

```
M.add_multiple_of_row(1,0, -4); M
```

```
[ 1  2  3]
[ 0 -3 -6]
[ 7  8  9]
```

```
M.add_multiple_of_row(1,0,4); M #prekliemo
```

```
[1 2 3]
[4 5 6]
[7 8 9]
```

Seveda lahko to storimo tudi s stolpci:

```
M.add_multiple_of_column(0,2, -3); M
```

```
[ -8  2  3]
[ -14  5  6]
[ -20  8  9]
```

```
M.add_multiple_of_column(0,2,3); M #prekliemo
```

```
[1 2 3]
[4 5 6]
[7 8 9]
```

Zamenjamo lahko dve vrstici z metodo `swap_rows(i,j)`:

```
M.swap_rows(1,0); M
```

```
[4 5 6]
[1 2 3]
[7 8 9]
```

podobno, z metodo `swap_columns(i,j)` zamenjamo dva stolpca:

```
M.swap_columns(0,2); M
```

```
[6 5 4]
[3 2 1]
[9 8 7]
```

```
M.swap_rows(1,0) #prekliemo
```

```
M.swap_columns(0,2); M
```

```
[1 2 3]
[4 5 6]
[7 8 9]
```

e elimo dobiti eelonsko obliko matrike uporabimo metodo `echelon_form()` ali metodo `echelonize()`. Razlika je v tem, da prva vrne matriko, druga pa to naredi na mestu.

```
M.echelon_form()
```

```
[ 1  0 -1]
[ 0  1  2]
[ 0  0  0]
```

```
M
```

```
[1 2 3]
[4 5 6]
[7 8 9]
```

```
M.echelonize(); M
```

```
[ 1  0 -1]
[ 0  1  2]
[ 0  0  0]
```

Lahko uporabimo ta nain za reevanje sistemov linearnih enab. Predpostavimo, da imamo sistem

$$\begin{aligned} 2x_1 + 4x_2 + 6x_3 + 2x_4 + 4x_5 &= 56 \\ x_1 + 2x_2 + 3x_3 + x_4 + x_5 &= 23 \\ 2x_1 + 4x_2 + 8x_3 &= 34 \\ 3x_1 + 6x_2 + 7x_3 + 5x_4 + 9x_5 &= 101 \end{aligned} \tag{24}$$

Matrika, ki je povezana s tem sistemom, je

$$\begin{pmatrix} 2 & 4 & 6 & 2 & 4 \\ 1 & 2 & 3 & 1 & 1 \\ 2 & 4 & 8 & 0 & 0 \\ 3 & 6 & 7 & 5 & 9 \end{pmatrix} \tag{25}$$

```
M = matrix(QQ, [[2,4,6,2,4],[1,2,3,1,1],[2,4,8,0,0],[3,6,7,5,9]]); M
```

```
[2 4 6 2 4]
[1 2 3 1 1]
[2 4 8 0 0]
[3 6 7 5 9]
```

Iemo reitve sistema $Mx = b$, kjer je x (stolpni) vektor $(x_1, x_2, x_3, x_4, x_5)$ in b (stolpni) vektor $(56, 23, 34, 101)$.

Z metodo `aug()` lahko ustvarimo poveano matriko $M|b$:

```
b = vector(QQ, [56, 23, 34, 101])
M_aug=M.augment(b); M_aug
```

```
[ 2  4  6  2  4 56]
[ 1  2  3  1  1 23]
[ 2  4  8  0  0 34]
[ 3  6  7  5  9 101]
```

Zdaj lahko uporabimo metodo `echelon_form()` na poveani matriki, da dobimo eelonsko obliko:

```
M_aug.echelon_form()
```

```
[ 1  2  0  4  0 21]
[ 0  0  1 -1  0 -1]
[ 0  0  0  0  1  5]
[ 0  0  0  0  0  0]
```

To pomeni, da je na prvotni sistem enab enak

$$\begin{aligned}x_1 + 2x_2 + 4x_4 &= 21 \\x_3 - x_4 &= -1 \\x_5 &= 5\end{aligned}\tag{26}$$

To pomeni, da imamo dve prosti spremenljivki (recimo x_2 v x_4) in da so zato mnoica reitev vsi vektorji oblike $t(-4, 0, 1, 1, 0) + s(-2, 1, 0, 0, 0) + (21, 0, -1, 0, 5)$, kjer sta t in s poljubni tevili.

Lahko preverimo to eksperimentalno. Iz naega znanja o linearni algebri je razvidno, da je dovolj preveriti $(s, t) = (1, 0)$ in $(s, t) = (0, 1)$.

```
s=1; t=0
x=s*vector(QQ,[ -4,0,1,1,0])+t*vector(QQ,[ -2,1,0,0,0])+vector(QQ,[21,0, -1,0,5])
M*x
```

```
(56, 23, 34, 101)
```

```
s=0; t=1
x=s*vector(QQ,[ -4,0,1,1,0])+t*vector(QQ,[ -2,1,0,0,0])+vector(QQ,[21,0, -1,0,5])
M*x
```

```
(56, 23, 34, 101)
```

0.3.3 Analiza in SageMath

Funkcije

Opozorilo!

SageMath je zelo podoben kot Python. V Pythonu (in v SageMathu) beseda **funkcije** je pogosto uporabljena za deli kode, ki jih lahko pokliemo z imenom in argumenti. Uporabljamo beseda **def** za definiranje teh funkcij.

V tem razdelku pa bomo govorili o **matematinih funkcijah**, kot so $f(x) = x^2$ ali $g(x) = \sin(x)$. Te funkcije lahko definiramo in analiziramo v SageMathu.

V Sage funkcije definiramo z naslednjo sintakso:

```
f(x)=(2*x+1)^3 # Pazite * za mnoenje
latex(f)
```

```
x \ {\mapsto} \ {\left(2 \, x + 1\right)}^3
```

Koda zgoraj definira funkcijo

$$f : x \mapsto (2x + 1)^3. \quad (27)$$

Funkcijo lahko nato pokliemo z doloenimi vrednostmi:

```
f(2) # Vrednost funkcije pri x=2
125
```

Seveda,

$$f(2) = (2 * 2 + 1)^3 = 5^3 = 125. \quad (28)$$

Z metodo `expand()` lahko izraz funkcije razirimo:

```
f.expand()
x | - -> 8*x^3 + 12*x^2 + 6*x + 1
```

Obratno, z metodo `simplify_full()` lahko izraz funkcije poenostavimo:

```
g(x) = (x^2 - 1)/(x - 1)
g.simplify_full()
x + 1
```

e imamo funkcijo definirano po delih, lahko uporabimo metodo `piecewise()`, npr. za funkcijo

$$h(x) = \begin{cases} x^2, & \text{e } x < 0 \\ x + 1, & \text{e } x \geq 0 \end{cases} \quad (29)$$

uporabimo:

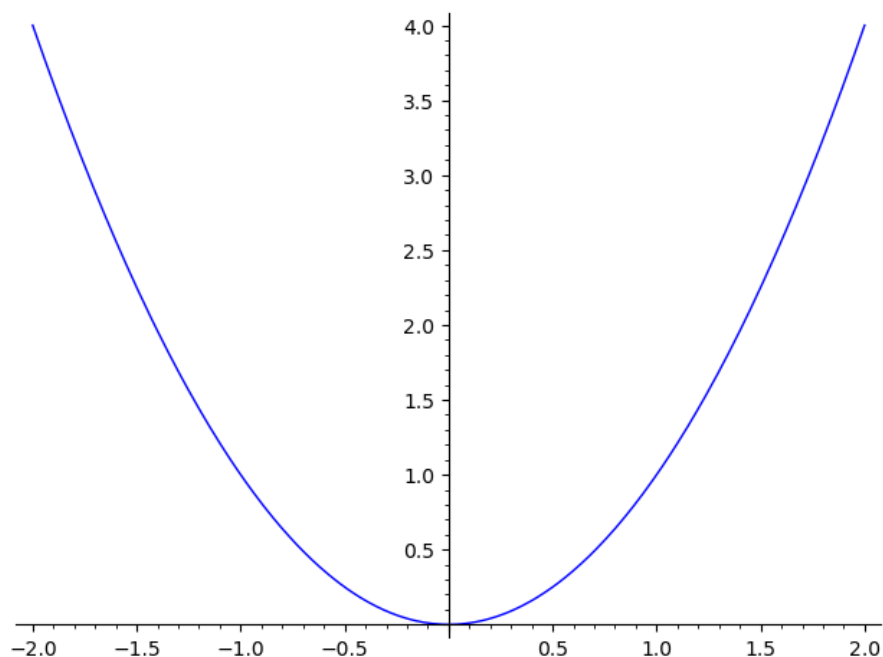
```
h = piecewise([(x<0), x^2], [(x>=0), x + 1])
h(-2), h(0), h(3)
(4, 1, 4)
```

$$\begin{aligned} h(-2) &= (-2)^2 = 4, \\ h(0) &= 0 + 1 = 1, \\ h(3) &= 3 + 1 = 4. \end{aligned} \quad (30)$$

Risanje grafov funkcij

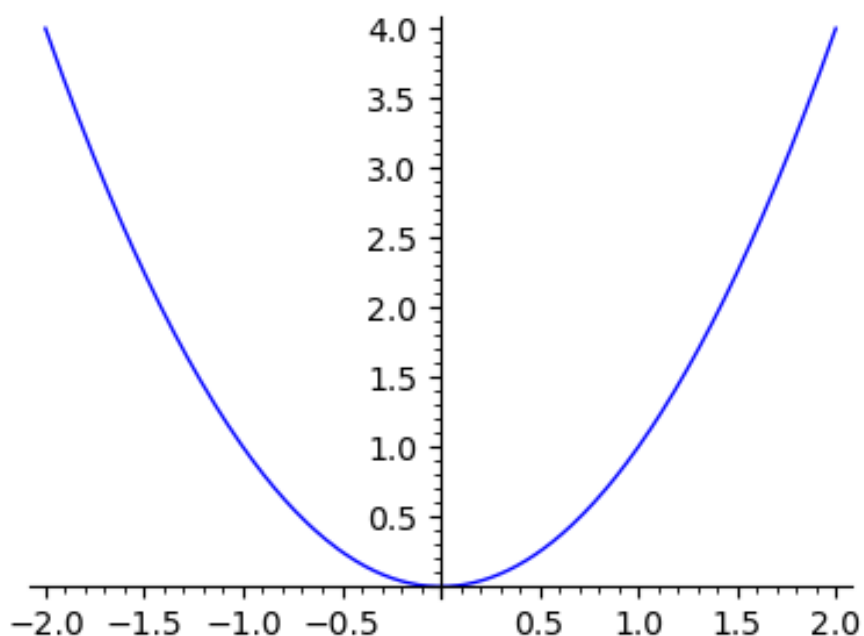
Graf funkcije f s spremenljivko x v intervalu $[a, b]$ nariemo z ukazom `plot(f, x, a, b)`:

```
f(x) = x^2
plot(f,x, -2,2)
```



Velikost grafa lahko prilagodimo z argumentoma `figsize=<velikost>`, npr. `figsize=4` za velikost 4x4 palcev:

```
plot(f,x, -2,2, figsize=4)
```

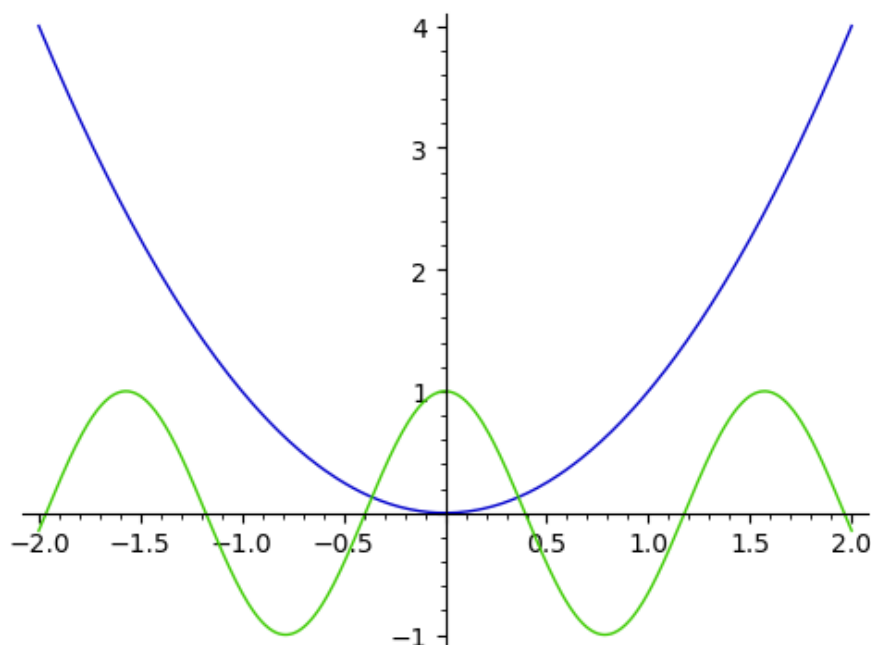


Privzete vrednosti lahko globalno nastavimo z ukazom:

```
sage.plot.graphics.Graphics.SHOW_OPTIONS['figsize']=5
```

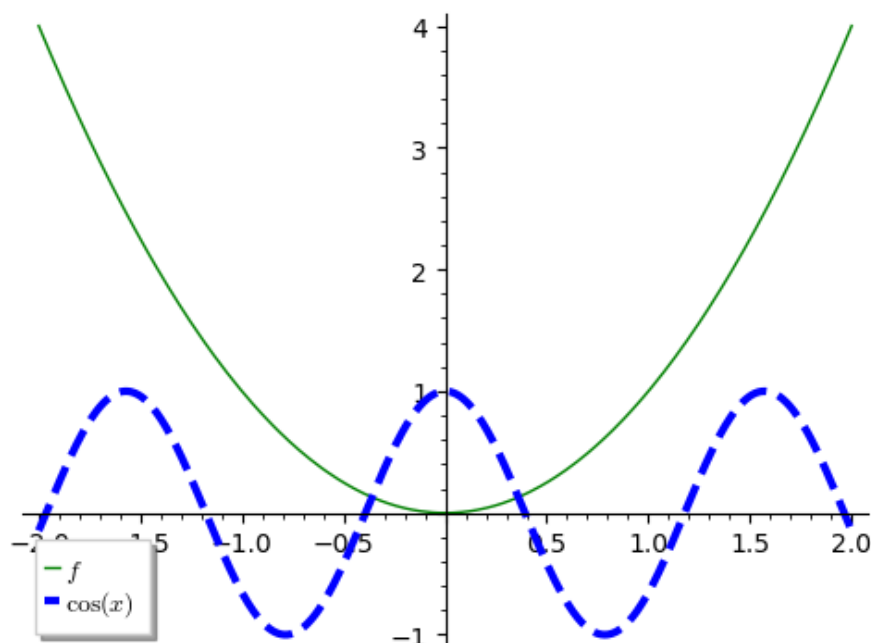
Grafa dveh ali več funkcij lahko nariemo skupaj, e namesto ene funkcije podamo seznam funkcij:

```
plot([f, cos(4*x)], x, -2, 2)
```



Drugi način je uporaba operatorja `+` za setevanje grafov. S tem pristopom lahko neodvisno nastavljamo lastnosti posameznih grafov:

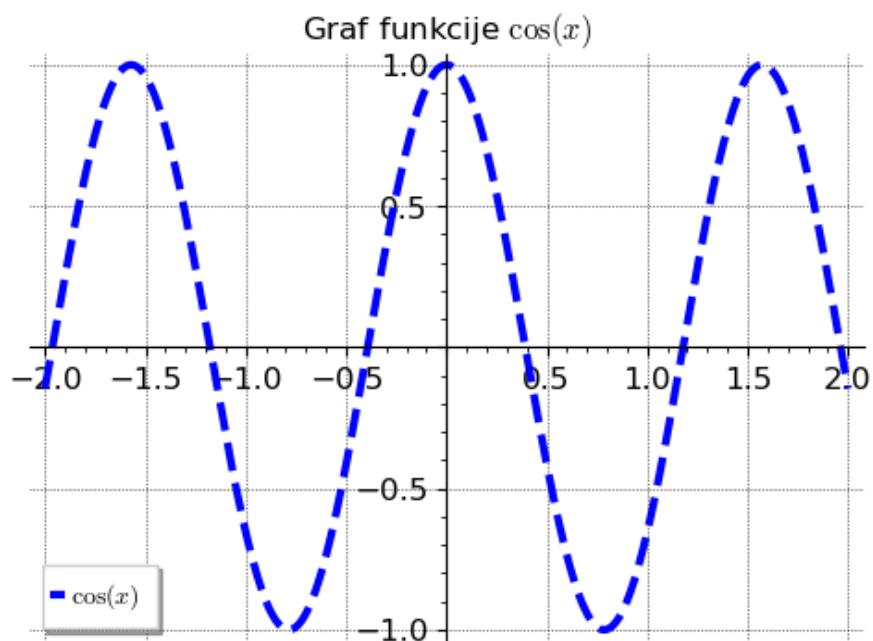
```
d1=plot(f,x, -2,2, color= 'green', legend_label='$f$')
d2=plot(cos(4*x),x, -2,2,
        thickness=3, linestyle=' - -',
        legend_label='$\\cos(x)$' ) # pazite dvojni poevnici za LaTeX
d1+d2
```



Vsak grafinski objekt v SageMath ima metodo `show()`, ki prikaže graf. Z njo lahko tudi nastavimo lastnosti grafa, kot so naslov, oznake osi in mreža:

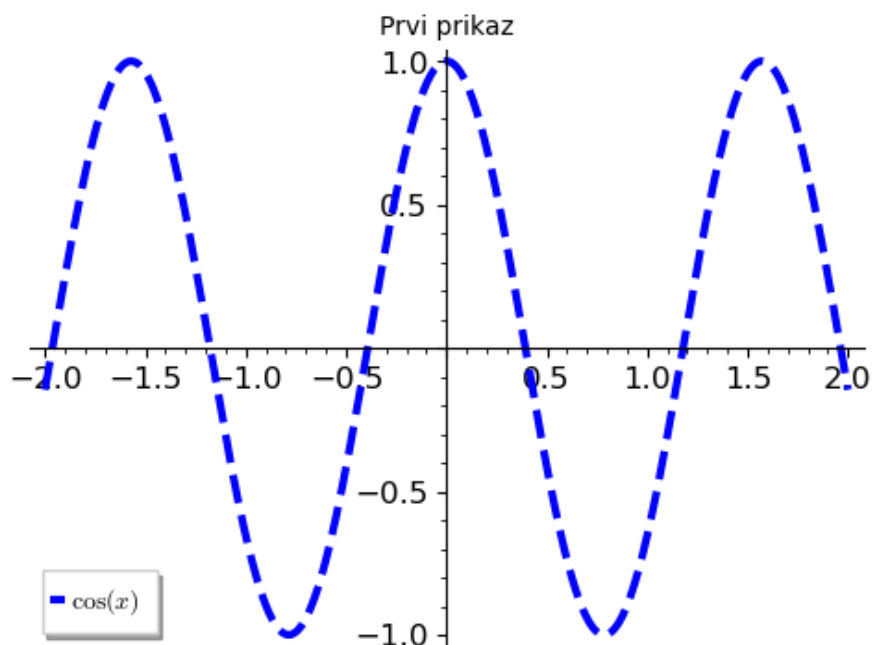
```
d2.show(
title='Graf funkcije $\\cos(x)$', #naslov
fontsize=12,
```

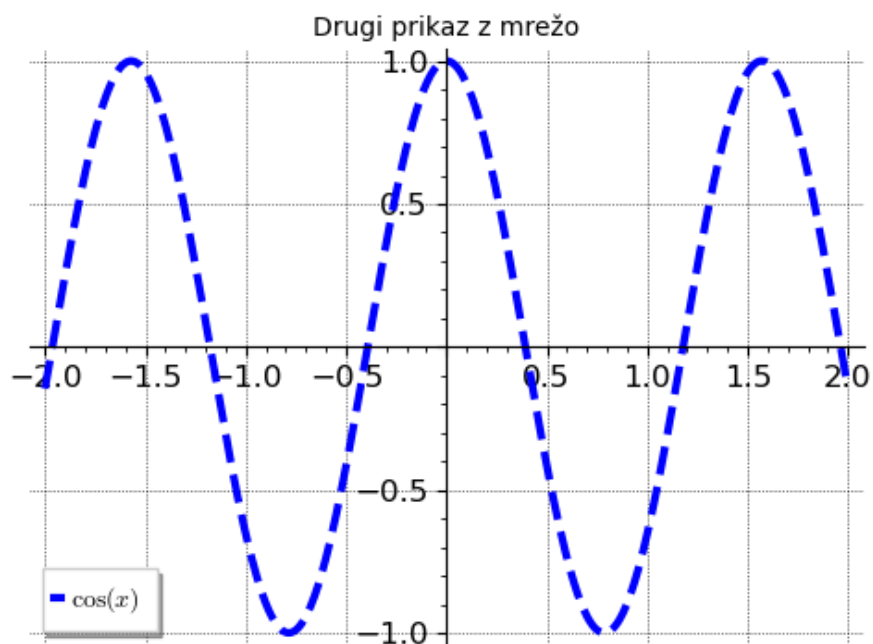
```
gridlines='mayor', #mrene rte,
)
```



Pazite, da metoda `show()` ne spremeni samega grafičnega objekta, ampak le nain njegovega prikaza, to pomeni, da lahko večkrat pokliemo `show()` z razlinimi nastavitvami:

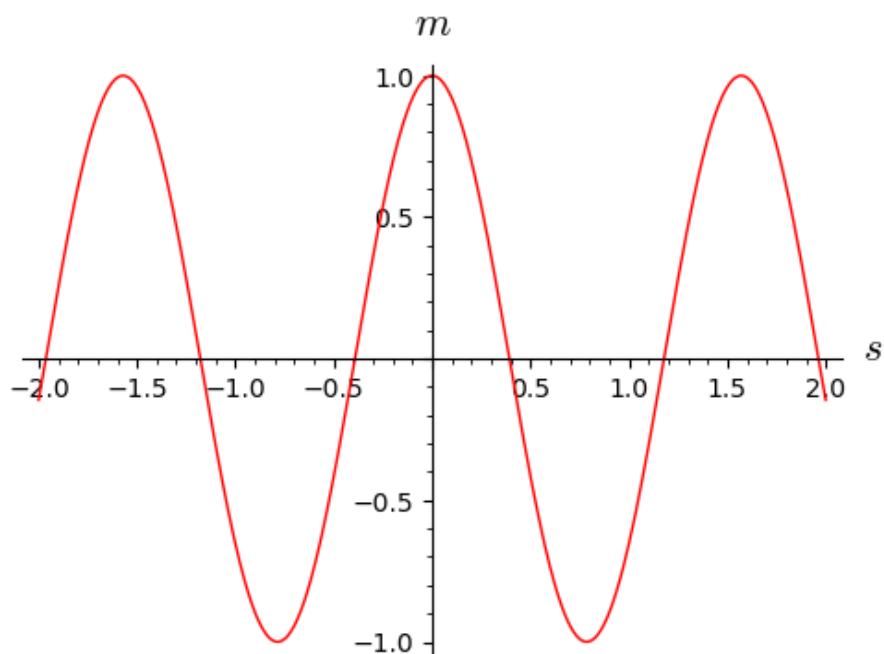
```
d2.show(title='Prvi prikaz')
d2.show(title='Drugi prikaz z mrejo', gridlines=True)
```



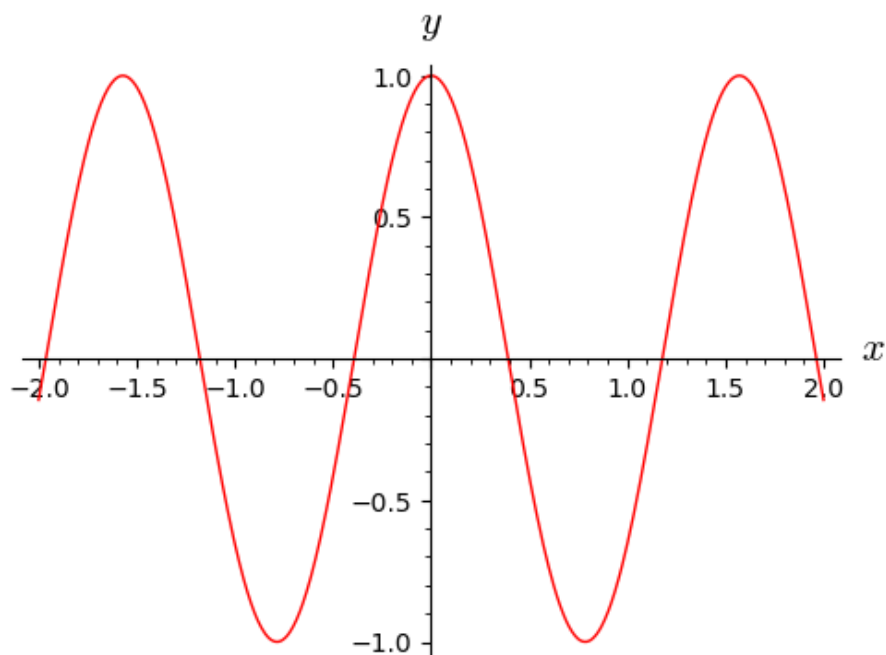


Ampak, e elimo trajno spremeniti lastnosti grafa, moramo to narediti pri ustvarjanju grafinnega objekta:

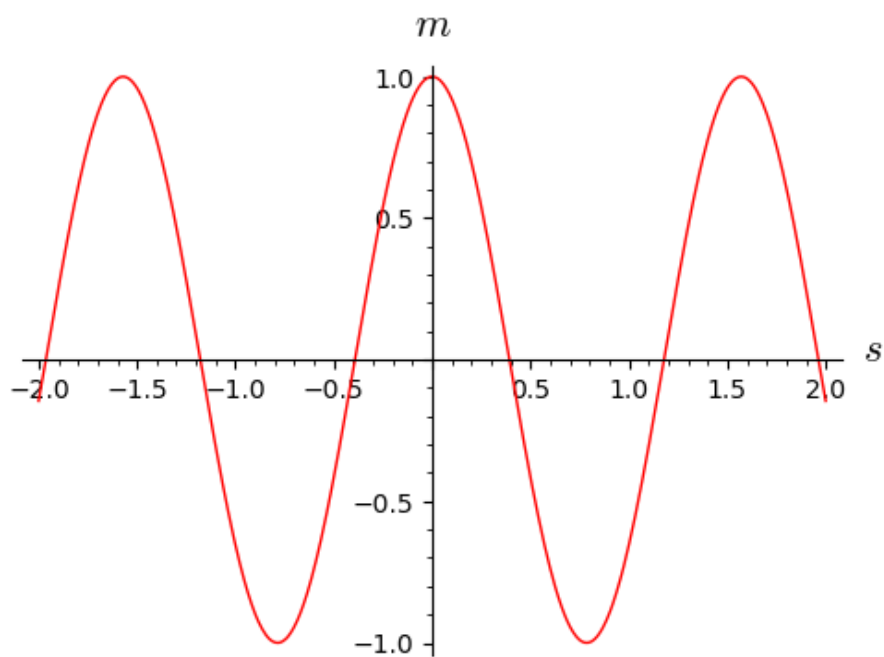
```
d2=plot(cos(4*x),x, -2,2,
        axes_labels=['$s$','$m$'],
        #oznake osi ['os x','os y']
        color='red',
        figsize=5,
        )
d2.show()
```



```
d2.show(axes_labels=['$x$','$y$'])
```



d2

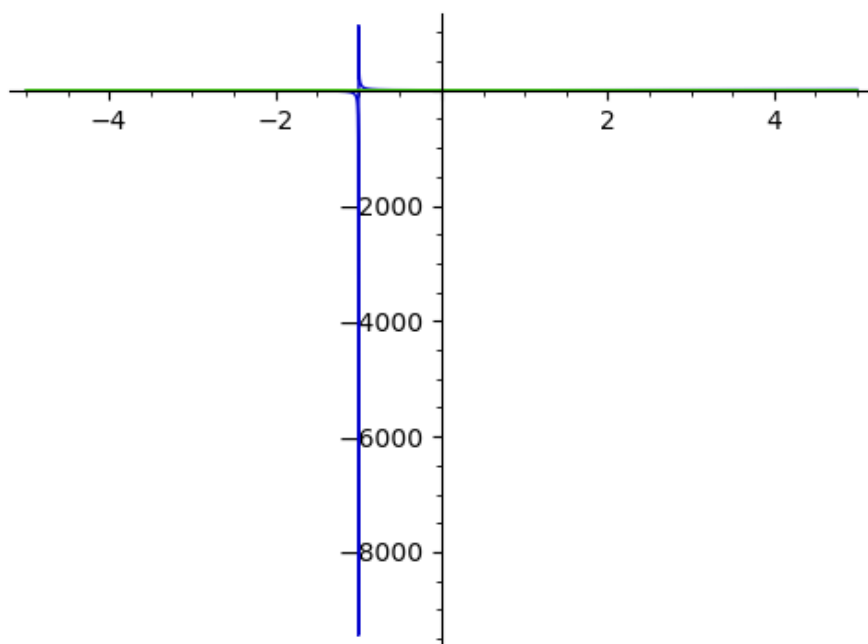


Naj bosta funkciji f in g s predpisom

$$f(x) = \frac{2x^2 + x + 2}{x + 1} \quad g(x) = \frac{\cos(2x)}{2} \quad (31)$$

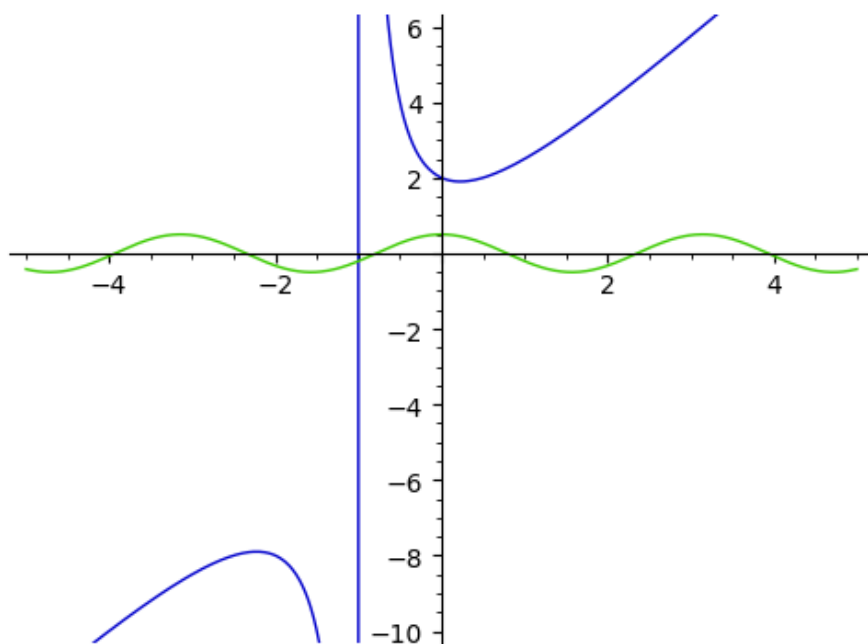
kaj se zgodi, e elimo narisati njuna grafa?

```
f=(2*x^2+x+2)/(x+1)
g=(cos(2*x))/2
plot([f,g], x, -5,5)
```



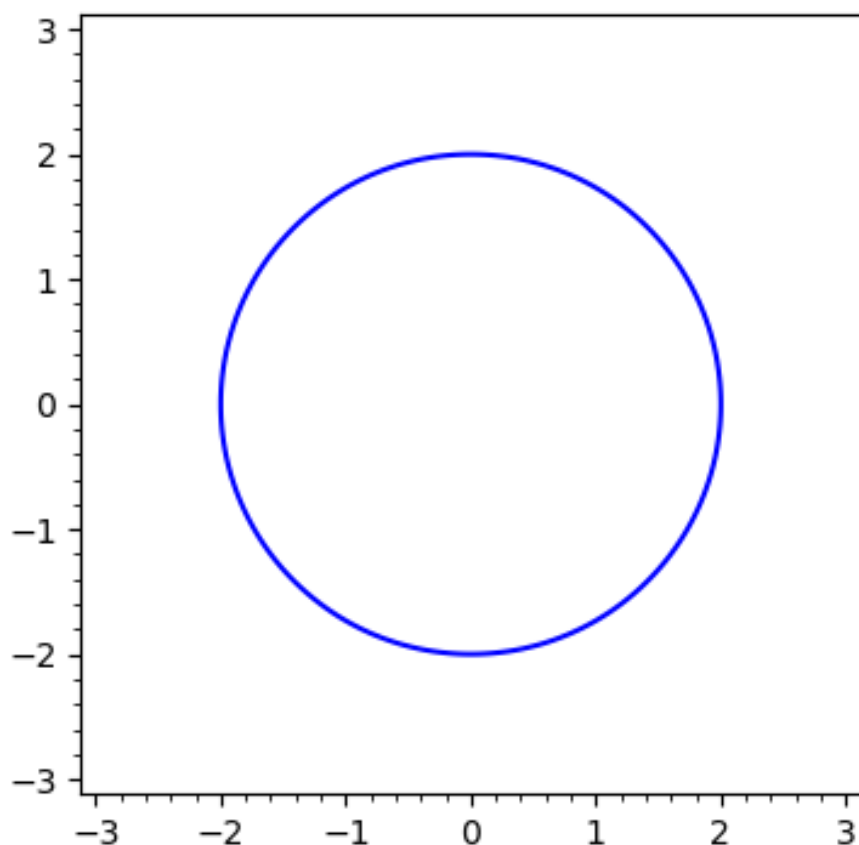
Funkcija g je prikazna slabo, ker je njegovo obmoje interval $[-1, 1]$, medtem ko je obmoje funkcije f veliko veje $(-\infty, \infty)$. Reitev je, da omejimo obmoje grafa z argumentom `ymin` in `ymax`:

```
plot(((2*x^2+x+2)/(x+1)),(cos(2*x))/2),x, -5,5,ymin= -10,
ymax=6)
```



Z ukazom `implici_plot()` lahko nariemo grafe, ki jih določajo implicitne enabe, npr. krog s središčem v izhodišču in polmerom 2:

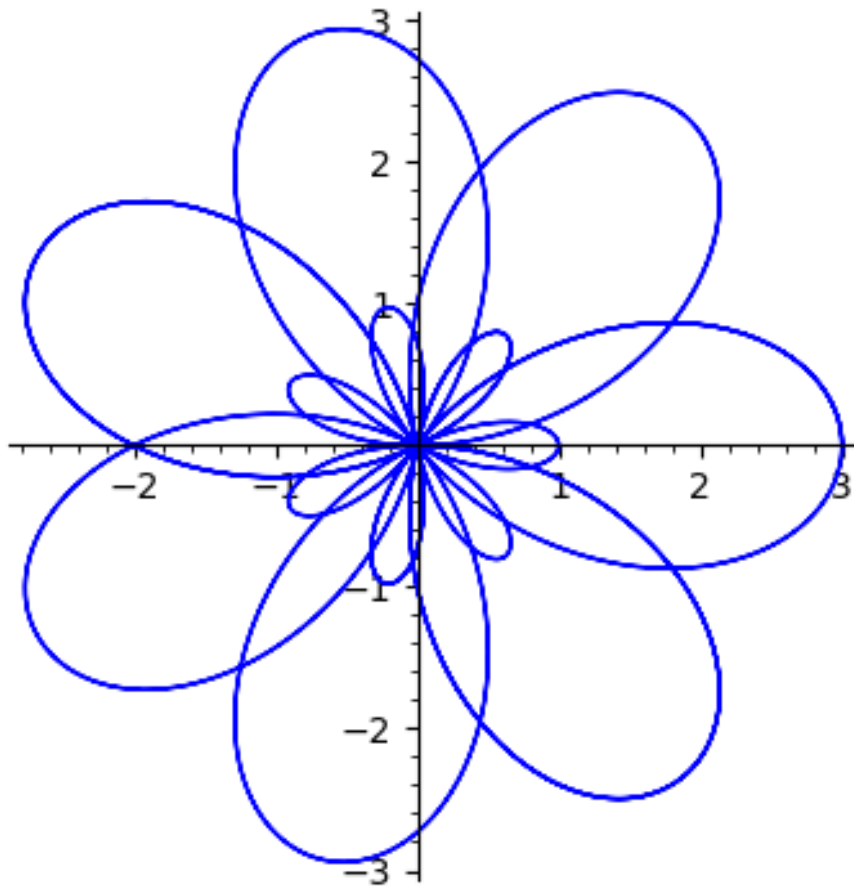
```
x,y=var('x,y')
implicit_plot(x^2+y^2==4, (x, -3, 3), (y, -3,3))
```



Polarne grafe lahko nariemo z ukazom `polar_plot()`. Za funkcijo $r(t)$ v polarnih koordinatah nariemo graf z ukazom `polar_plot(r,t, a, b, plot_points=M)`, kjer t je kot v radianih, interval $[a, b]$ pa določa obseg kota. Parameter `plot_points=M` določatevilo tok, ki se uporabijo za risanje grafa (privzeta vrednost je 100):

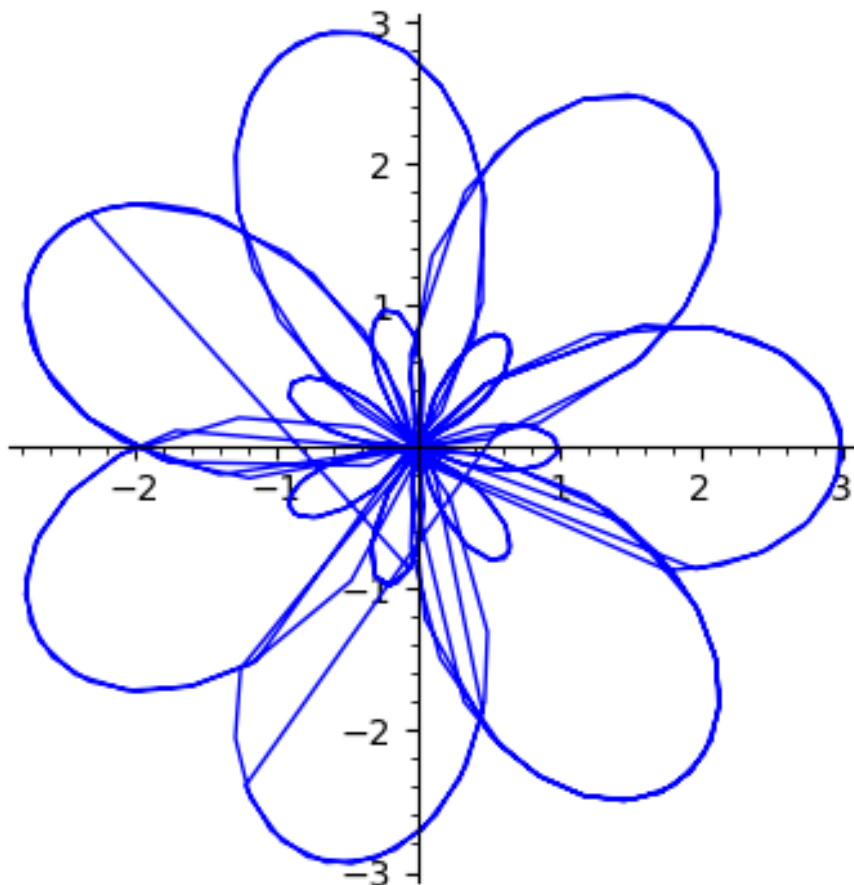
Npr. naslednja koda izpiše družino krivulj, imenovanih krivulje vrtnice, katerih enabe so v obliki $r(t) = 1 + a \cos(kt)$:

```
t=var('t'); a=2; k=7/3
polar_plot(1+a*cos(k*t),t,0,k*6*pi,plot_points=4000)
```



SageMath dejansko ne rie krivulj, temve priblike s rtami, argument `plot_points` doloa, kako dober je pribliek.

```
polar_plot(1+a*cos(k*t),t,0,k*6*pi,plot_points=20)
```



Limite

e je f funkcija in a toka, lahko izračunamo limitno vrednost funkcije f v toki a z ukazom `limit(f, x=a)`:

```
g(x) = x^2 * cos(2*x)
limit(g, x=2)
```

```
x | - -> 4*cos(4)
```

Lahko uporabimo SageMath za izračun nekaterih vrednosti v okolici toke $a = 2$:

```
g(x)=x^2 * cos(2*x)
zacetek =float(2 -1/10) # Zanemo v bližini 2
korak=float(1/100) # Poasi se približujemo 2
st_priblizkov= 9 # t. korakov, da se zelo približamo
for i in range(9): print(g(zacetek+i*korak))
```

```
-2.85539344001104
-2.84031282386950
-2.82329296564705
-2.80430939046852
-2.78333849763835
-2.76035758672103
-2.73534488331575
-2.70827956450633
-2.67914178396811
```

Z desni strani:

```
zacetek = 2 + 1/10
for i in range(9): print(g(zacetek - i * korak))
```

```
-2.16205022211248
-2.21721753287371
-2.27015899052650
-2.32088091383678
-2.36939086604876
-2.41569763530606
-2.45981121451626
-2.50174278067548
-2.54150467367024
```

Izgleda, da bi morala biti limita okoli -2.6. To lahko preverimo s SageMathom:

```
l=limit(g,x=2)
print("l=",l)
print("ki je priblino", l.n())

l= x | - -> 4*cos(4)
ki je priblino -2.61457448345445
```

Funkcija g ni zelo zanimiva, ker je zvezna v toki $a = 2$. To pomeni, da

$$\lim_{x \rightarrow 2} g(x) = g(2) \quad (32)$$

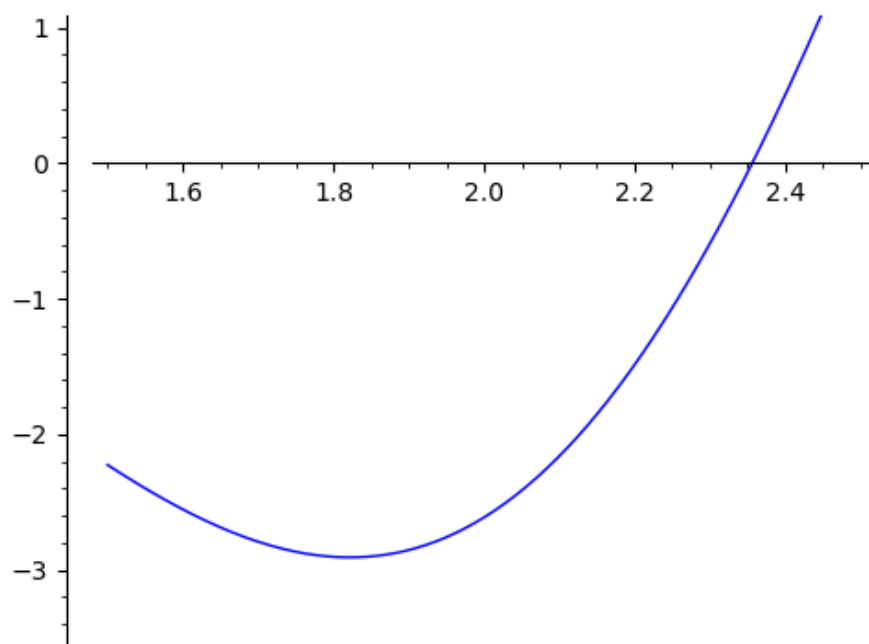
.

```
bool(limit(g,x=2)==g(2))
```

True

To laho vidimo tudi tako, da nariemo graf funkcije g v okolici toke 2:

```
plot(g,x,1.5,2.5,ymin= -3.5, ymax=1)
```



Zdaj si oglejmo funkcijo

$$h(x) = \frac{x^2 + x - 2}{x - 4}. \quad (33)$$

Najdimo

$$\lim_{x \rightarrow 4} h(x). \quad (34)$$

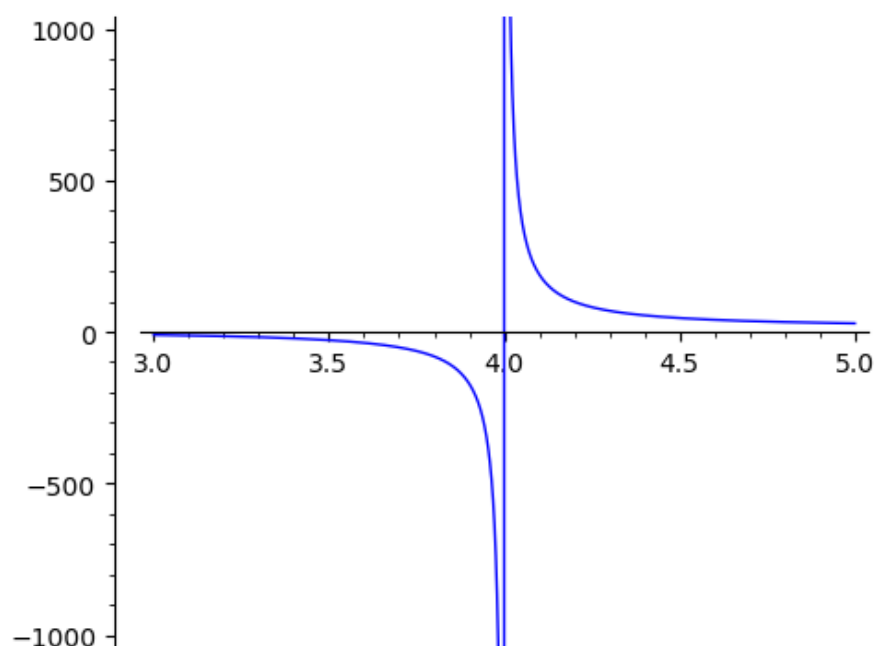
`h(x)=(x^2 + x - 2)/ (x -4)`

`limit(h,x=4)`

`x | - -> Infinity`

Ko pa nariemo graf funkcije h opazimo, da:

`plot(h,x, 3,5,ymax=1000,ymin= -1000)`



$$\lim_{x \rightarrow 4^-} h(x) = -\infty \quad \text{in} \quad \lim_{x \rightarrow 4^+} h(x) = \infty \quad (35)$$

To pomeni, da limita

$$\lim_{x \rightarrow 4} h(x) \quad (36)$$

ne obstaja. Ugotovimo lahko tudi s SageMathom, da

$$\lim_{x \rightarrow 4^-} h(x) = -\infty \quad \text{in} \quad \lim_{x \rightarrow 4^+} h(x) = \infty \quad (37)$$

s ukaza:

`limit(h,x=4, dir='left') #ali dir='minus'`

`x | - -> -Infinity`

`limit(h,x=4, dir='plus') #ali dir='right'`

`x | - -> +Infinity`

Odvod

Odvodi lahko izračunamo z ukazom `diff(f, x)`, kjer je f funkcija in x spremenljivka:

```
f(x) = x*exp(x)
fp=diff(f,x)
fp
```

```
x | - -> x*e^x + e^x
```

Ukaz `derivative()` deluje enako:

```
derivative(f,x)
```

```
x | - -> x*e^x + e^x
```

Prvi argument je funkcija, ki jo elimo odvesti, drugi argument pa spremenljivka glede na katero odvajamo.

```
y=var('y')
p=(x+1)*(y^2+y+1);
print(p)
print(derivative(p,x));
print(derivative(p,y))
```

```
(y^2 + y + 1)*(x + 1)
y^2 + y + 1
(x + 1)*(2*y + 1)
```

Ukaza `diff()` in `derivative()` vrne drugo funkcijo, ki jo lahko ovrednotimo kot katerokoli drugo funkcijo.

```
print(fp)
fp(2)
```

```
x | - -> x*e^x + e^x
```

```
3*e^2
```

```
h(x) = (x^2 + x - 2)/(x - 4)
hp = derivative(h,x)
hp
```

```
x | - -> (2*x + 1)/(x - 4) - (x^2 + x - 2)/(x - 4)^2
```

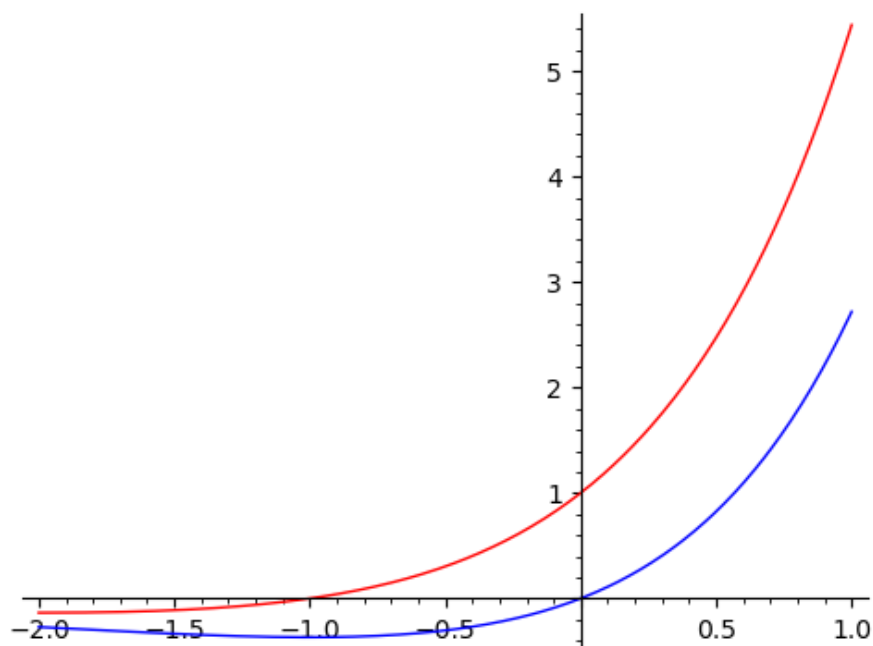
Uporabimo lahko ukaz `solve()` za iskanje kritinih tok funkcije, torej tok, kjer je odvod enak ni:

```
solve( fp(x) == 0,x)
```

```
[x == -1]
```

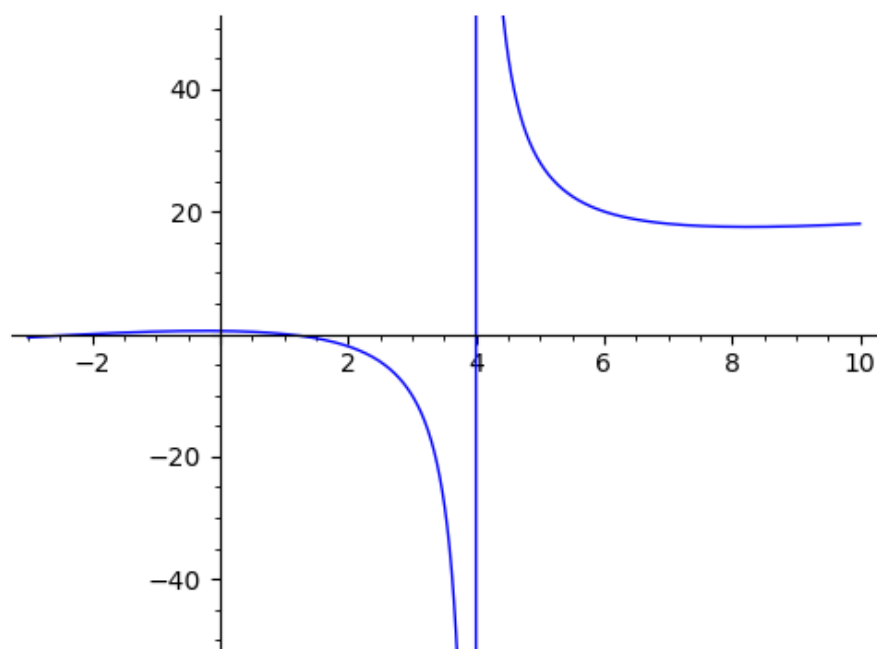
To lahko preverimo z risanjem grafa funkcije in njenega odvoda:

```
g1=plot(f,x, -2,1 )
g2=plot(fp,x, -2,1, color='red' )
g1+g2
```



Za funkcijo h :

```
solve( hp(x) == 0, x)
[x == -3*sqrt(2) + 4, x == 3*sqrt(2) + 4]
plot(h,x, -3,10,ymin= -50, ymax=50)
```

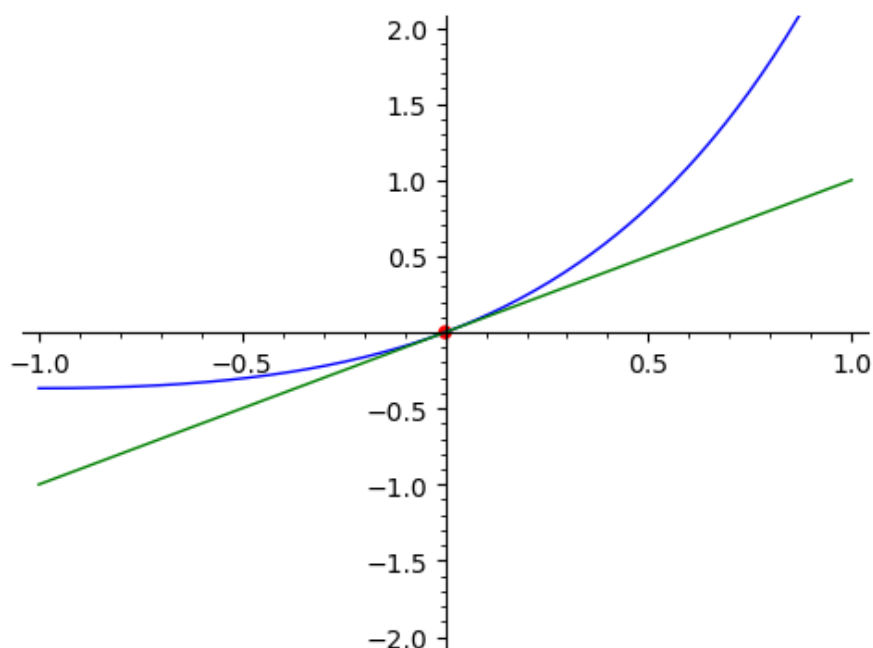


Sestavljamo tangente na nae funkcije v toki $(a, f(a))$. Tangentna rta funkcije $f(a)$ je predstavljena z enabo:

$$y = f(a) + f'(a)(x - a). \quad (38)$$

```
T_f = fp(0)*( x - 0 ) + f(0)
```

```
p1=plot(f,x, -1,1,ymin= -2, ymax=2,color='blue')
p2=plot(T_f,x, -1,1,ymin= -2, ymax=2, color='green' )
p3=point([0,f(0)], color='red', size=30)
p1+p2+p3
```



Integrali

SageMath lahko izrauna določene in nedoločene integrale.

Najprej se bomo osredotoili na nedoločeni integral, ki ga izračunamo z ukazom `integral()`, ki ima podobne argumente kot ukaz `derivative()`.

```
f(x) = x*exp(x)
I_f= integral(f,x) #nedoločena integrala
I_f
```

```
x | - -> (x - 1)*e^x
```

```
h(x) = (x^2 + x - 2)/(x - 4)
I_h= integral(h,x); I_h
```

```
x | - -> 1/2*x^2 + 5*x + 18*log(x - 4)
```

Rezultat je funkcija, ki je le ena od *primitivnih funkcij*. Ostale se razlikujejo po konstanti. Da je to pravilno, lahko preverimo tako, da izračunamo odvod.

```
print(f); derivative(I_f,x)
```

```
x | - -> x*e^x
```

```
x | - -> (x - 1)*e^x + e^x
```

```
print(h); derivative(I_h,x)
```

```
x | - -> (x^2 + x - 2)/(x - 4)
```

```
x | - -> x + 18/(x - 4) + 5
```

Nista videti povsem enaka, vendar lahko to preverimo z malo algebre... ali...

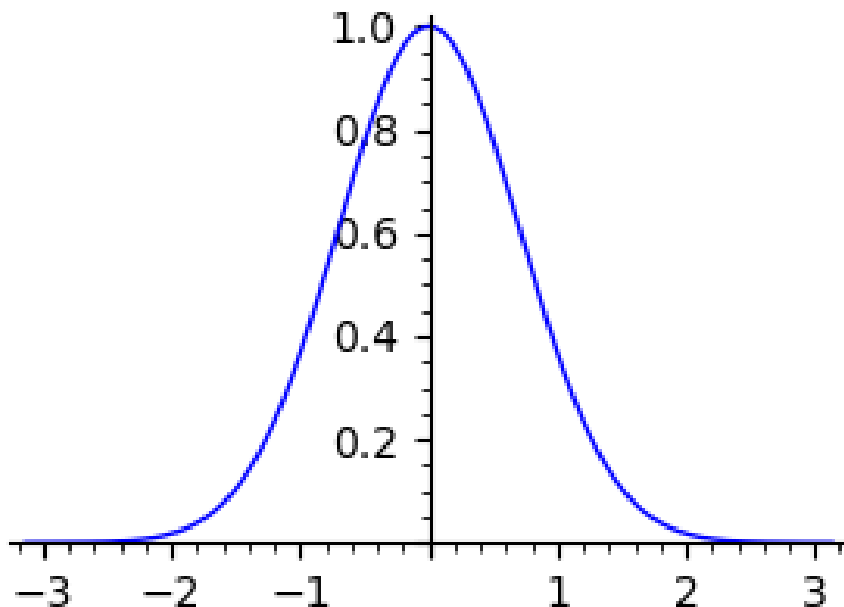
```
print( bool(f==derivative(I_f,x)),
        bool(h==derivative(I_h,x)) )
```

True True

Obstaja nekaj funkcij, ki so videti nedolne, vendar njihovi primitivi nimajo *zaprte oblike*.

Klasini primer je funkcija e^{-x^2} , ki je temeljna v verjetnosti.

```
plot(exp( -x^(2)),x, -pi,pi, figsize=3)
```



```
integral(exp( -x^(2)),x)
```

```
1/2*sqrt(pi)*erf(x)
```

Rezultat vkljuuje funkcijo $\text{erf}(x)$, ki se sicer imenuje *funkcija napake* (ang. *error function*), ki nima preprostega izraza.

Za izraun doloenega integrala moramo ukaz `integral()` podati integracijski limiti.

```
integral(f, x,0,1)
```

```
1
```

```
integral(h, x,0,1)
```

```
18*log(3) - 36*log(2) + 11/2
```

V vsakem primeru je rezultat tevilko (po priakovanjih).

Kot smo e omenili, bo SageMath vedno dajal prednost simbolnim izraunom, e elimo dobiti numerine priblike, lahko uporabimo funkcijo `n()`.

```
n(integral(f, x,0,1))
```

```
1.0000000000000000
```

```
n(integral(h, x,0,1))
```

```
0.321722695867944
```

Funkcija e^{-x^2} je pomembna v verjetnosti, ker predstavlja gostoto normalne porazdelitve. Celoten integral od $-\infty$ do ∞ (ker je to celotna verjetnost). Preverimo to s SageMathom:

```
integral(exp( -x^(2)),x, -infinity, infinity)
sqrt(pi)
n(integral(exp( -x^(2)),x, -infinity, infinity))
1.77245385090552
```

To je

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} \approx 1.77245385090552. \quad (39)$$

0.4 Priloge

0.4.1 Nameanje LaTeXa na svoj računalnik

LaTeX je popolnoma brezplačen in ga lahko enostavno namestite na svoj računalnik. Uradno spletno stran za namestitev si lahko ogledate tukaj (v angleščini).

Za uporabo LaTeXa potrebujete v bistvu dve orodji:

- distribucijo LaTeX, ki je sam program LaTeX, s katerim boste ustvarjali svoje dokumente.
- urejevalnik besedil, ki je vmesnik, ki ga boste uporabljali za pisanje in izdelavo dokumentov. Verjetno imate urejevalnik besedil na svojem računalniku (npr. Notepad ali TextEdit). Ti urejevalniki so venamenski in imajo običajno zelo malo orodij. Na tej strani priporočamo nekaj monosti, ki vam bodo olajale delo.

Windows

Namestitev distribucije

1. Odprite spletno stran MiKTeX.
2. Prenesite namestitveni program za Windows.
3. Zaenite namestitev in sledite navodilom (pustite privzete nastavitve).
4. Po zaključku preverite namestitev tako, da v ukazni vrstici (Command Prompt) vpiete:

```
pdflatex - -version
```

Priporočeni urejevalniki

- Visual Studio Code (potrebujete tudi raziritev *LaTeX Workshop*). e boljše, predlagamo, da namestite VSCodium, brezplačno/odprtokodno različico vscode.
- TeXworks (preprost, pogosto nameen z MiKTeX).
- TeXstudio (bolj napreden, uporabniku prijazen).

macOS

Namestitev distribucije

1. Odprite spletno stran MacTeX.
2. Prenesite paket **MacTeX.pkg** (velik ~4 GB).
3. Odprite preneseno datoteko in sledite navodilom za namestitev.
4. Preverite namestitev v Terminalu:

```
pdflatex - -version
```

Priporočeni urejevalniki

- Visual Studio Code (raziritev *LaTeX Workshop*). e boljše, predlagamo, da namestite VSCodium, brezplačno/odprtokodno različico vscode.
- TeXShop (priložen MacTeX-u).
- TeXstudio.

Linux (primer: Ubuntu/Debian)

e uporabljate Linux, verjetno ne potrebujete naših navodil.

Namestitev distribucije

1. Odprite Terminal.
2. Namestite TeX Live:

```
sudo apt update  
sudo apt install texlive -full
```

3. Preverite namestitev:

```
pdflatex - -version
```

Priporoeni urejevalniki

- Visual Studio Code. (raziritev *LaTeX Workshop*). e bolje, predlagamo, da namestite VSCodium, brezplačno/odprtokodno različico vscode.
- Kile (posebej priljubljen v Linux okolju).
- TeXstudio.
- TeXworks.

0.4.2 Izvoz datotek iz Overleaf projekta

e elite izvoziti celoten projekt iz Overleafa kot `.zip` ali `pdf` datoteko, sledite naslednjim korakom:

1. **Odprite projekt** v Overleafu, iz katerega elite prenesti datoteke.
2. V zgornjem levem kotu kliknite na gumb **Menu**.
3. V spustnem meniju izberite monost **Download >Source** (e elite `.zip` datoteko) ali **Download >PDF** (e elite `.pdf` dokument).
4. Overleaf bo ustvaril in prenesel `.zip` datoteko, ki vsebuje vse datoteke projekta (npr. `.tex` datoteke, slike, `.bib` datoteke, ipd.) oziroma `.pdf` dokument.
5. Shrani preneseno datoteko na svoj računalnik in jo po potrebi raziri.

0.4.3 Doline v LaTeXu

Doline (*lengths*) so pomemben del LaTeXa, saj omogoajo natanen nadzor nad razmikom, velikostjo elementov, zamiki, robovi in e ve. Vsaka dolina je shranjena kot spremenljivka, ki ji lahko dodelimo vrednost, jo poveamo ali zmanjamo, in uporabimo pri razlinih ukazih.

Osnovne merske enote

V LaTeXu lahko doline izrazimo z razlinimi enotami. Najpogosteje uporabljene so:

Enota	Opis
pt	toka (1/72,27 palca) – osnovna enota v TeXu
mm	milimeter
cm	centimeter
in	palec (2.54 cm)
ex	viina male rke x v trenutni pisavi
em	irina rke M v trenutni pisavi
bp	velika toka (1/72 palca)
pc	pica = 12 tok
dd	didotova toka
cc	cicer = 12 didotov
sp	najmanja notranja enota v TeXu (65536 sp = 1 pt)

Privzete dolinske spremenljivke

LaTeX ima ve e vnaprej doloenih dolin, ki vplivajo na izgled dokumenta:

Dolina	Opis
<code>\textwidth</code>	irina glavnega besedila
<code>\textheight</code>	viina glavnega besedila
<code>\linewidth</code>	irina vrstice znotraj okolja (npr. v <code>minipage</code>)
<code>\columnwidth</code>	irina stolpca v dvostolpnem nainu
<code>\paperwidth, \paperheight</code>	dimenzije papirja
<code>\parindent</code>	zamik prve vrstice odstavka
<code>\parskip</code>	razmik med odstavki
<code>\baselineskip</code>	navpina razdalja med vrsticami besedila
<code>\topmargin, \oddsidemargin,</code>	robovi strani
<code>\evensidemargin</code>	